



รายงาน Lab 2 Controller Design
รายวิชา FRA233: Control Engineering for Robotics

จัดทำโดย
นักศึกษากลุ่มที่ 4

นายจักพรรณ	ศรียอดบาง	รหัสนักศึกษา 67340500005
นางสาวญาณิศา	รอดเจริญ	รหัสนักศึกษา 67340500009
นายณัฐพงศ์	หวังจิ	รหัสนักศึกษา 67340500010
นายพีรกานต์	ยู	รหัสนักศึกษา 67340500031
นายสงกรานต์	ปราณีบุตร	รหัสนักศึกษา 67340500042
นายสมัชญ์	ช้อยศิริณ	รหัสนักศึกษา 67340500043

สถาบันวิทยาการหุ่นยนต์ภาคสนาม มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี

สารบัญ

เรื่อง	หน้า
สารบัญ	ก
สารบัญรูปภาพ	ค
สารบัญตาราง	ง
คำอธิบายสัญลักษณ์และคำย่อ	จ
DC Motor Parameters and Mathematical Modeling	ฉ
System Parameters	ฉ
DC Motor Mathematical Modeling	ช
DC Motor with Vertical Pendulum Mathematical Modeling	ฌ
Simulink Single Loop Control Setup	ญ
Simulink Cascade Loop Control Setup	ฎ
Lab 2 Controller Design	1
การทดลองที่ 1 Hardware In The loop	1
การทดลองที่ 2 การศึกษาผลกระทบจาก Disturbance ที่เกิดจากแรงโน้มถ่วง	21
การทดลองที่ 3 การศึกษาพฤติกรรมของค่าคงที่ K_p ในระบบ P Controller	29
การทดลองที่ 4 การศึกษาพฤติกรรมของค่าคงที่ K_I ในระบบ PI Controller	36
การทดลองที่ 5 การศึกษาพฤติกรรมของค่าคงที่ K_D ในระบบ PD Controller	42
การทดลองที่ 6 การศึกษาพฤติกรรมของระบบใน Continuous Domain และ Discrete Domain	49

การทดลองที่ 7 การศึกษาผลกระทบต่อระบบ Cascade Control จาก Transition Rate ความถี่ของ Inner Loop และ outer Loop และ ระบบที่เปลี่ยนแปลงไปจากตำแหน่งการ วาง Saturation และ Transition Block ภายใน Cascade Loop Control	55
Labs 2 Tuning Requirements Section	63
Requirement 1	63
Requirement 2	65
Requirement 3	67
Labs 2 Discussion about Tools and Hand Calculation	Error! Bookmark not defined.
Labs 2 Discussion about Testing Tools	Error! Bookmark not defined.
ภาคผนวก ก ขั้นตอนการคำนวณโดยละเอียด	68
ภาคผนวก ข อื่น ๆ	Error! Bookmark not defined.

สารบัญรูปภาพ

เรื่อง

หน้า

No table of figures entries found.

สารบัญตาราง

เรื่อง

หน้า

No table of figures entries found.

คำอธิบายสัญลักษณ์และคำย่อ

สัญลักษณ์	คำอธิบาย
A	แอมแปร์ (Ampere)
H	เฮนรี (Henry)
I	กระแสไฟฟ้า [A]
I_m	กระแสไฟฟ้าที่มอเตอร์ [A]
I_{sh}	กระแสไฟฟ้าที่ R-Shunt [A]
L_m	ค่าความเหนี่ยวนำมอเตอร์ [H]
R_m	ค่าความต้านทานไฟฟ้ามอเตอร์ [Ω]
R_{sh}	ค่าความต้านทานไฟฟ้า R-Shunt [Ω]
R-Shunt	ตัวต้านทานไฟฟ้าชนิด
s	วินาที (Second) [s]
τ	ค่าคงที่เวลา (Tau) [s]
V	โวลต์ (Volt) [V]
V_{in}	แรงดันไฟฟ้าขาเข้า (Input) [V]
V_m	แรงดันไฟฟ้าตกคร่อมมอเตอร์ (Motor) [V]
V_{max}	แรงดันไฟฟ้าสูงสุด (Maximum) [V]
V_{sh}	แรงดันไฟฟ้าตกคร่อม R-Shunt [V]
V_τ	แรงดันไฟฟ้าที่ค่าคงที่เวลา [V]
Ω	โอห์ม (Ohm) [Ω]
μ	ประสิทธิภาพ [%]
K_e	ค่าคงที่ Back EMF [$\frac{V}{rad/s}$]
B	ค่าความเสียดทานหนืด [$Nm \cdot s$]
J	Moment of Inertia [$Kg \cdot m^2$]

DC Motor Parameters and Mathematical Modeling

ใน Labs ที่ 2 Controller Design แบ่งเป็น 2 ส่วนหลัก ได้แก่ การคำนวณสัญญาณการควบคุมผ่าน Hardware หรือ Hardware-In-Loop (HIL) โดยการทำให้ Discretization และ การควบคุมระบบ Vertical Pendulum โดยใช้ DC Motor เป็นตัวส่งกำลัง (Plant) เพื่อควบคุมตำแหน่งตาม Requirement ผ่านการออกแบบตัวควบคุมทั้งในรูปแบบ PID Controller, Disturbance Feed-forward รวมถึงโครงสร้างแบบ Single Loop และ Cascade Loop

เพื่อความสม่ำเสมอในการวิเคราะห์และออกแบบ ผู้จัดทำจึงได้กำหนดพารามิเตอร์ของมอเตอร์ (System Parameters) และแบบจำลองทางคณิตศาสตร์ (Mathematical Model) ไว้ในส่วนนี้ เพื่อใช้เป็นข้อมูลอ้างอิงสำหรับการทดลองย่อยทั้งหมดที่เกิดขึ้นในรายงานฉบับนี้

System Parameters

ตารางที่ 1 DC Motor Parameters

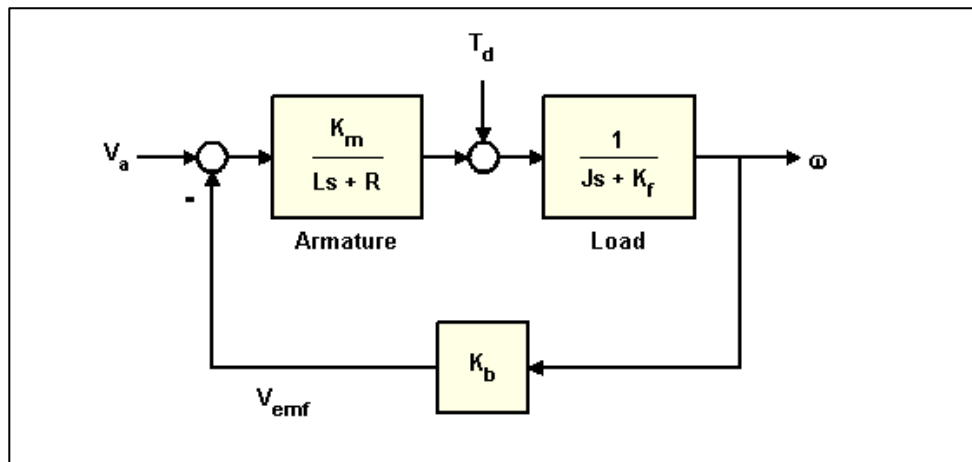
สัญลักษณ์	หน่วย	ค่าจากการทดลอง (Experimental)	ค่าจาก Datasheet (Datasheet)
K_m	$[N \cdot m/A]$	50.00×10^{-3}	50.6000×10^{-3}
K_e	$[V \cdot s/rad]$	50.10×10^{-3}	52.8000×10^{-3}
L_m	$[H]$	2.8530×10^{-3}	2.8544×10^{-3}
R_m	$[\Omega]$	3.3999	3.3133
B_m	$[N \cdot s/M]$	0.6590×10^{-6}	77.8510×10^{-6}
J_m	$[kg \cdot m^2]$	11.700×10^{-6}	58.5590×10^{-6}

ตารางที่ 2 Pendulum Parameters

สัญลักษณ์	หน่วย	ค่าที่กำหนดมา (Specify Value)
M_p	$[kg]$	0.05
L_p	$[m]$	0.1
$J_p = M_p \cdot L_p^2$	$[kg \cdot m^2]$	500.00×10^{-6}

DC Motor Mathematical Modeling

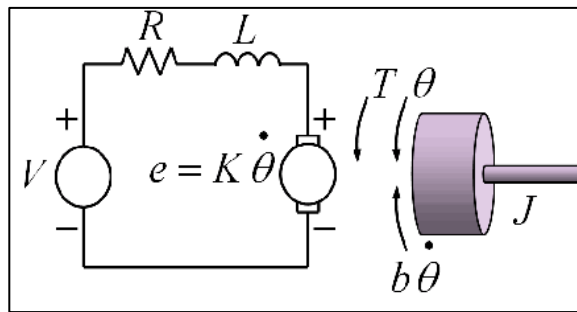
1. Block Diagram



รูปที่ 1 ภาพ Block Diagram ของระบบ DC Motor

จากภาพรวมระบบของ Lab 2 Controller Design นั้นจะเป็นการควบคุมระบบ Vertical Pendulum โดยใช้ DC Motor เป็นตัวส่งกำลัง (Plant) ดังนั้น System นี้จะเป็น DC Motor System ส่ง Torque เข้าสู่ Vertical Pendulum จึงได้ System Diagram ดังรูปที่ 1

2. Free Body Diagram และ Circuit Diagram



รูปที่ 2 การ Free Body Diagram และ Circuit Diagram

ในการทำ Modelling ของ DC Motor รวมกับ Vertical Pendulum นั้น จะเป็น Modelling ทั้ง Mechanical Term และ Electrical Term โดยที่ Mechanical Term จะมี Term Moment of Inertia และ Torque Load ที่เกิดขึ้นจากมวลของ Vertical Pendulum

3. Equation of Motion

$$\text{Mechanical Term} \quad \Sigma T_m(t) = J_m \ddot{\theta}(t) + b_m \dot{\theta}(t) \quad (1.1)$$

$$\text{Electrical Term} \quad V_{in}(t) = L_m \dot{I}(t) + R_m I(t) + K_e \dot{\theta}(t) \quad (1.2)$$

$$\text{Coupling Term} \quad T_m = K_m I(t) \quad (1.3)$$

4. Transfer Function

$$\text{Coupling Term} \quad T_m(s) = K_m I(s) \quad (1.4)$$

$$T_m(s) = [J_m s + b_m] \omega(s)$$

$$K_m I(s) = [J_m s + b_m] \omega(s)$$

$$\text{Mechanical Term} \quad I(s) = \frac{[J_m s + b_m] \omega(s)}{K_m} \quad (1.5)$$

$$\text{Electrical Term} \quad V_{in}(s) = (L_m s + R_m) I(s) + K_e \omega(s) \quad (1.6)$$

แทนสมการ (1.4) เข้าสมการ (1.5) สำหรับ Mechanical Term และ แทนสมการ (1.5) เข้าสมการ (1.6) สมการ Electrical Term จึงได้สมการ (1.7) โดยสมการ Transfer Function นี้จะถูกใช้ต่อภายใน Labs และ หากอยากได้ Transfer Function ของ Angular Position สามารถเติม Integrator $\left(\frac{1}{s}\right)$ อีก 1 ครั้ง เป็นสมการที่ (1.8)

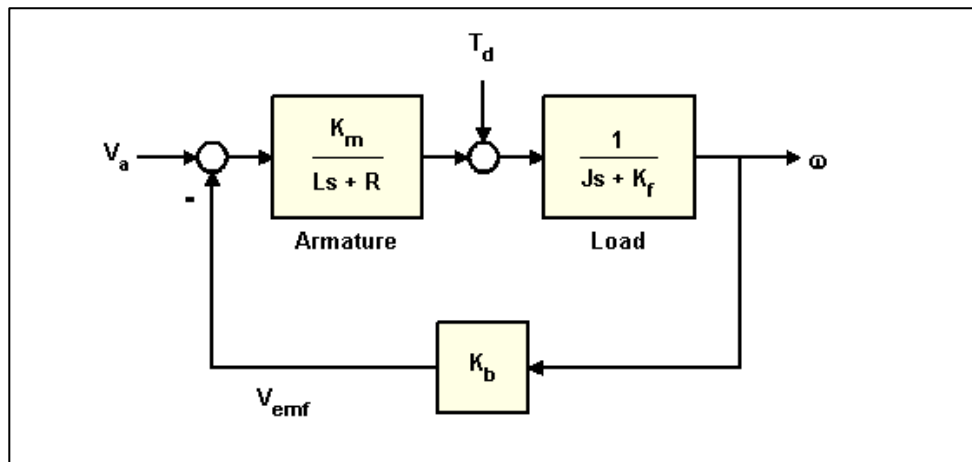
$$V_{in}(s) = (L_m s + R_m) \left[\frac{[J_m s + b_m] \omega(s)}{K_m} \right] + K_e \omega(s)$$

$$\frac{\omega(s)}{V_{in}(s)} = \frac{K_m}{(L_m s + R_m)(J_m s + b_m) + K_e K_m} \quad (1.7)$$

$$\frac{\theta(s)}{V_{in}(s)} = \frac{1}{s} \left[\frac{\omega(s)}{V_{in}(s)} \right] = \frac{K_m}{s(L_m s + R_m)(J_m s + b_m) + K_e K_m} \quad (1.8)$$

DC Motor with Vertical Pendulum Mathematical Modeling

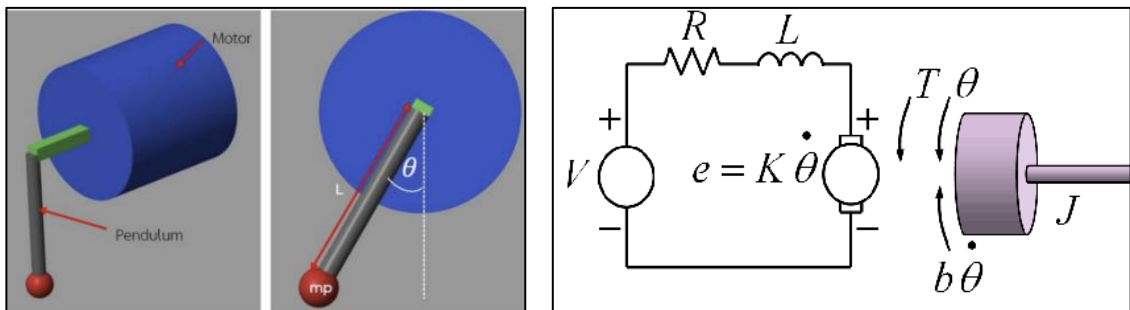
1. System Diagram



รูปที่ 3 ภาพ Block Diagram ของระบบโดยไม่มี DFF และ Controller

จากภาพรวมระบบของ Lab 2 Controller Design นั้นจะเป็นการควบคุมระบบ Vertical Pendulum โดยใช้ DC Motor เป็นตัวส่งกำลัง (Plant) ดังนั้น System นี้จะเป็น DC Motor System ส่ง Torque เข้าสู่ Vertical Pendulum จึงได้ System Diagram ดังรูปที่ 1

2. Free Body Diagram และ Circuit Diagram



รูปที่ 4 การ Free Body Diagram Vertical Pendulum และ Circuit Diagram

ในการทำ Modelling ของ DC Motor รวมกับ Vertical Pendulum นั้น จะเป็น Modelling ทั้ง Mechanical Term และ Electrical Term โดยที่ Mechanical Term จะมี Term Moment of Inertia และ Torque Load ที่เกิดขึ้นจากมวลของ Vertical Pendulum

3. Equation of Motion

$$\Sigma T_{all}(t) = J_{all}\ddot{\theta}(t) - b_m\dot{\theta}(t)$$

$$J_{all}\ddot{\theta}(t) = T_{all}(t) - b_m\dot{\theta}(t)$$

$$(J_m + M_p L_p^2)\ddot{\theta}(t) = T_m(t) - M_p L \sin \theta(t) - b_m\dot{\theta}(t)$$

$$\text{Mechanical Term} \quad T_m(t) = (J_m + M_p L_p^2)\ddot{\theta}(t) + M_p L \sin \theta(t) + b_m\dot{\theta}(t) \quad (1.9)$$

$$\text{Electrical Term} \quad V_{in}(t) = L_m \dot{I}(t) + R_m I(t) + K_e \dot{\theta}(t) \quad (1.10)$$

$$\text{Coupling Term} \quad T_m = K_m I(t) \quad (1.11)$$

4. Transfer Function

$$\text{Let } \sin \theta \approx \theta$$

$$\text{Mechanical Term} \quad T_m(s) = [(J_m + M_p L^2)s^2 + b_m s + M_p g L]\theta(s) \quad (1.12)$$

$$V_{in}(s) = (L_m s + R_m)I(s) + K_e s\theta(s)$$

$$\text{Electrical Term} \quad I(s) = \frac{V_{in}(s) - K_e s\theta(s)}{(L_m s + R_m)} \quad (1.13)$$

$$\text{Coupling Term} \quad T_m(s) = K_m I(s) \quad (1.14)$$

ใช้สมการที่ (1.8) เปลี่ยนเป็น S-domain ได้สมการที่ (1.11) เป็น Mechanical Terms ใช้สมการ (1.9) เปลี่ยนเป็น S-domain และจัดรูปดังสมการ (1.12) เป็น Electrical Terms แทนสมการ (1.11) และ (1.12) ใส่สมการ (1.13) เพื่อทำมาจัดรูปเป็น Transfer Function ต่อ ซึ่งจะเป็น Transfer Function ที่ใช้ต่อภายใน Labs

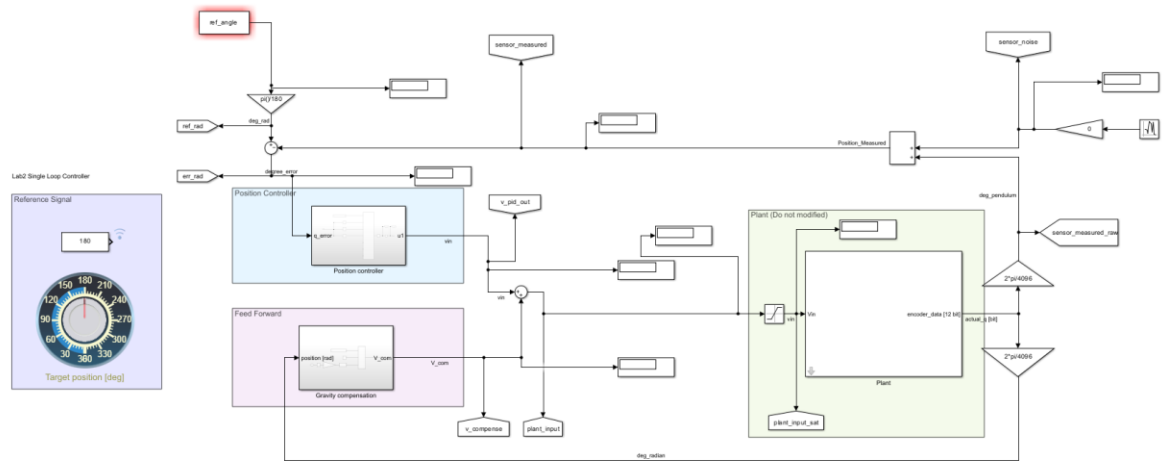
$$[(J_m + M_p L^2)s^2 + b_m s + M_p g L]\theta(s) = K_m \left[\frac{V_{in}(s) - K_e s\theta(s)}{(L_m s + R_m)} \right]$$

$$V_{in}(s)K_m = (L_m s + R_m)[(J_m + M_p L^2)s^2 + b_m s + M_p g L]\theta(s) + K_m K_e s\theta(s)$$

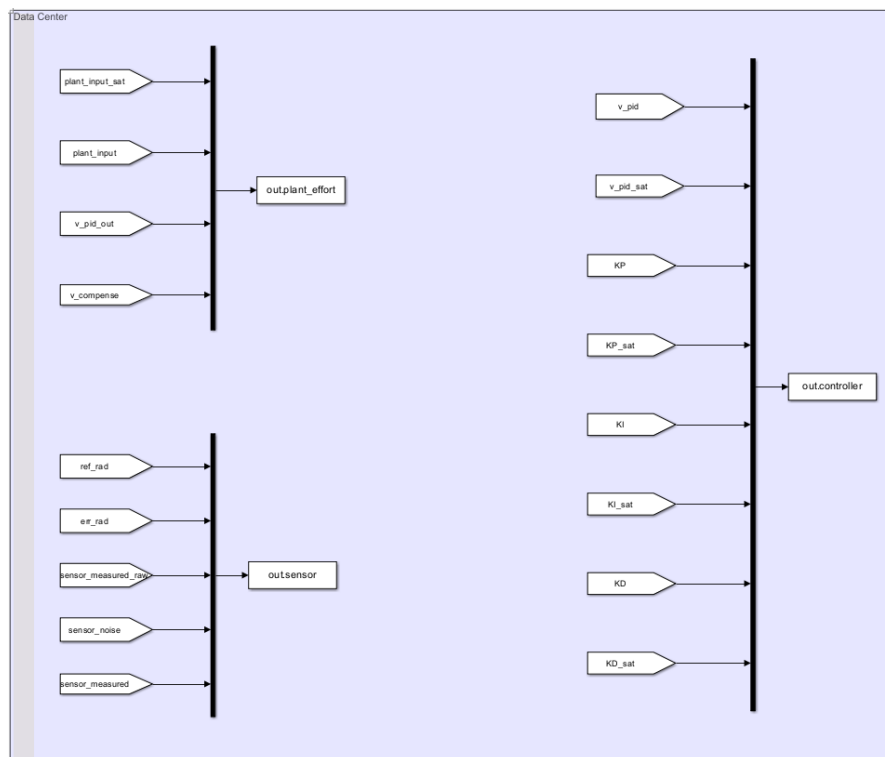
$$\frac{\theta(s)}{V_{in}(s)} = \frac{K_m}{(L_m s + R_m)[(J_m + M_p L^2)s^2 + b_m s + M_p g L] + K_m K_e s} \quad (1.15)$$

Simulink Single Loop Control Setup

1. Block Diagram in Simulink Setup

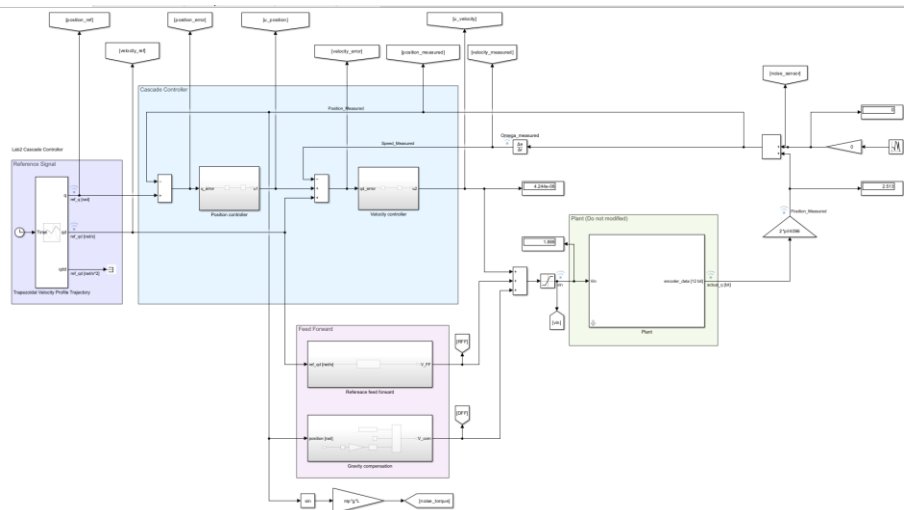


2. Output Parameter

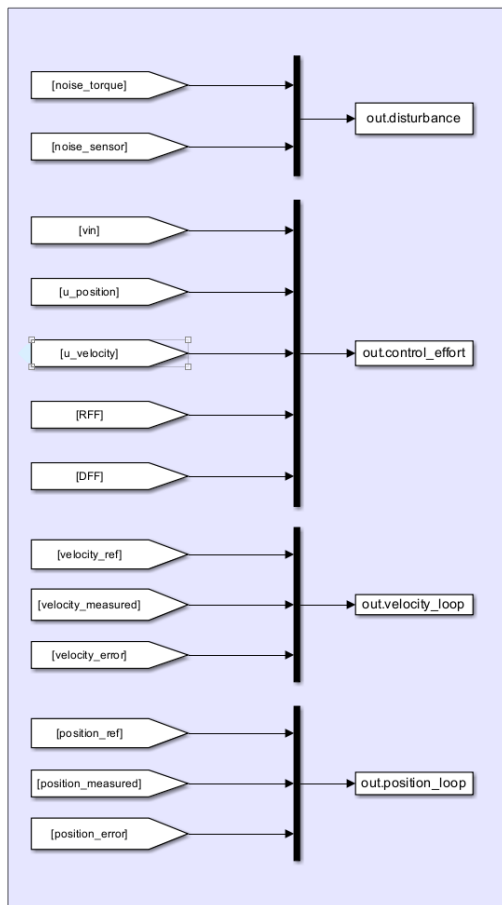


Simulink Cascade Loop Control Setup

1. Block Diagram Simulink Setup



2. Output Parameter



Lab 2 Controller Design

การทดลองที่ 1 Hardware In The loop

จุดประสงค์

1. เพื่อศึกษาการ Discretize สมการ Second Order Transfer Function ของ DC Motor
2. เพื่อศึกษาผลลัพธ์มอเตอร์ที่ได้จากการทำ Hardware in the loop
3. เพื่อเปรียบเทียบพฤติกรรมมอเตอร์ที่มาจากการจำลองและความเร็วมอเตอร์จริง
4. เพื่อศึกษาสาเหตุและปัจจัยที่ส่งผลต่อลักษณะของสัญญาณจำลอง

สมมติฐาน

สมการ Second Order Transfer Function ที่ผ่านการทำ Estimation ประเภทต่าง ๆ ทำให้ได้สัญญาณจำลองที่ต่างกัน มีความเสถียรและความแม่นยำที่ต่างกัน ซึ่งเมื่อเปรียบเทียบกับสัญญาณของความเร็วมอเตอร์จริง จะเกิดความแตกต่างอันเนื่องมาจากคุณลักษณะของมอเตอร์และข้อจำกัดจากการจำลอง และเพื่อความสะดวกในการจำลองพฤติกรรมของมอเตอร์จึงควรเลือกใช้การ Discretize ที่เหมาะสม มีความถูกต้องและแม่นยำที่สุด ซึ่งคือ Trapezoidal Estimation

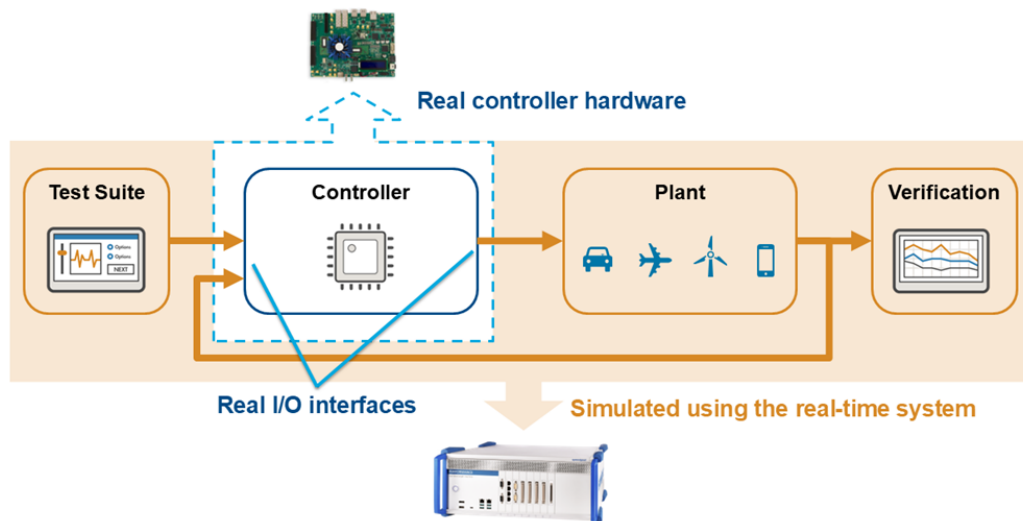
ตัวแปร

1. ตัวแปรต้น:
 - รูปแบบสัญญาณ Input ได้แก่ Sin Wave 1 Hz ในช่วง -12 ถึง 12 V และ Ramp slope 1
 - รูปแบบการ Discretize สมการ ได้แก่ Forward Estimation, Backward Estimation และ Trapezoidal Estimation
 - ค่า Sampling Time [T_s]
2. ตัวแปรตาม:
 - ลักษณะสัญญาณจำลองมอเตอร์
 - ค่าความคลาดเคลื่อนของสัญญาณ (RMSE)
3. ตัวแปรควบคุม:
 - สมการ Second Order Transfer Function
 - การตั้งค่า IOC และการเขียนโปรแกรมบน STM32CubeIDE
 - เก็บค่าเป็นเวลา 20 s.

เอกสารและงานวิจัยที่เกี่ยวข้อง

1. Hardware In Loop (HIL)

คือ วิธีการจำลองระบบเพื่อพัฒนาและทดสอบการทำงาน ซึ่งมาจากการเชื่อมโยงค่า Input และ Output ที่เกิดขึ้นในสภาพแวดล้อมจำลองที่มีหลักฟิสิกส์เป็นพื้นฐาน มักใช้ในการยืนยันความเชื่อมต่อกันของระบบ Hardware และระบบ Software โดยเชื่อมต่อ Controller ที่นำมาควบคุมระบบ เข้ากับ Plant หรือระบบที่ถูกจำลองขึ้นมา ที่สื่อสารกันผ่าน Protocols ทำให้สามารถประเมินความสามารถและข้อจำกัดของระบบ



รูปที่ ? Hardware In Loop Simulation

ที่มา: <https://www.mathworks.com/discovery/hardware-in-the-loop-hil.html>

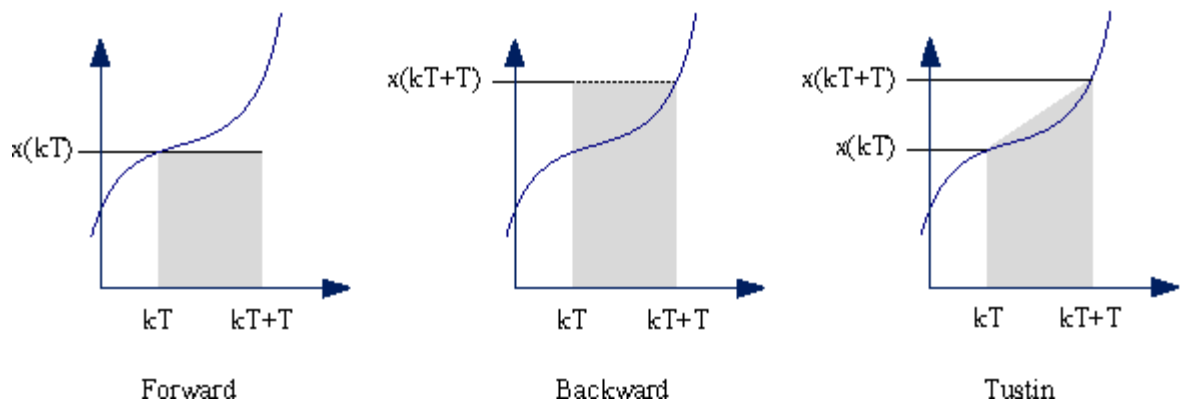
ส่วนประกอบในการจำลอง HIL ได้แก่

- Model ทางคณิตศาสตร์ของระบบ หรือ Plant ซึ่งใช้ในการบอกความสัมพันธ์ของระบบทั้งหมด ความเชื่อมโยงกันของระบบในทางกลและทางไฟฟ้า
- การจำลองแบบ Real-Time บน Software เนื่องจากความสัมพันธ์ของ Model นั้นขึ้นอยู่กับเวลา ระบบจึงจะสามารถตอบสนองได้ตรงกับสภาพแวดล้อมจริง
- Controller เป็นตัวกลางที่ใช้ในการสื่อสารกับ Plant และประมวลผลการตอบสนองให้ใกล้เคียงกับการควบคุมในสภาพแวดล้อมจริง
- การรับและส่งกลับข้อมูลกับ Controller และ Plant ใช้ในการลด Error ที่อ้างอิงมาจากตอบสนองของ Controller เอง เพื่อให้ระบบมีความแม่นยำขึ้น
- การทดสอบซ้ำเพื่อยืนยันความถูกต้องและเสถียรภาพของระบบจำลอง

2. Discretization

เป็นกระบวนการที่เปลี่ยน Transfer Function ในรูปแบบ Continuous System หรือ s-domain (Laplace) เป็น Discrete System หรือ z-domain (discrete) ซึ่งทำให้สามารถนำไปใช้ในการควบคุมระบบ ผ่านคอมพิวเตอร์และ Controller ได้ แต่ในการ Discrete ระบบจาก Continuous System จะทำให้เกิด Discretization Error เพราะเป็นการประมาณค่าจาก Model โดยวิธีการ Discretization ที่ใช้ในการทดลองนี้จะมีอยู่ 3 ประเภท ได้แก่

1. Forward Euler Discretization
2. Backward Euler Discretization
3. Trapezoidal Discretization (Tustin)



รูปที่ ? Discretization

ที่มา: <https://www.semanticscholar.org/paper/Bilinear-Discrete-PID%C3%97%28n-2%29-stage-PD-cascade-for-Smerpitak-Ukakimaparn/4c3b9d24d9c8a3af1189585f226fad599fe828c8/figure/6>

ในการ Discretization ประเภทต่าง ๆ จะต้องนำสมการ Transfer Function มาเปลี่ยนค่าตัวแปร s โดยการแทนตัวแปรเข้าไปใหม่ ดังนี้

Forward Discretization

$$s = \frac{z - 1}{T_s}$$

Backward Discretization

$$s = \frac{1 - z^{-1}}{T_s}$$

Trapezoidal Discretization

$$s = \frac{2}{T_s} \cdot \frac{z - 1}{z + 1}$$

โดยที่แต่ละวิธีการ Discretization แต่ละประเภทจะให้ความเสถียรของระบบที่ต่างกัน ซึ่งมาจากการนำ left half-plane ใน s-plane มาอยู่บน z-plane โดยเมื่อ Poles นั้นมีตำแหน่งอยู่ในวงกลม 1 หน่วยก็จะมีเสถียรมาก หรือกล่าวโดยสรุปได้ดังนี้

- Forward Discretization

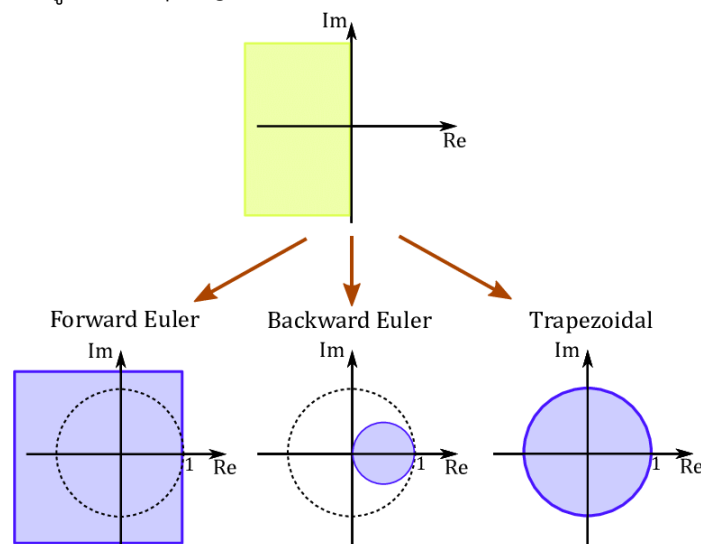
มักให้ค่าที่ไม่เสถียรถึงแม้ Continuous System ที่นำมาใช้จะเสถียรก็ตาม แต่มีแนวโน้มที่ค่าจะเสถียรเมื่อให้ Sampling Time ที่ค่าไม่ใหญ่มาก Discretization Error ก็จะต่ำเพราะตำแหน่ง Poles ก็จะอยู่ภายในพื้นที่วงกลมมากขึ้น

- Backward Discretization

มีความเสถียรสูงเนื่องจากตำแหน่ง Poles มีขนาดเล็กกว่าพื้นที่วงกลม แต่ลักษณะสัญญาณจะดูช้ากว่าความเป็นจริง และเมื่อนำ Continuous System ที่เสถียรมาใช้ ค่าที่ออกมาจะมีความเสถียร

- Trapezoidal Discretization

มีความเสถียรมากที่สุดเนื่องจากตำแหน่ง Poles ทับซ้อนกับพื้นที่วงกลมพอดี และความเสถียรของวิธีนี้ไม่ขึ้นอยู่กับ Sampling Time ทำให้เป็นวิธีการ Discrete ที่นิยมนำมาใช้มากที่สุด



รูปที่ ความสัมพันธ์ของ Poles ใน s-domain และ z-domain

ที่มา: https://www.researchgate.net/figure/Relation-between-poles-in-the-Laplace-s-domain-top-graph-and-corresponding-poles-in_fig27_336880897

ขั้นตอนการดำเนินงาน

ตอนที่ 1 การคำนวณ Discretization ด้วย Transfer Function ของ DC motor จากสมการ (1.7)

$$w[k] = -a_1 \times w[k^{-1}] - a_2 \times w[k^{-2}] + b_0 \times v[k] + b_1 \times v[k^{-1}] + b_2 \times v[k^{-2}]$$

1. Forward Euler (b0, b1, b2, a1, a2) คืออะไร แยก Term แล้วจับตามสมการ

$$b_0 = 0.0$$

$$b_1 = 0.0$$

$$b_2 = 1.497898448477$$

$$a_1 = -0.808250720429$$

$$a_2 = -0.116637445451$$

2. Backward Euler

$$b_0 = 0.660780865414$$

$$b_1 = 0.0$$

$$b_2 = 0.0$$

$$a_1 = -1.408003895913$$

$$a_2 = 0.441138627312$$

3. Trapezoidal (Tustin)

$$b_0 = 0.231922713598$$

$$b_1 = 0.463845427196$$

$$b_2 = 0.231922713598$$

$$a_1 = -1.215397098415$$

$$a_2 = 0.261915980680$$

สมการ Second Order Transfer Function จากการ Discretize จึงมีดังนี้

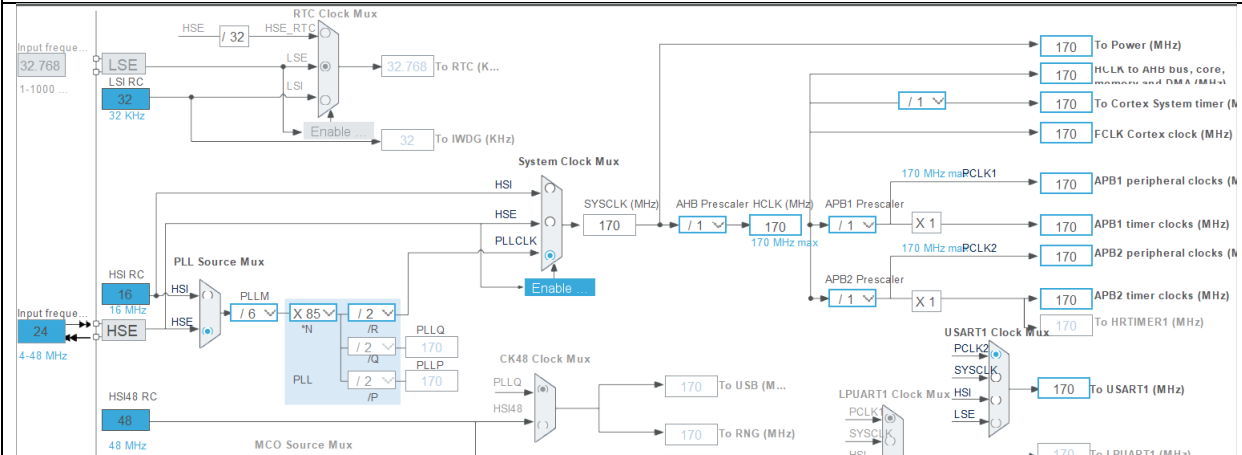
$$\begin{array}{l} \text{Forward} \\ \text{Discretization} \end{array} \quad w[k] = 0.808250720429 \times w[k^{-1}] + 0.116637445451 \times w[k^{-2}] \\ + 1.497898448477 \times v[k^{-2}]$$

$$\begin{array}{l} \text{Backward} \\ \text{Discretization} \end{array} \quad w[k] = 1.408003895913 \times w[k^{-1}] - 0.441138627312 \times w[k^{-2}] \\ + 0.660780865414 \times v[k]$$

$$\begin{array}{l} \text{Trapezoidal} \\ \text{Discretization} \end{array} \quad w[k] = 1.215397098415 \times w[k^{-1}] - 0.261915980680 \times w[k^{-2}] \\ + 0.231922713598 \times v[k] + 0.463845427196 \times v[k^{-1}] \\ + 0.231922713598 \times v[k^{-2}]$$

ตอนที่ 2 การดำเนินการทดสอบ และ เก็บข้อมูล

1. วิธีการตั้ง IOC ใน Cube IDE

Setting	Mode
System Core : NVIC	
TIM2 global interrupt	Enabled
Timer : TIM2	
Clock Source	Internal Clock
Parameter Setting : Prescaler	1699
Parameter Setting : Counter Period	99
Clock Configuration	
	

2. การเขียนโปรแกรมและ Function ที่ใช้ในการเขียน

<pre> 62 63 float t = 0.0f; 64 const float Ts = 0.01f; 65 66 float IN_Ramp = 0.0f; 67 float IN_Sin = 0.0f; 68 69 struct Sin_Estimation { 70 float motor_y; 71 float control_u; 72 float setpoint; 73 float error; 74 float e_n1; 75 float u_n1; 76 float u_n2; 77 float y_n1; 78 float y_n2; 79 } Sin_Estimation[3]; //0 Forward 1 Backward 2 Trapezoidal 80 81 struct Ramp_Estimation { 82 float motor_y; 83 float control_u; 84 float setpoint; 85 float error; 86 float e_n1; 87 float u_n1; 88 float u_n2; 89 float y_n1; 90 float y_n2; 91 } Ramp_Estimation[3]; //0 Forward 1 Backward 2 Trapezoidal 92 </pre>	ประกาศตัวแปรที่ใช้ในการเก็บค่า
<pre> 354/* USER CODE BEGIN 4 */ 355 // This function runs at 1000 Hz automatically based on Timer 2 : Prescaler 1699 + Period 99 356 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) { //set point 357 if (htim->Instance == TIM2) { 358 // Toggle the Green LED (LD2) 359 HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5); 360 361 t += Ts; 362 363 // input Ts 1000000.0 364 // IN_Sin = sinf(0.000000040f * 3.14159f * 0.5f * t) * 12; //for 1Hz 365 // IN_Ramp += Ts * 0.000000010; // for slope 1 366 367 // input Ts 0.01 ioc 1000 Hz 368 IN_Sin = sinf(0.40f * 3.14159f * 0.5f * t) * 12; //for 1Hz 369 IN_Ramp += Ts * 0.10; // for slope 1 370 371 // input Ts 0.000000001 372 // IN_Sin = sinf(40000000.0f * 3.14159f * 0.5f * t) * 12; //for 1Hz 373 // IN_Ramp += Ts * 10000000.0; // for slope 1 374 375 for (int i = 0; i < 3; i++) { 376 377 //set SIN Wave input 378 Sin_Estimation[i].control_u = IN_Sin; 379 Sin_Estimation[i].setpoint = IN_Sin; 380 Sin_Estimation[i].error = Sin_Estimation[i].setpoint 381 - Sin_Estimation[i].motor_y; </pre>	Function Timer และกำหนดค่า Input
<pre> 382 383 //output 384 if (i == 0) { //forward 385 // -> w[k] = -a1*w[k-1] - a2*w[k-2] + b0*v[k] + b1*v[k-1] + b2*v[k-2] 386 Sin_Estimation[0].motor_y = 0.808250720429f 387 * Sin_Estimation[0].y_n1 388 + 0.116637445451f * Sin_Estimation[0].y_n2 389 + 0.000000000000f * Sin_Estimation[0].control_u 390 + 0.000000000000f * Sin_Estimation[0].u_n1 391 + 1.497898448477f * Sin_Estimation[0].u_n2; 392 393 } else if (i == 1) { //backward 394 // -> w[k] = -a1*w[k-1] - a2*w[k-2] + b0*v[k] + b1*v[k-1] + b2*v[k-2] 395 Sin_Estimation[1].motor_y = 1.408003895913f 396 * Sin_Estimation[1].y_n1 397 - 0.441138627312f * Sin_Estimation[1].y_n2 398 + 0.660780865414f * Sin_Estimation[1].control_u 399 + 0.000000000000f * Sin_Estimation[1].u_n1 400 + 0.000000000000f * Sin_Estimation[1].u_n2; 401 402 } else if (i == 2) { //trapezoidal 403 // -> w[k] = -a1*w[k-1] - a2*w[k-2] + b0*v[k] + b1*v[k-1] + b2*v[k-2] 404 Sin_Estimation[2].motor_y = 1.215397098415f 405 * Sin_Estimation[2].y_n1 406 - 0.261915980680f * Sin_Estimation[2].y_n2 407 + 0.231922713598f * Sin_Estimation[2].control_u 408 + 0.463845427196f * Sin_Estimation[2].u_n1 409 + 0.231922713598f * Sin_Estimation[2].u_n2; 410 </pre>	Discrete

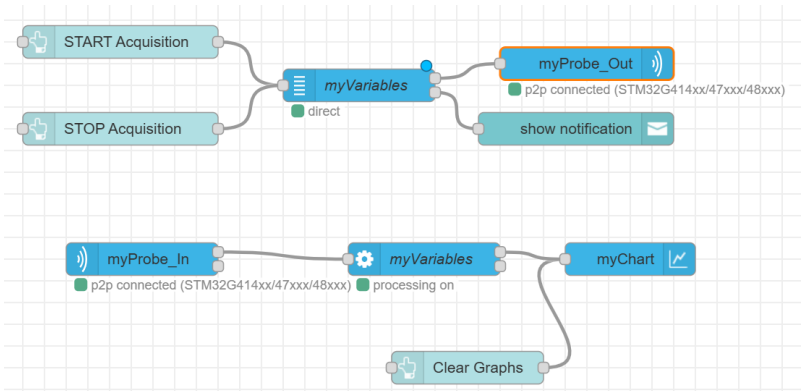
```

436
437 //update state
438 Sin_Estimation[i].y_n2 = Sin_Estimation[i].y_n1;
439 Sin_Estimation[i].y_n1 = Sin_Estimation[i].motor_y;
440
441 Sin_Estimation[i].u_n2 = Sin_Estimation[i].u_n1;
442 Sin_Estimation[i].u_n1 = Sin_Estimation[i].control_u;
443 }
444
445 for (int i = 0; i < 3; i++) {
446
447 //set RAMP Wave input
448 Ramp_Estimation[i].control_u = IN_Ramp;
449 Ramp_Estimation[i].setpoint = IN_Ramp;
450 Ramp_Estimation[i].error = Ramp_Estimation[i].setpoint
451 - Ramp_Estimation[i].motor_y;
452
453 //output
454 if (i == 0) { //forward
455 // -> w[k] = -a1*w[k-1] - a2*w[k-2] + b0*v[k] + b1*v[k-1] + b2*v[k-2]
456 Ramp_Estimation[0].motor_y = 0.808250720429f
457 * Ramp_Estimation[0].y_n1
458 + 0.116637445451f * Ramp_Estimation[0].y_n2
459 + 0.000000000000f * Ramp_Estimation[0].control_u
460 + 0.000000000000f * Ramp_Estimation[0].u_n1
461 + 1.497898448477f * Ramp_Estimation[0].u_n2;
462
463 } else if (i == 1) { //backward
464 // -> w[k] = -a1*w[k-1] - a2*w[k-2] + b0*v[k] + b1*v[k-1] + b2*v[k-2]
465 Ramp_Estimation[1].motor_y = 1.408003895913f
466 * Ramp_Estimation[1].y_n1
467 - 0.441138627312f * Ramp_Estimation[1].y_n2
468 + 0.660780865414f * Ramp_Estimation[1].control_u
469 + 0.000000000000f * Ramp_Estimation[1].u_n1
470 + 0.000000000000f * Ramp_Estimation[1].u_n2;
471
472 } else if (i == 2) { //trapezoidal
473 // -> w_k = (-a2 * w_k1) - (a3 * w_k2) + (b1 * v_k) + (b2 * v_k1) + (b3 * v_k2);
474 Ramp_Estimation[2].motor_y = 1.215397098415f
475 * Ramp_Estimation[2].y_n1
476 - 0.261915980600f * Ramp_Estimation[2].y_n2
477 + 0.231922713598f * Ramp_Estimation[2].control_u
478 + 0.463845427196f * Ramp_Estimation[2].u_n1
479 + 0.231922713598f * Ramp_Estimation[2].u_n2;
480
481 }
482
483 //update state
484 Ramp_Estimation[i].y_n2 = Ramp_Estimation[i].y_n1;
485 Ramp_Estimation[i].y_n1 = Ramp_Estimation[i].motor_y;
486
487 Ramp_Estimation[i].u_n2 = Ramp_Estimation[i].u_n1;
488 Ramp_Estimation[i].u_n1 = Ramp_Estimation[i].control_u;
489
490 }
491
492 }
493
494 }
495
496 }
497
498 }
499
500 }
501
502 }
503
504 }
505
506 }
507
508 }
509
510 }
511
512 }
513
514 }
515

```

Discrete

3. การเชื่อมต่อกับ Cube Monitor



รูปที่ การเชื่อมต่อ Cube Monitor ผ่าน ST-Link

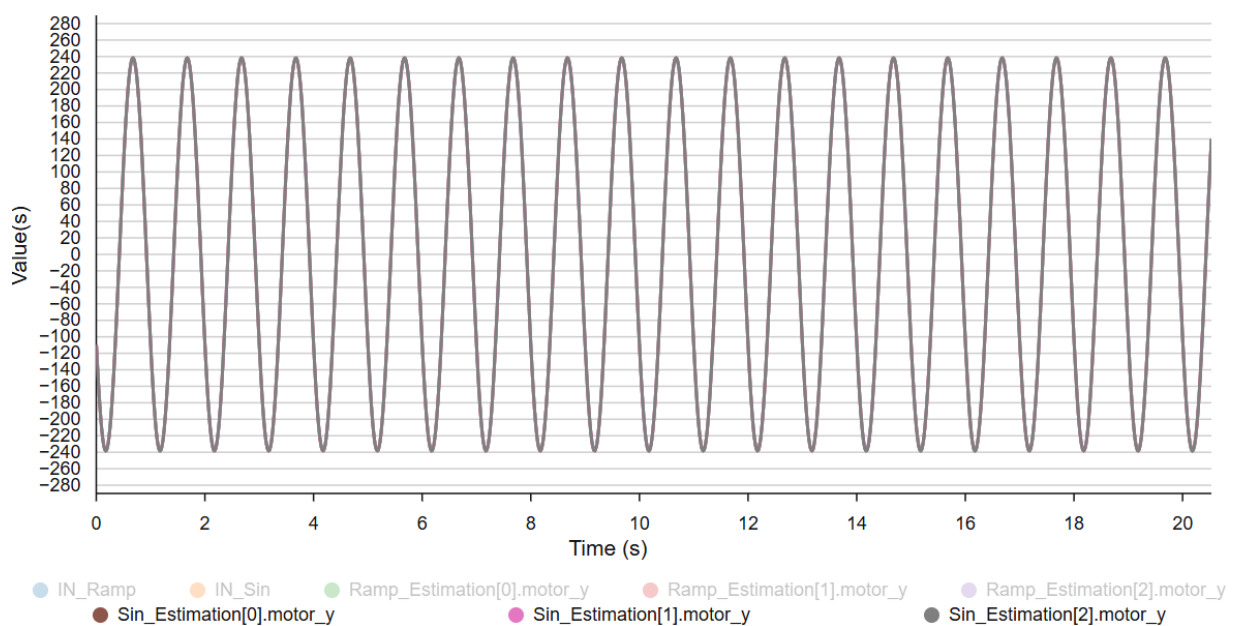
4. การเก็บค่าลงไฟล์ .csv เพื่อนำไปวิเคราะห์ผล

ตอนที่ 3 วิธีการวิเคราะห์ผล

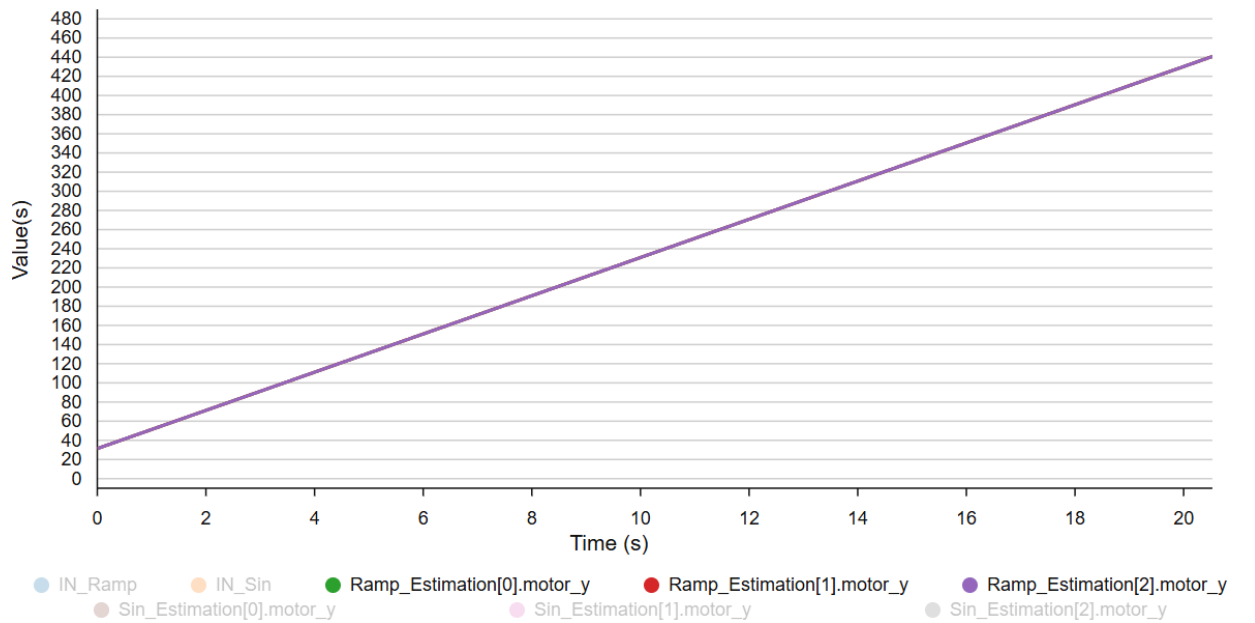
1. เพื่อศึกษาและเปรียบเทียบความแตกต่างของสัญญาณจำลองจากการ Discretize ประเภทที่ต่างกัน นำข้อมูลการตอบสนองจากการจำลองที่มี Input และ Sampling Time เดียวกัน มา plot บน MATLAB
2. เพื่อวิเคราะห์ความแตกต่างระหว่างสัญญาณจำลองและสัญญาณมอเตอร์จริง นำข้อมูลการตอบสนองทั้งสองชุดเมื่อให้ Input แบบ Sine Wave 1 Hz และ Ramp Wave slope 1 ในช่วงเวลา 0-20 s. มา plot บนโปรแกรม MATLAB
3. เพื่อศึกษาและเปรียบเทียบความแตกต่างของสัญญาณจำลองจาก Sampling Time ที่ต่างกัน นำข้อมูลการตอบสนองจากการจำลองที่มี Input และ Discretization เดียวกัน มา plot บน MATLAB

ผลการทดลอง

เมื่อได้สมการ Discretize แต่ละประเภทจาก Second Order Transfer Function ของ DC Motor จึงนำไปเขียนบน STM32G474RE ซึ่งจะได้สัญญาณ Response ที่อยู่ในรูปแบบ Sine Wave และ Ramp Wave แสดงผลบน STM32Cube Monitor ดังนี้



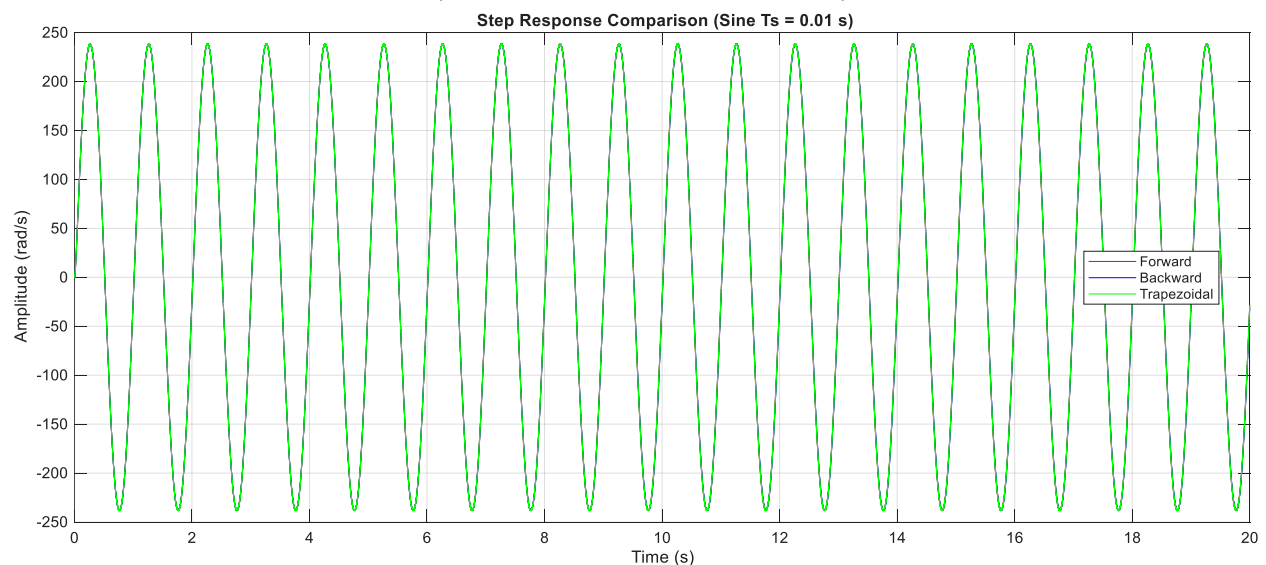
รูปที่ สัญญาณตอบสนองแบบ Sine Wave ของการ Discretize ทั้ง 3 ประเภท



รูปที่ สัญญาณตอบสนองแบบ Ramp Wave ของการ Discretize ทั้ง 3 ประเภท

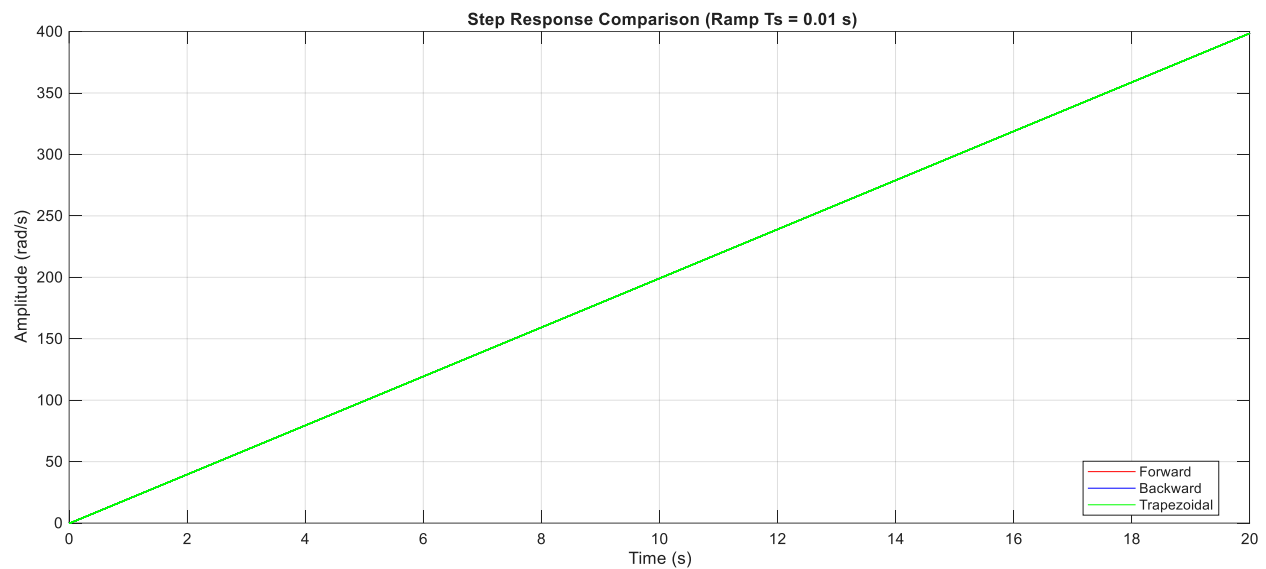
นอกเหนือจากการแสดงการตอบสนองบน STM32Cube Monitor เพื่อให้สามารถนำมาวิเคราะห์ได้ละเอียดขึ้นจึงมีการเก็บค่าการตอบสนองบนโปรแกรม STM32Cube IDE มาแสดงบน MATLAB เพิ่มเติมเพื่อวิเคราะห์ความแตกต่างระหว่างประเภทของ Discretization, Sampling Time ที่เปลี่ยนไป และการตอบสนองที่เกิดขึ้นจริง

การทดลองเพื่อเปรียบเทียบสัญญาณจำลองมอเตอร์ Sine Wave ด้วยการทำ Forward Discretization, Backward Discretization และ Trapezoidal Discretization ที่ Sampling Time 0.01 s. เป็นเวลา 20 s.



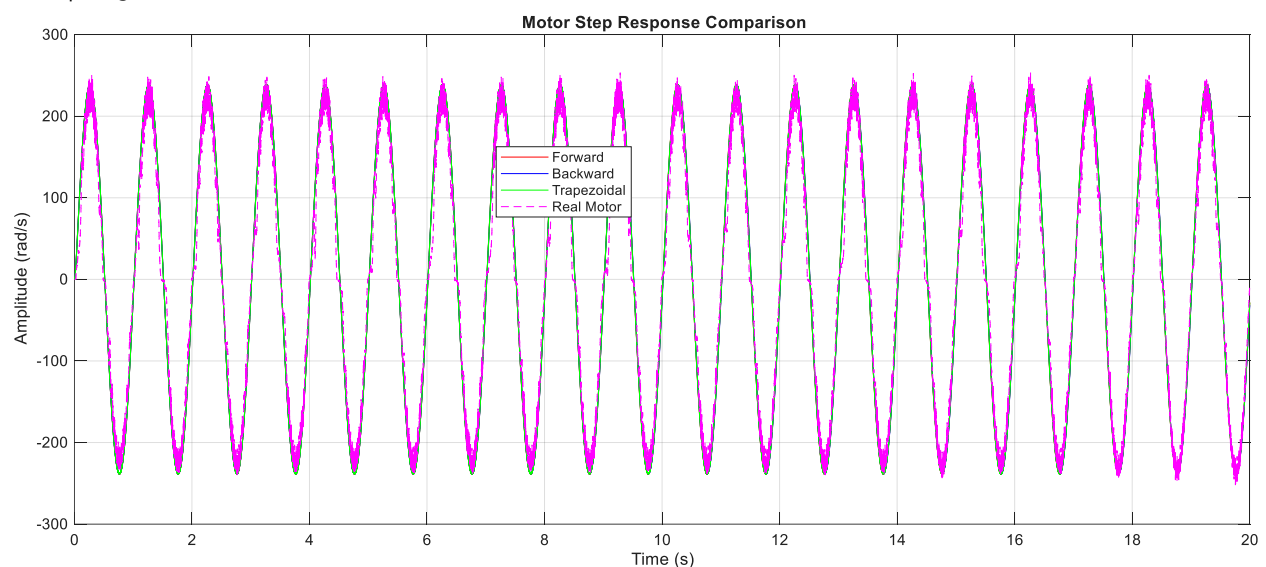
รูปที่ การตอบสนองแบบ Sin Wave ของการ Discretize ทั้ง 3 ประเภทบน MATLAB

การทดลองเพื่อเปรียบเทียบสัญญาณจำลองมอเตอร์ Ramp Wave ด้วยการทำ Forward Discretization, Backward Discretization และ Trapezoidal Discretization ที่ Sampling Time 0.01 s. เป็นเวลา 20 s.

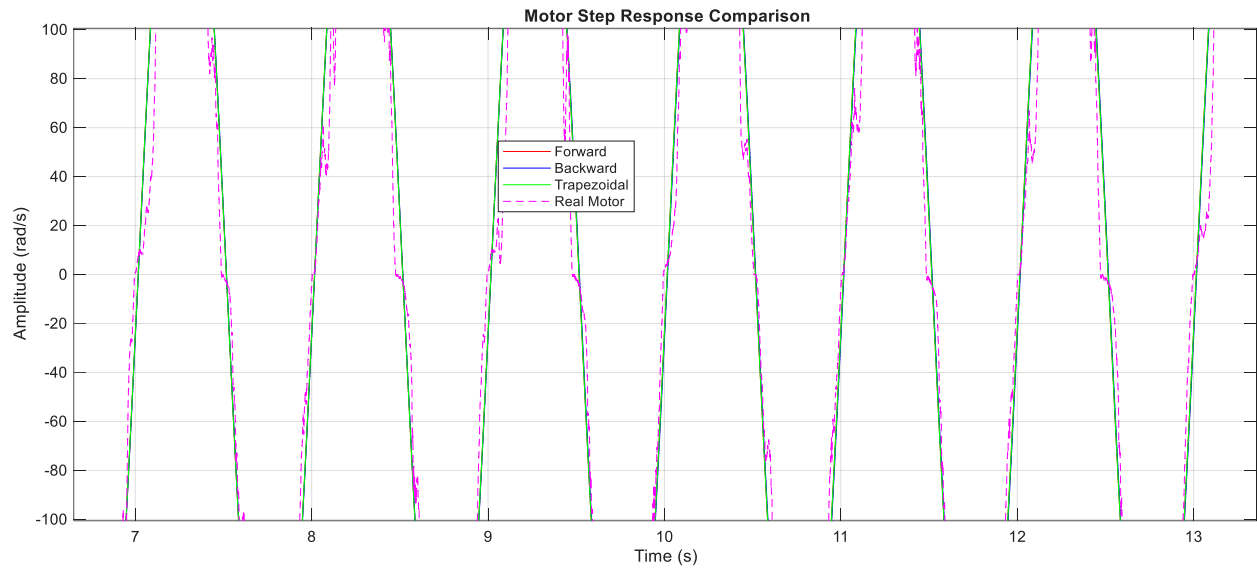


รูปที่ การตอบสนองแบบ Ramp Wave ของการ Discretize ทั้ง 3 ประเภทบน MATLAB

การทดลองเพื่อเปรียบเทียบสัญญาณจำลองมอเตอร์กับสัญญาณมอเตอร์จริงที่มี Input เป็น Sine Wave 1 Hz ด้วยการทำ Forward Discretization, Backward Discretization และ Trapezoidal Discretization ที่ Sampling Time 0.01 s. เป็นเวลา 20 s.

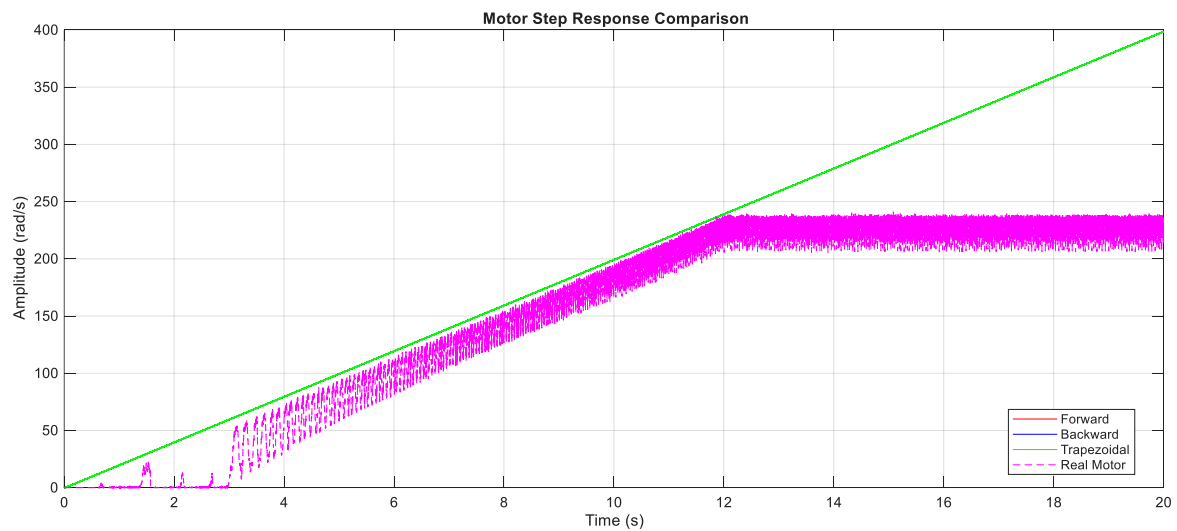


รูปที่ การตอบสนองแบบ Sin Wave ของสัญญาณจำลองและสัญญาณจริง

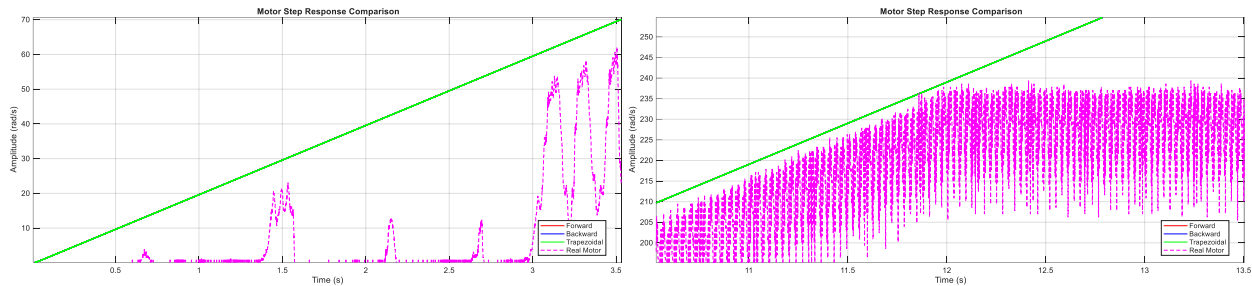


รูปที่ การตอบสนองแบบ Sin Wave ของสัญญาณจำลองและสัญญาณจริง

การทดลองเพื่อเปรียบเทียบสัญญาณจำลองมอเตอร์กับสัญญาณมอเตอร์จริงที่มี Input เป็น Ramp Wave slope 1 ด้วยการทำ Forward Discretization, Backward Discretization และ Trapezoidal Discretization ที่ Sampling Time 0.01 s. เป็นเวลา 20 s.

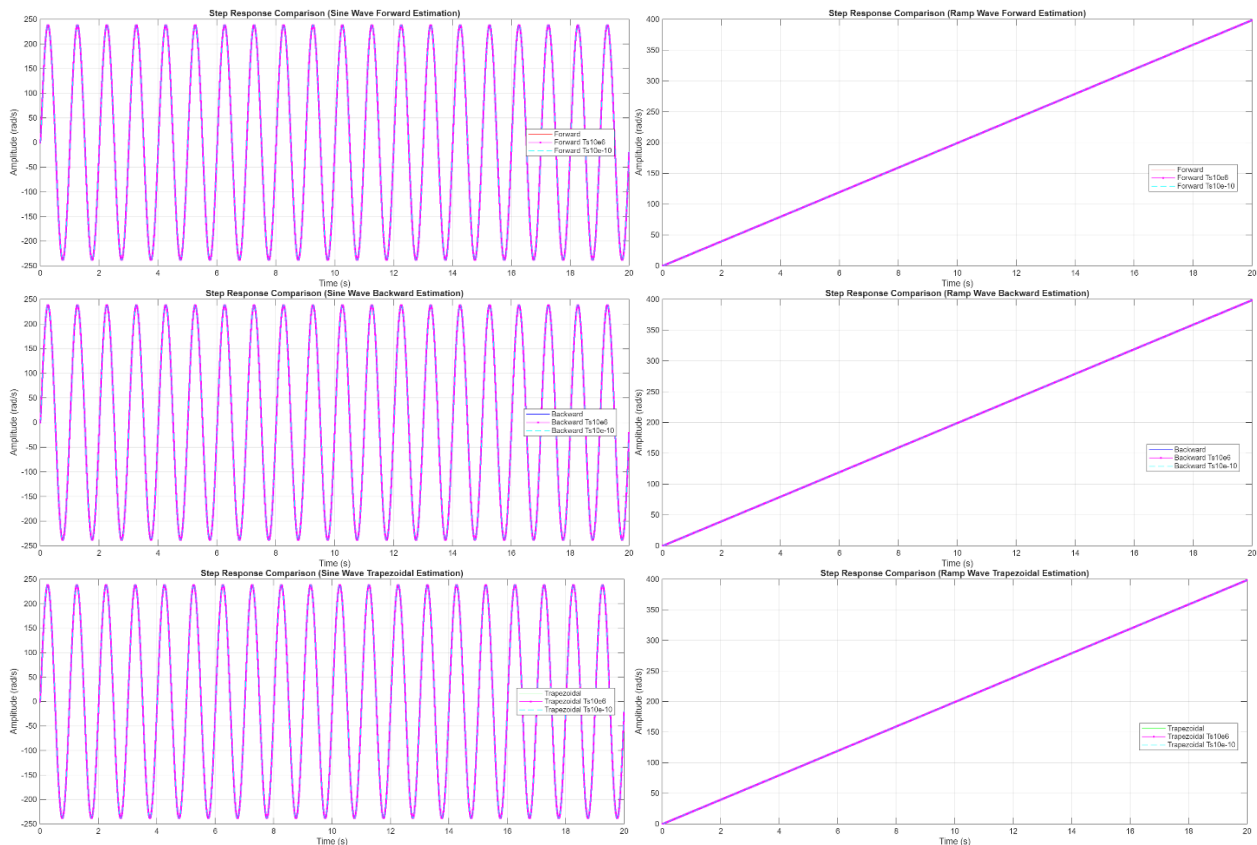


รูปที่ การตอบสนองแบบ Ramp Wave ของสัญญาณจำลองและสัญญาณจริง



รูปที่ การตอบสนองแบบ Ramp Wave ของสัญญาณจำลองและสัญญาณจริง

การทดลองเพื่อเปรียบเทียบสัญญาณจำลองมอเตอร์ Forward Discretization, Backward Discretization และ Trapezoidal Discretization เมื่อเปลี่ยน Sampling Time เป็น 10^{-6} s, 0.01 s. และ 10^{-10} s.

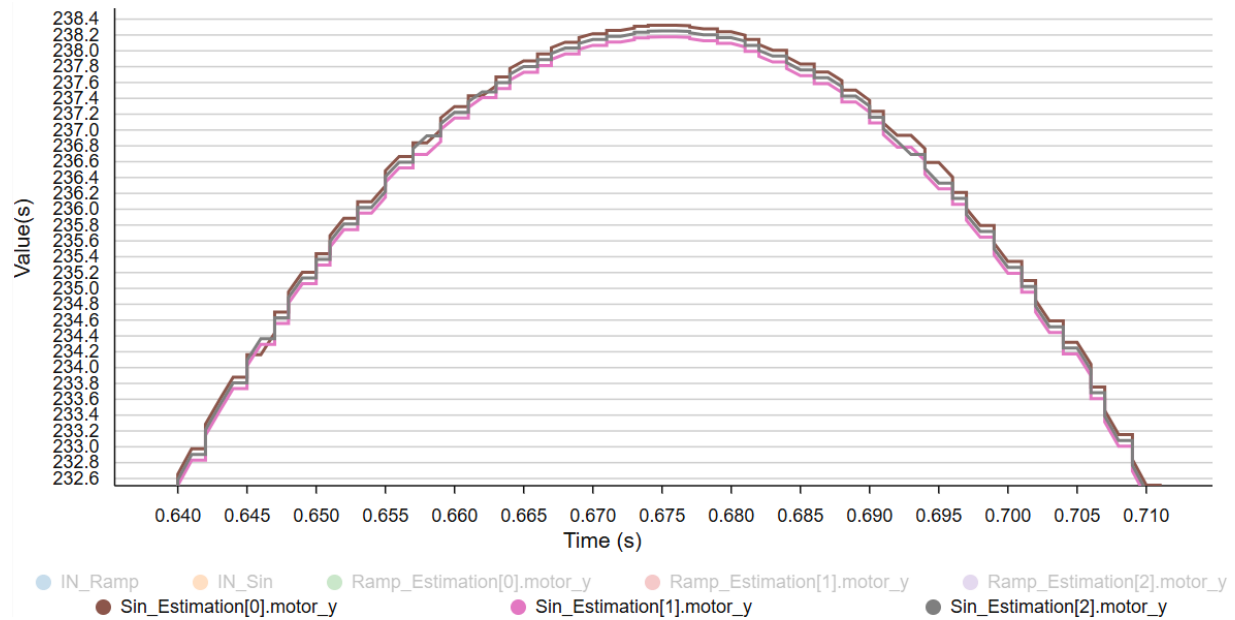


รูปที่ การตอบสนองของสัญญาณจำลองของ Sine Wave และ Ramp Wave ที่ Sampling Time ต่าง ๆ

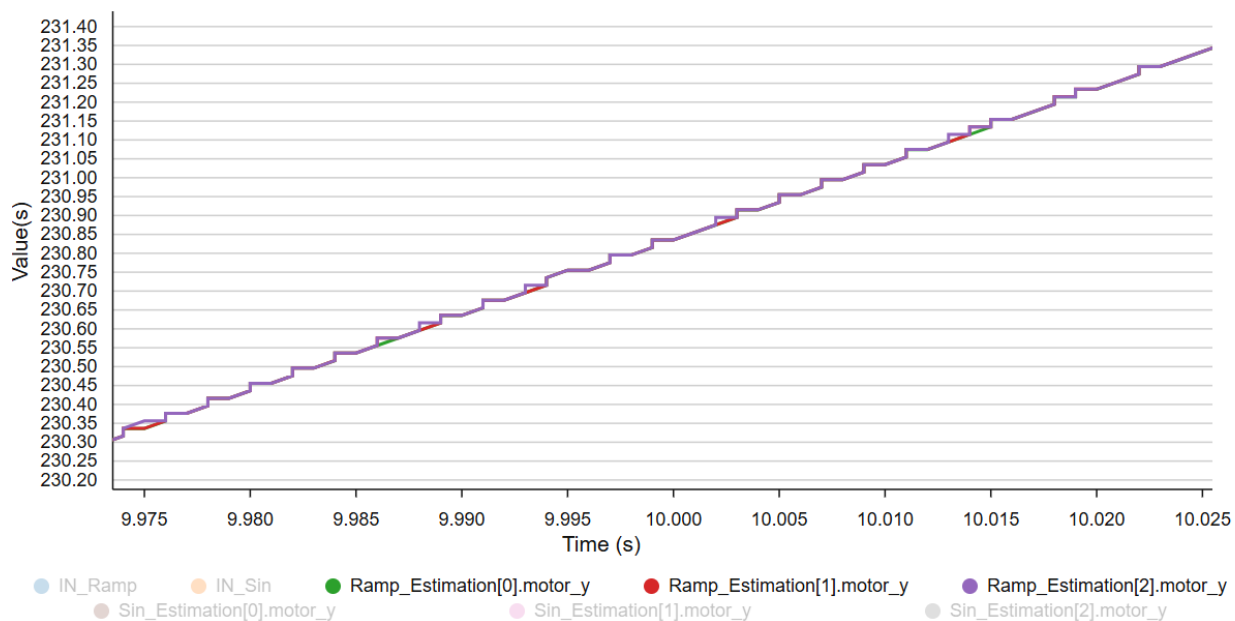
สรุปผลการทดลอง

การตอบสนองที่แสดงบน STM32Cube Monitor สามารถนำค่า Output ที่มาจากโปรแกรมได้โดยตรงผ่าน ST-Link และสามารถแสดงตัวแปรได้พร้อมกันหลายตัว โดยสัญญาณที่แสดงดังภาพคือสัญญาณจำลอง

Forward Discrete (Estimation[0]), Backward Discrete (Estimation[1]) และ Trapezoidal Discrete (Estimation[2])



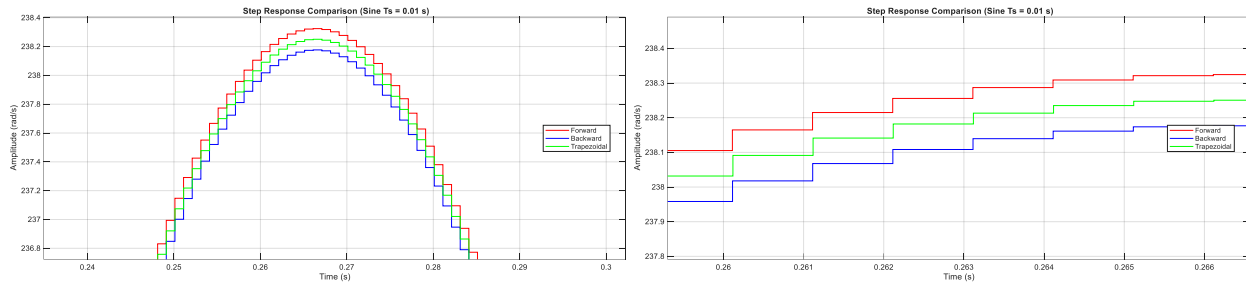
รูปที่ สัญญาณตอบสนองแบบ Sine Wave ของการ Discretize ทั้ง 3 ประเภท



รูปที่ สัญญาณตอบสนองแบบ Ramp Wave ของการ Discretize ทั้ง 3 ประเภท

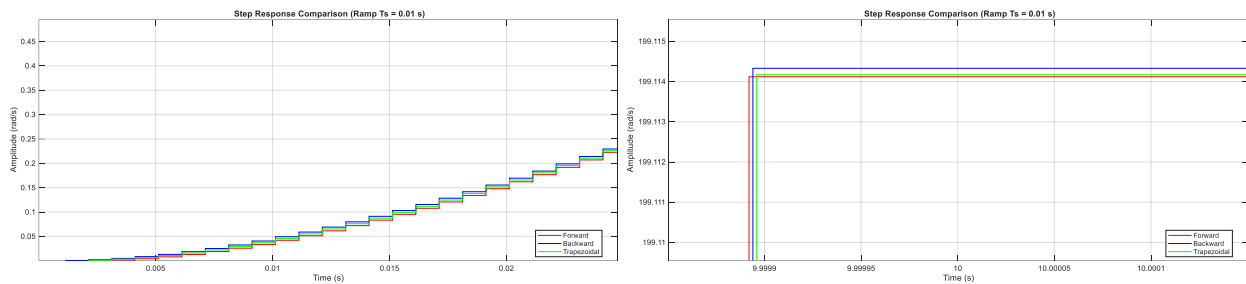
จากการทดลองเปรียบเทียบการ Discrete แต่ละประเภท จะสามารถสังเกตได้ว่า สัญญาณ Sine Wave และ Ramp Wave จากการ Discretize ทำให้ลักษณะสัญญาณมีแนวโน้มที่ใกล้เคียงกัน แตกต่างกันเพียงเล็กน้อย

ในช่วง Sin Wave สัญญาณแบบ Forward Estimation (สีแดง) จะมี Amplitude ที่สูงที่สุดตามด้วย Trapezoidal Estimation (สีเขียว) และ Backward Estimation (สีฟ้า) ตามลำดับ



รูปที่ สัญญาณตอบสนองแบบ Sine Wave ของการ Discretize ทั้ง 3 ประเภท บน MATLAB

และสำหรับ Ramp Wave สัญญาณแบบ Backward Estimation (สีฟ้า) จะมี Amplitude ที่สูงที่สุดตามด้วย Trapezoidal Estimation (สีเขียว) และ Forward Estimation (สีแดง) ตามลำดับ และการตอบสนองมีความแตกต่างกันเล็กน้อย โดยของ Forward Estimation ตอบสนองเร็วที่สุด ตามด้วย Backward Estimation และ Trapezoidal Estimation

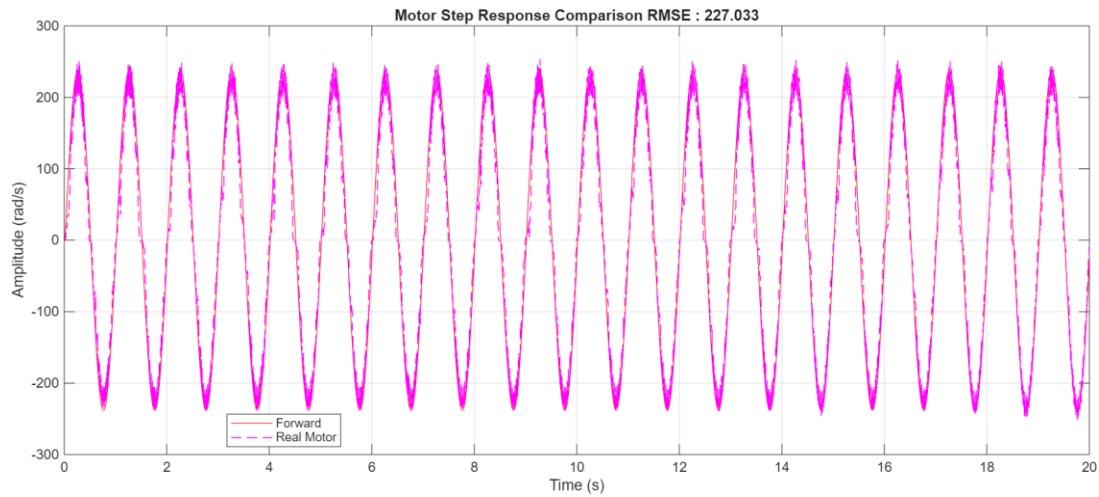


รูปที่ สัญญาณตอบสนองแบบ Ramp Wave ของการ Discretize ทั้ง 3 ประเภท บน MATLAB

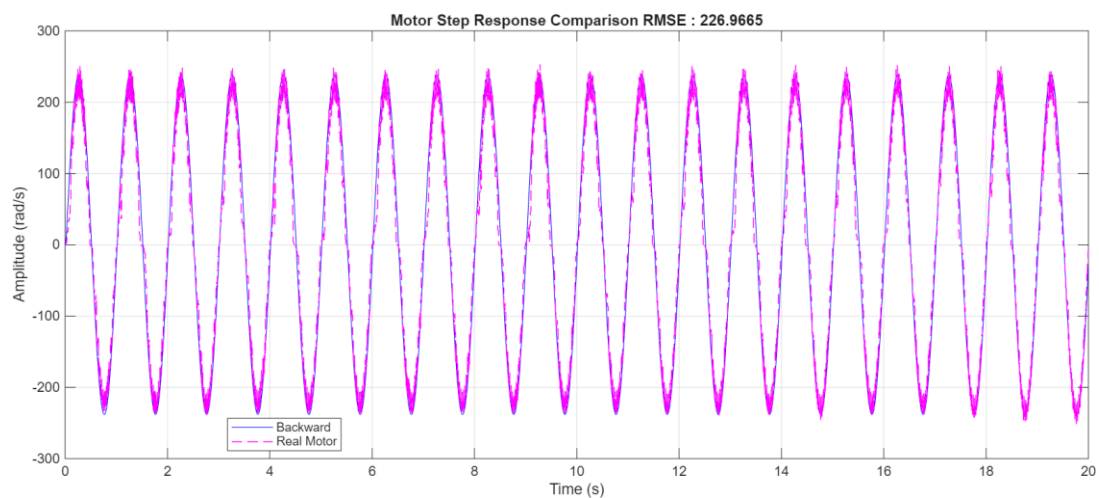
เมื่อนำสัญญาณจำลองที่ผ่านการ Discretize ด้วยวิธีต่าง ๆ มาเปรียบเทียบกับสัญญาณจริงของมอเตอร์ในรูปแบบ Sin Wave จะได้ว่าลักษณะสัญญาณจำลองมีแนวโน้มที่ใกล้เคียงสัญญาณจริงที่ไม่เท่ากัน เพื่อให้สามารถเห็นความแตกต่างจึงนำข้อมูลมาวิเคราะห์ค่าความคลาดเคลื่อนหรือ Root Mean Square Error (RMSE) โดยสัญญาณแบบ Forward Estimation (รูปที่ 22) มี RMSE อยู่ที่ 227.033 สัญญาณแบบ Backward Estimation (รูปที่ 23) มี RMSE อยู่ที่ 226.9665 และสัญญาณแบบ Trapezoidal Estimation (รูปที่ 24) มี RMSE อยู่ที่ 226.9941

และการเปรียบเทียบในรูปแบบ Ramp Wave จะได้ว่าลักษณะสัญญาณที่เกิดขึ้นจริงไม่ได้มีลักษณะเป็นเส้นตรงเหมือนการจำลอง และสัญญาณจริงนั้นมีช่วงที่เกิดค่าที่คงที่หรือช่วง Saturation และช่วงที่มอเตอร์กำลังเริ่มหมุนที่เวลาประมาณ 0-3 s. สัญญาณจริงมอเตอร์มีค่าอยู่ที่ 0 โดยที่สัญญาณแบบ Forward Estimation (รูปที่

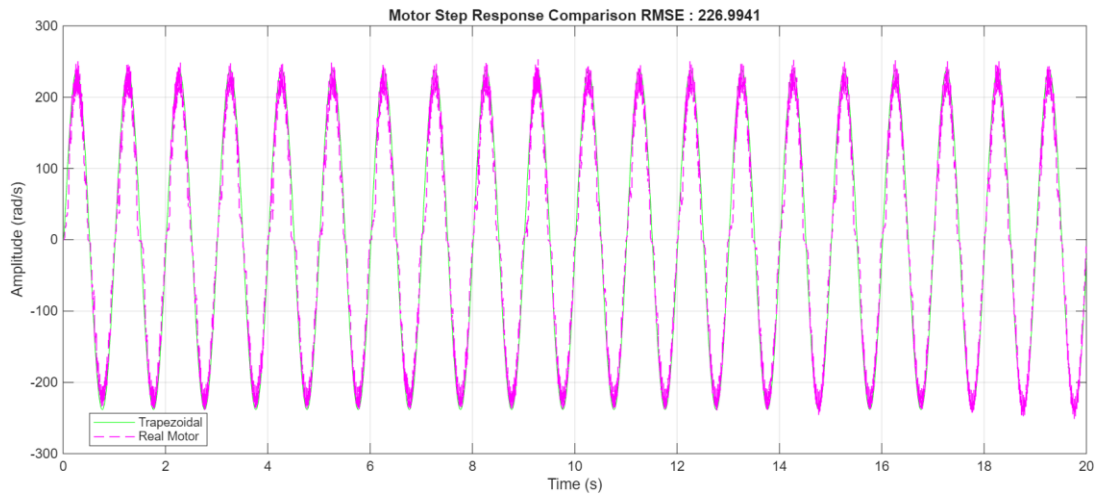
25) มี RMSE อยู่ที่ 134.6212 สัญญาณแบบ Backward Estimation (รูปที่ 26) มี RMSE อยู่ที่ 134.6214 และสัญญาณแบบ Trapezoidal Estimation (รูปที่ 27) มี RMSE อยู่ที่ 134.6213



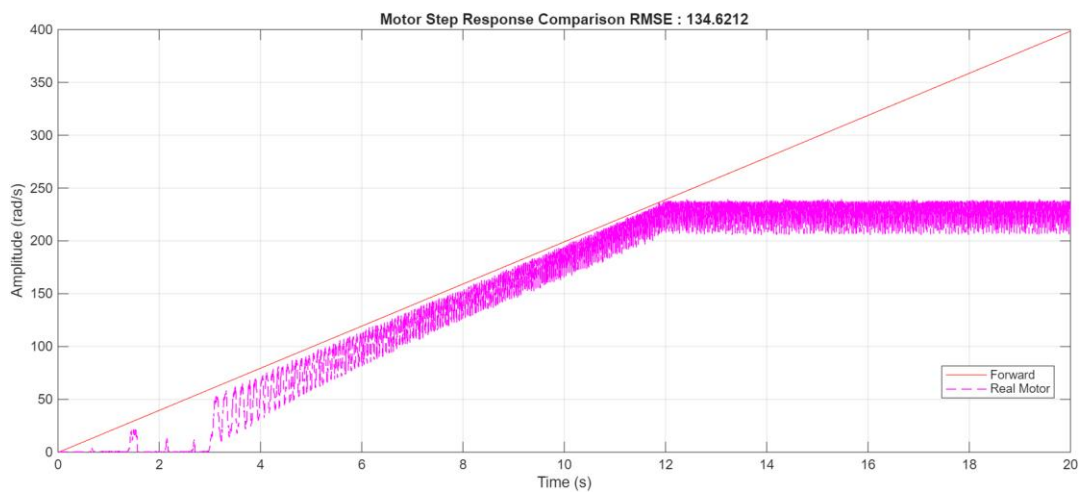
รูปที่ การตอบสนองแบบ Forward Estimation ของ Sine Wave ของสัญญาณจำลองและสัญญาณจริง



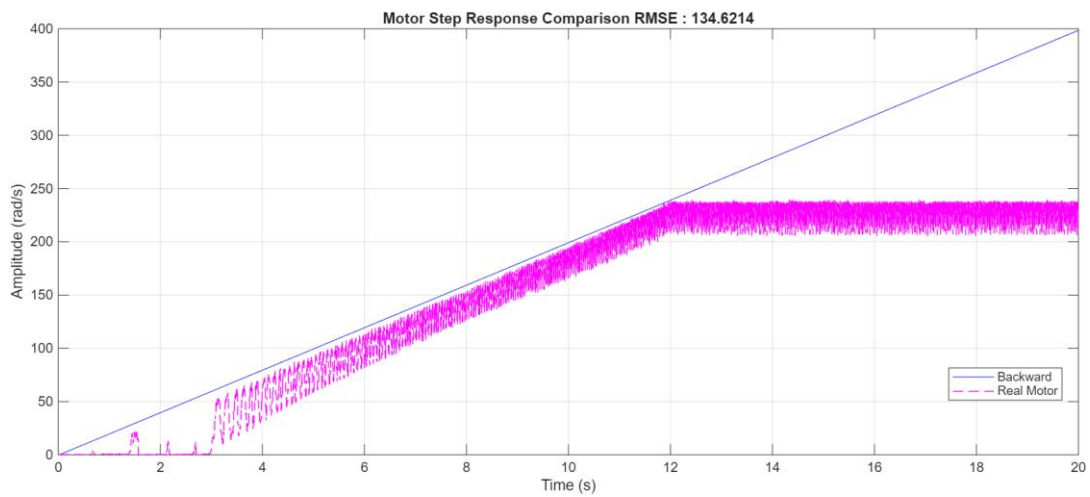
รูปที่ การตอบสนองแบบ Backward Estimation ของ Sine Wave ของสัญญาณจำลองและสัญญาณจริง



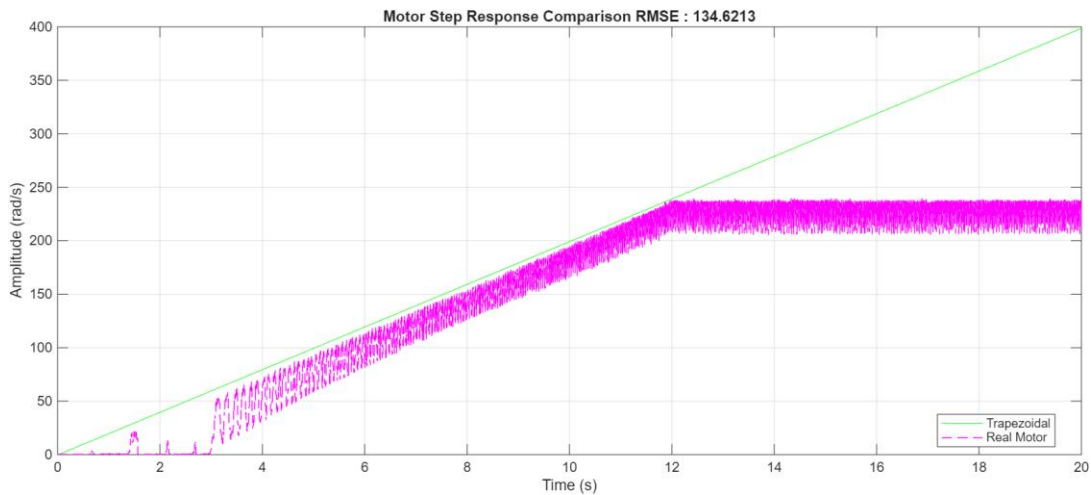
รูปที่ การตอบสนองแบบ Trapezoidal Estimation ของ Sine Wave ของสัญญาณจำลองและสัญญาณจริง



รูปที่ การตอบสนองแบบ Forward Estimation ของ Ramp Wave ของสัญญาณจำลองและสัญญาณจริง

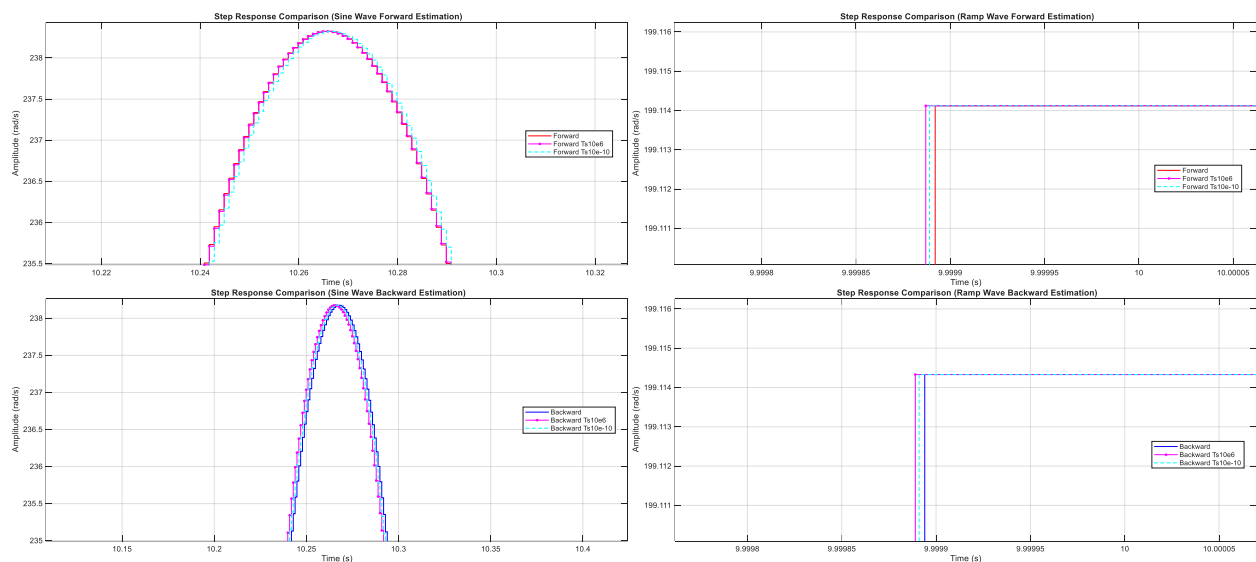


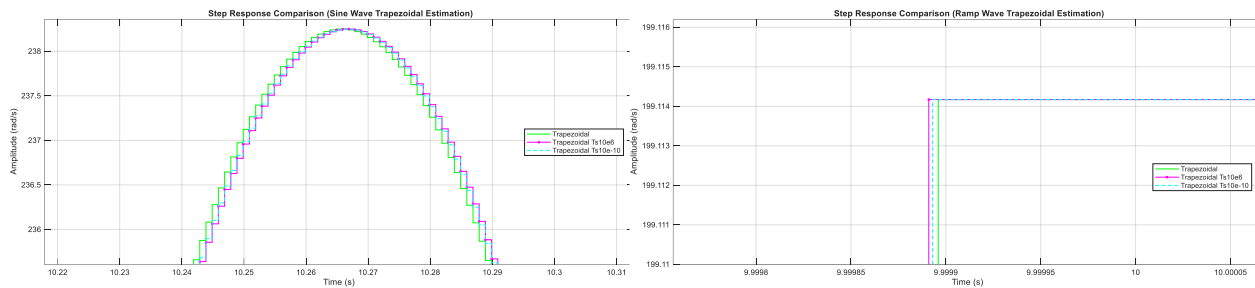
รูปที่ การตอบสนองแบบ Backward Estimation ของ Ramp Wave ของสัญญาณจำลองและสัญญาณจริง



รูปที่ การตอบสนองแบบ Trapezoidal Estimation ของ Ramp Wave ของสัญญาณจำลองและสัญญาณจริง

เนื่องจากการ Discretize ด้วยวิธีต่าง ๆ มีข้อจำกัดหรือจะมีลักษณะที่ต่างออกไปเมื่อ Sampling Time เปลี่ยนแปลงไป โดยจากการทดลองสัญญาณ Ramp Wave ที่ Discretization ประเภทเดียวกันเมื่อเปลี่ยน Sampling Time มีลักษณะที่ไม่แตกต่างกันมากและมีแนวโน้มคล้ายกัน แต่สัญญาณจำลองแบบ Sine Wave เมื่อเปลี่ยน Sampling Time สัญญาณที่ผ่านการ Discretize ประเภทต่างกันมีแนวโน้มของสัญญาณที่ต่างกันชัดเจน





รูปที่ การตอบสนองของสัญญาณจำลองของ Sine Wave และ Ramp Wave ที่ Sampling Time ต่าง ๆ

อภิปรายผล

จากการทดลองเปรียบเทียบการทำ Discretization ด้วยวิธี Forward Estimation, Backward Estimation และ Trapezoidal Estimation แสดงให้เห็นว่าการทำ Discrete มีลักษณะที่ต่างกัน เนื่องมาจากตำแหน่งของค่าที่นำมาใช้ประมาณค่าคนละตำแหน่งกัน โดยวิธี Forward คือการนำค่าด้านหน้ามาใช้ แต่ Backward คือการนำค่าด้านหลังมาใช้ ส่วน Trapezoidal คือการนำค่าตรงกลางมาใช้ จึงเป็นเหตุผลว่าการใช้ Trapezoidal Discretization เหมาะสมที่สุดเพราะค่าที่นำมาใช้คือค่ากลาง อีกทั้ง Amplitude ของ Discretization วิธีต่าง ๆ ของ Sin Wave ไม่เท่ากันนั้นเองก็เกิดมาจากตำแหน่งของค่าที่นำมาประมาณด้วย

สัญญาณจำลองและสัญญาณของมอเตอร์จริงไม่เท่ากันและลักษณะไม่เหมือนกันเนื่องจาก Motor Characteristic โดยเฉพาะ Ramp Wave ที่สามารถสังเกตได้ชัดเจนเพราะสัญญาณที่มอเตอร์ตอบสนองเป็นเหมือนสัญญาณ Sine Wave ที่มีความถี่สูงมาก การตอบสนองที่ออกมาจากมอเตอร์จึงเป็นเหมือนแทบสี และมีบริเวณที่เกิด Rotational Friction เนื่องจากการที่มอเตอร์ได้รับแรงไม่เพียงพอที่จะต้านแรงเสียดทานจนเกิดการหมุน อีกทั้งสัญญาณมอเตอร์จริงมีบริเวณ Saturation ที่ทำให้มอเตอร์เคลื่อนที่ด้วยความเร็วคงที่ไปเรื่อย ๆ

แต่ละวิธีการของ Discretization มีความเสถียรที่แตกต่างกัน ซึ่งความเสถียรของบางวิธีขึ้นอยู่กับ Sampling Time เพราะความเสถียรของระบบมาจากการนำตำแหน่ง poles ที่อยู่บน s-plane มาไว้บน z-plane และตำแหน่ง poles เมื่ออยู่ภายในพื้นที่วงกลม 1 หน่วย จะยังมีความเสถียร ซึ่ง Forward Estimation มีตำแหน่ง poles ทั้งในพื้นที่วงกลมและนอกทำให้เมื่อยิ่งใช้ Sampling Time ที่ค่าใหญ่มากระบบจะไม่เสถียร กลับกัน Backward Estimation และ Trapezoidal Estimation มีตำแหน่ง poles ที่อยู่ภายในขอบเขตของวงกลม ทำให้ระบบมีความเสถียรมากกว่าและในหลายสถานการณ์มากกว่า

อ้างอิง

<https://ctms.engin.umich.edu/CTMS/?example=MotorSpeed§ion=SystemModeling>
<https://www.engr.siu.edu/staff/spezia/Web438A/Lecture%20Notes/lesson14et438a.pdf>
<https://www.engr.siu.edu/staff/spezia/Web438A/Lecture%20Notes/lesson14et438a.pdf>

https://engineering.purdue.edu/~zak/ECE_382-Fall_2018/ME360ArmatureDCMotorTransferFunction%5B1%5D.pdf
<https://www.mathworks.com/discovery/hardware-in-the-loop-hil.html>
<https://x-engineer.org/discretizing-transfer-function/>
<https://www-verimag.imag.fr/~tdang/DocumentsCours/Discretization.pdf>

การทดลองที่ 2 การศึกษาผลกระทบจาก Disturbance ที่เกิดจากแรงโน้มถ่วง

จุดประสงค์

1. เพื่อศึกษาผลกระทบจาก Disturbance ที่เกิดจากแรงโน้มถ่วงต่อระบบ
2. เพื่อศึกษาการประยุกต์ใช้ Disturbance Feed Forward (DFF) ที่ส่งผลต่อระบบ P Controller

สมมติฐาน

ในระบบนี้ได้เลือกให้ $T_{load} = M_p g L \sin \theta$ ที่เกิดจากมวลติดปลาย (M_p) เป็น Disturbance ของระบบ โดยมีการเปลี่ยนแปลงค่าระหว่างการทำงานที่เนื่องจากแรงโน้มถ่วง โดยการทำให้ DFF จะชดเชย T_{load} สำหรับ M_p จนมากเพียงพอที่จะทำให้ระบบมีลักษณะทำงานเหมือนไม่มี T_{load} หรือในอีกความหมายหนึ่งก็คือ เมื่อสั่งให้ Motor มีค่า θ_{init} ใด ๆ ตำแหน่งของ M_p จะยังสามารถคงสภาพเดิมได้ โดยสามารถสังเกตพฤติกรรมนี้ได้จากการปล่อย M_p ร่วงลงมาอย่างอิสระ หรือในกรณีที่มีการใช้ร่วมกับ Controller เช่น P Controller จะทำให้มีค่าคงตัว หรือ พฤติกรรมของ System Response ที่แตกต่างกัน

ตัวแปร

1. ตัวแปรต้น:
 - θ_{init} [rad]
 - θ_{ref} [rad]
 - $K_p = \{0,1\}$
2. ตัวแปรตาม:
 - Tracking $\theta_{measured}$ [rad]
 - ลักษณะของ System Response
 - System Characteristics ที่มี DFF และ ไม่มี DFF
3. ตัวแปรควบคุม:
 - $K_I, K_D = 0$
 - DC Motor Parameter ใช้ค่าจากการทดลองดังตารางที่ 1

เอกสารและงานวิจัยที่เกี่ยวข้อง

1. **Disturbance Feed Forward (DFF)** คือ การชดเชยผลกระทบที่เกิดจาก Disturbance โดยที่ Disturbance จะต้อง มี 2 คุณสมบัติดังนี้ 1. Disturbance นี้ส่งผลอย่างมีนัยยะสำคัญต่อการควบคุม เช่น แรงโน้มถ่วง 2. Disturbance นี้ต้องสามารถวัดได้ หรือสามารถคำนวณได้ในทางคณิตศาสตร์ DFF มีรูปแบบการควบคุมแบบ Proactive ซึ่งหมายถึง ระบบจะตอบสนองก่อนได้รับค่า Feedback กลับมาจาก Sensors โดยมีความสัมพันธ์ทางคณิตศาสตร์สมการนี้

$$G_{DFF}(s) = \underbrace{G_P^{-1}}_{\text{Inverse Transfer Function}} \underbrace{\left(\frac{1}{\tau s + 1}\right)^r}_{\text{Low Pass Filter}} \quad (2.1)$$

หรือในบางแหล่งอ้างอิงจะใช้เป็นอัตราส่วน ของ Plant Transfer Function ($G_P(s)$) และ Disturbance Transfer Function ($G_D(s)$) ได้ดังสมการนี้

$$G_{DFF}(s) = -\frac{G_D(s)}{G_P(s)} \quad (2.2)$$

2. **Gravity Compensation** คือ การประยุกต์ใช้เรื่อง DFF โดยเลือก Disturbance เป็น แรงโน้มถ่วง ดังนั้นสิ่งที่จะชดเชยในระบบนี้ก็คือ ระบบที่มีการชดเชยผลกระทบจากแรงโน้มถ่วง ในเชิงการประยุกต์ใช้เข้ากับการ Control การใช้ Gravity Compensation จะทำให้ระบบเสมือนทำงานในสภาพไร้แรงโน้มถ่วง

ขั้นตอนการดำเนินงาน

ตอนที่ 1 การคำนวณ Transfer Function ของ DFF

1. เนื่องจากในระบบนี้ได้มีการเลือก Disturbance เป็น $T_{load} = M_p g L \sin \theta$ จากจะทำการทำ Disturbance Feed Forward เพื่อทำให้ระบบไม่ได้รับผลกระทบจาก T_{load}
2. ใช้สมการที่ (2.2) เป็นสมการในการทำ Disturbance Feed Forward
3. ให้ $G_p(s)$ เป็นสมการ Transfer Function ของ DC Motor with Vertical Pendulum หรือ สมการข้อที่ (1.14)
4. ให้ $G_D(s)$ เป็นสมการ Transfer Function ของ Disturbance ระบบ ดังสมการนี้

$$G_D(s) = \frac{L_m s + R_m}{(L_m s + R_m)[(J_m + M_p L^2)s^2 + b_m s + M_p g L] + K_m K_e s} \quad (2.3)$$

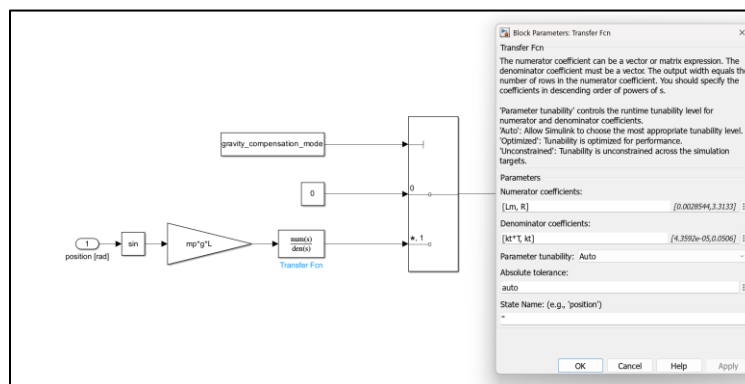
5. เมื่อกำหนดด้วย Disturbance Feed Forward จะได้ผลการคำนวณดังนี้

$$G_{DFF}(s) = \frac{G_D(s)}{G_p(s)} = \frac{V_{in}(s)}{T_{load}(s)} = \frac{L_m s + R_m}{K_m} \quad (2.4)$$

6. แต่เนื่องจากสมการ $G_{DFF}(s)$ นั้นมี Order ของ Zeroes มากกว่า Poles นั้นจึงไม่สามารถใช้เป็น Transfer Function ได้ จึงได้เพิ่ม Order ของ Poles ด้วยการใส่ Terms Low Pass Filter และ จัดรูปให้เหมือนดังสมการ (2.1) ได้ดังนี้ โดยให้ $r = 1$ เนื่องจาก Disturbance Feed Forward เป็นระบบ First Order โดยแทนให้ $\tau \approx \frac{L_m}{R_m} [s]$

$$G_{DFF}(s) = \frac{L_m s + R_m}{K_m} \cdot \left[\left(\frac{1}{\tau s + 1} \right)^1 \right] \quad (2.5)$$

7. นำสมการ $G_{DFF}(s)$ มาใช้ร่วมกับ Block Transfer Function ภายใน Simulink ดังนี้



รูปที่ 5 รูปวิธีการต่อ Block ระบบ Disturbance Feed Forward

ตอนที่ 2 การดำเนินการทดสอบและเก็บข้อมูล

1. แบ่งกรณีทดสอบเป็น 4 กรณีโดยมีการเลือกใช้ตัวแปรดังตารางนี้

ตารางที่ 3 ตาราง Parameter ที่ใช้ทดสอบ Gravity Compensation

กรณีการทดสอบ	K_p	$\theta_{init}[rad]$
System ที่ไม่มี DFF แล้วทดสอบปล่อย Free Fall	0	$\frac{\pi}{2}$
System มี DFF แล้วทดสอบปล่อย Free Fall	0	
System ที่ไม่มี DFF และ มี P Controller	0.5	
System ที่มี DFF และ มี P Controller	0.5	

2. จากการทดลองนี้ได้มีการบันทึกค่าตัวแปรดังนี้

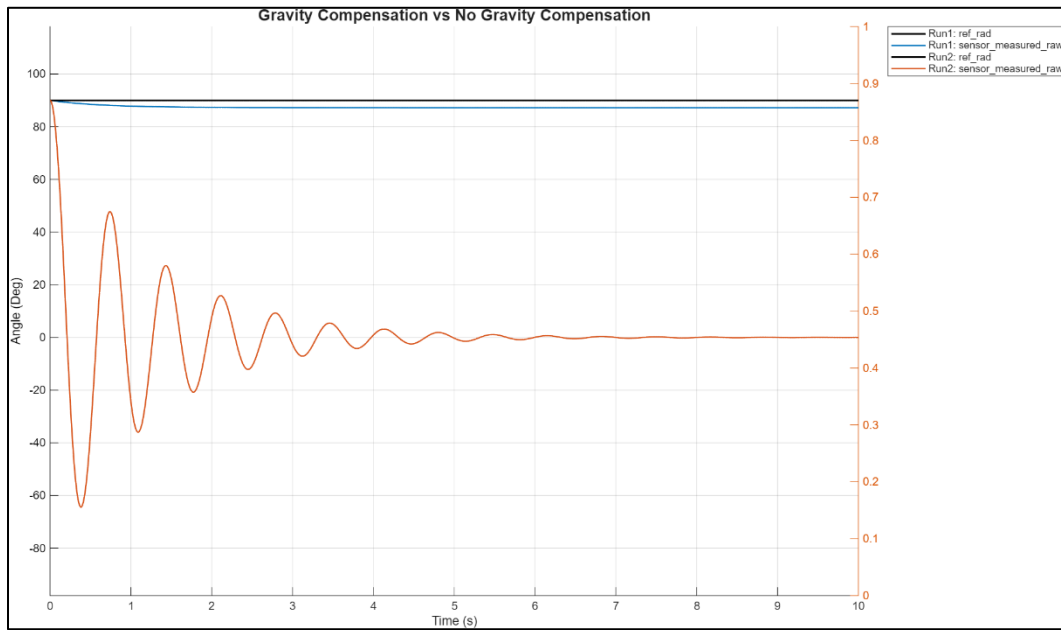
2.1. $\theta_{measured}[rad]$

2.2. $\theta_{ref}[rad]$

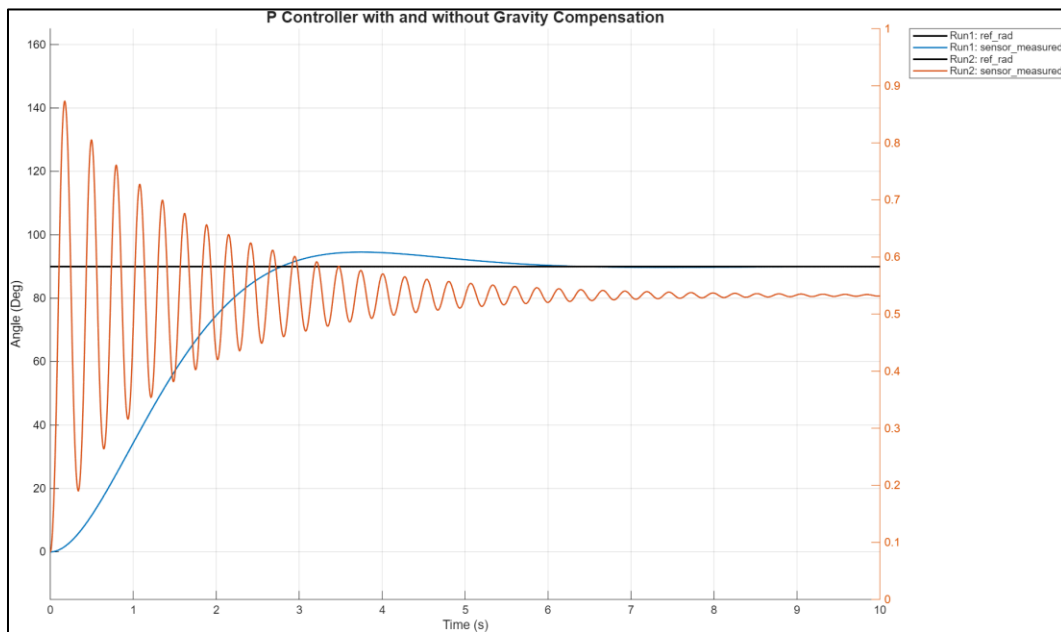
ตอนที่ 3 วิธีการวิเคราะห์ผล

1. นำค่า $\theta_{measured}$ มาเปรียบเทียบระหว่าง กรณีที่ 1 - 4 เพื่อวิเคราะห์ผลต่อในส่วนผลการทดลองโดยแบ่งเป็น 2 ประเด็นที่จะดำเนินการวิเคราะห์ผล
 - a. เปรียบเทียบกรณีที่ 1 และ 2 เพื่อดูการตอบสนองของ Gravity Compensation
 - b. เปรียบเทียบ Steady State Error ของระบบ P Controller ที่มี / ไม่มี Gravity Compensation

ผลการทดลอง



รูปที่ 6 ภาพเปรียบเทียบกรณีที่ 1 (สีส้ม) และ 2 (สีฟ้า) การทดสอบปล่อย Free Fall



รูปที่ 7 ภาพเปรียบเทียบกรณีที่ 3 (สีส้ม) และ 4 (สีฟ้า) เรื่อง Steady State Error ของระบบ P Controller

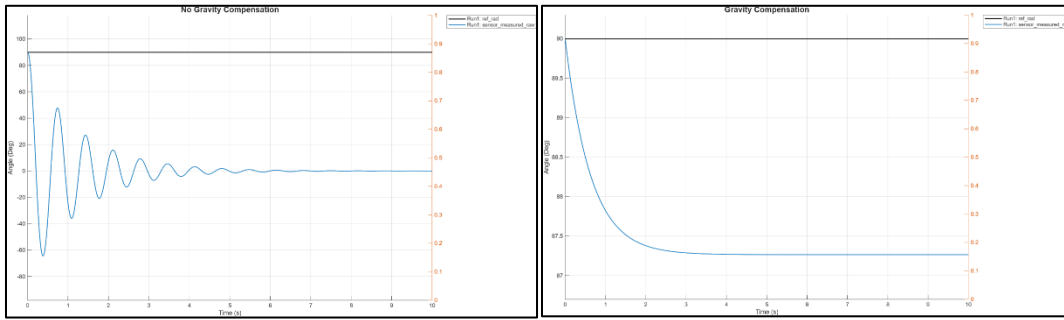
สรุปผลการทดลอง

จากการทดลองเกี่ยวกับ Gravity Compensation หรือเรียกว่า Disturbance Feed Forward (DFF) ภายในระบบ Controller นั้นสามารถสรุปออกมาได้ 2 ประเด็นได้แก่ ความสามารถในการชดเชยผลกระทบจาก Disturbance และ Steady State Error ของระบบ

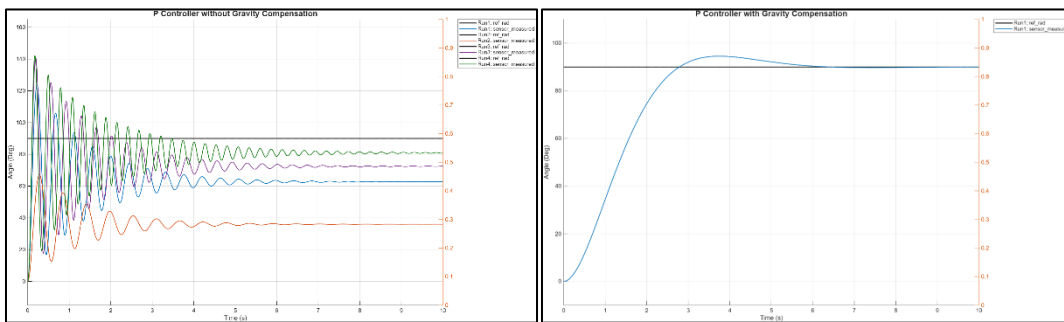
ประเด็นที่ 1 ความสามารถในการชดเชยผลกระทบจาก Disturbance จากภาพที่ 6 จะเห็นได้ว่า กรณีที่ 1 ซึ่งไม่มี DFF และ ไม่มีการจ่าย Voltage ใด ๆ เข้าสู่ Plant ซึ่งคือ DC Motor ที่มี Vertical Pendulum ก็จะสามารถสังเกตได้ว่าลักษณะของ $\theta_{measured}$ มีความ Oscillate เหมือน Free Fall Pendulum ในช่วง 0-8 วินาที และเริ่มเข้า Steady State หลัง 8 วินาที ไม่สามารถรักษาดำแหน่งเดิมของตัวเองได้ แต่หากพิจารณากรณีที่ 2 ที่มี DFF จะเห็นได้ว่าระบบ สามารถรักษาดำแหน่งเดิมเมื่อกำหนดให้ $\theta_{init} = \frac{\pi}{2} [rad]$ หรือ $90 [deg]$ ระบบจะมี $\theta_{measured} \approx 87.25 [deg]$ เนื่องจากความเป็น Non-Linear ของ Disturbance โดยสามารถสรุปในประเด็นนี้ได้ว่า การใช้ DFF จะช่วยให้ระบบชดเชยความคลาดเคลื่อนของ Disturbance ได้โดยไม่ต้องรอรับ Feedback ใด ๆ

ประเด็นที่ 2 Steady State Error ของระบบ ในการทดลองนี้ได้มีการใช้ P Controller มาเป็นส่วนหนึ่งในการทดสอบ DFF โดยพบว่า P Controller ที่ไม่ใช้ DFF จะเห็นพฤติกรรมการเกิด Overshoot, Oscillate และ Steady State Error โดยหากเพิ่มค่า K_p ก็จะทำให้มีขนาด Overshoot เพิ่มขึ้น และ Steady State Error ลดลงแต่ก็ไม่สามารถเข้าสู่ Setpoint ไป แต่หากมีการใช้ DFF พร้อมกับ P Controller จะพบว่า ระบบสามารถตอบสนองได้เสถียรมากกว่าดังรูปภาพที่ 7

อภิปรายผล



รูปที่ 8 การปล่อย Free Fall แบบมี DFF (ขวา) และไม่มี DFF (ซ้าย)



รูปที่ 9 รูปการใช้ P Controller แบบมี DFF (ขวา) และไม่มี DFF (ซ้าย)

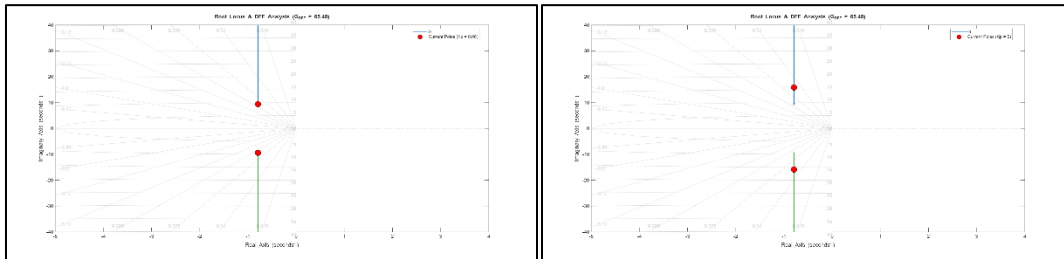
จากการสรุปผลเรื่อง Disturbance Feed Forward มี 3 ประเด็นที่สามารถอภิปราย ได้แก่ *Steady State Error* (E_{ss}) ตำแหน่งของ Poles และ ความไม่สมบูรณ์แบบของ Disturbance Feed Forward

ประเด็นที่ 1 *Steady State Error* (E_{ss}) จากรูปที่ 9 จะเห็นพฤติกรรมที่ การเพิ่มขนาด K_p โดยไม่ได้ใช้ DFF จะทำให้ระบบมีความเสถียรที่น้อยลง โดยสิ่งที่แตกต่างกันระหว่าง 2 ระบบนี้คือ

$$U_{plant}(s) = K_p e(s) + U_{DFF}(s) \quad (2.6)$$

$U_{DFF}(s)$ จะชดเชยค่าความคาดเคลื่อนจาก *Disturbance* โดยตรง ทำให้ระบบนี้มีลักษณะเหมือนมีเพียง *P Controller* ที่ไม่มี *Disturbance* จึงทำให้ไม่จำเป็นต้องใช้ K_p ที่มากจนทำให้ระบบไม่เสถียร

ประเด็นที่ 2 ตำแหน่งของ Poles หากพิจารณาที่ General Forms ของ Closed Loop Transfer Function จะพบว่า DFF ไม่มีการเพิ่ม Poles หรือ Zeroes ใด ๆ เพียงแต่ทำให้ K_p มีค่าที่ลดลงได้จาก Characteristics ของ Root Locus Path แบบนี้ จะทำให้ระบบมี Stability ที่ดีขึ้นเมื่อใช้ P Controller คู่กับ DFF



รูปที่ 10 ตำแหน่ง Poles ของ P Controller

ประเด็นที่ 3 ความไม่สมบูรณ์แบบของ Disturbance Feed Forward ในประเด็นนี้เกิดจากการทดลองเมื่อปล่อยระบบ Free Fall ตามการทดสอบในกรณีที่ 2 จะสังเกตเห็นได้ว่า ระบบไม่สามารถตอบสนองได้ในทันที จนทำให้เมื่อกำหนดให้ $\theta_{init} = \frac{\pi}{2} [rad]$ หรือ $90 [deg]$ ระบบจะมี $\theta_{measured} \approx 87.25 [deg]$ โดยเกิดจากขั้นตอนการทำ Disturbance Feed Forward โดยมีการกำหนดให้ $\sin \theta \approx \theta$ ซึ่งคือการทำ Linear Approximation เพื่อให้สามารถทำ Transfer Function ของ DFF ได้ เนื่องจาก Transfer Function เป็นระบบที่สามารถทำงานได้ใน Linear Time Invariant (LTI)

อ้างอิง

<https://x-engineer.org/proportional-controller/>

https://github.com/rubinsteina13/C_PID_CONTROLLERS_LIB

การทดลองที่ 3 การศึกษาพฤติกรรมของค่าคงที่ K_p ในระบบ P Controller

จุดประสงค์

1. เพื่อศึกษาพฤติกรรมของ K_p ที่ส่งผลต่อ P.O. [%]
2. เพื่อศึกษาพฤติกรรมของ K_p ที่ส่งผลต่อ Peaks Time (T_p) [s]
3. เพื่อศึกษาพฤติกรรมของ K_p ที่ส่งผลต่อ *Steady State Error* (E_{ss}) [rad]
4. เพื่อศึกษาความเป็น Scaling ของค่า K_p ต่อช่วง $\theta_{ref} = 0 - 2\pi$ [rad]

สมมติฐาน

ในการทำ P controller จะมีความสัมพันธ์กับ Reference Error ได้ดังสมการนี้ $u(t) = K_p e(t)$ เมื่อ Reference Error ($e(t) = r(t) - y(t)$) มีค่าเยอะจะส่งผลให้ค่า $u(t)$ มีค่ามากแบบเป็น Scaling และ ในสถานะที่ Reference Error มีค่าเท่ากัน ค่าคงที่ K_p หากใช้เยอะค่า $u(t)$ ก็จะเยอะด้วย ด้วยเหตุนี้การปรับค่าคงที่ ใน 2nd Order System จะส่งผลต่อ P.O. และ T_p แตกต่างกัน

ตัวแปร

1. ตัวแปรต้น:
 - ค่าคงที่ $K_p = \{0.1, 0.2, 0.5, 1, 2\}$
 - $\theta_{ref} = \{\frac{\pi}{6}, \frac{\pi}{3}, \frac{\pi}{2}, \frac{2\pi}{3}\}$ [rad]
2. ตัวแปรตาม:
 - P.O. [%]
 - Peaks Time (T_p) [s]
 - *Steady State Error* (E_{ss}) [rad]
3. ตัวแปรควบคุม:
 - ระบบนี้มี Disturbance Feed Forward
 - DC Motor Parameter ใช้ค่าจากการทดลองดังตารางที่ 1

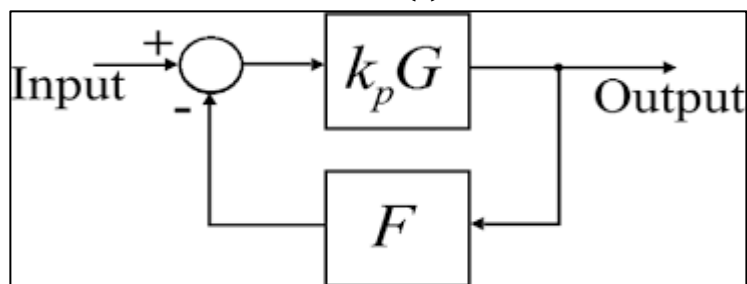
เอกสารและงานวิจัยที่เกี่ยวข้อง

1. **P Controller ใน Closed Loop System** คือ ระบบที่ควบคุมปรับค่าสัญญาณให้แปรผันตรงกับค่าerror ระหว่าง ค่าที่วัดได้จาก Output เทียบกับค่า Set Point โดยหากระบบยังอยู่ห่างจากเป้าหมายมาก ตัวควบคุมจะสั่งการด้วยสัญญาณที่แรงเพื่อให้ระบบตอบสนองอย่างรวดเร็ว และจะค่อยๆ ลดขนาดการสั่งการลงตามสัดส่วนเมื่อระบบเข้าใกล้เป้าหมาย แสดงได้ในสมการและรูปภาพดังนี้

$$u(t) = K_p e(t)$$

และเมื่อแปลงให้อยู่ในรูปของโดเมนความถี่ (s-domain) จะได้สมการดังนี้

$$C(s) = \frac{U(s)}{E(s)} = K_p$$



ซึ่งหากพิจารณาทั้งระบบแบบป้อนกลับที่มีฟังก์ชันถ่ายโอนของกระบวนการคือ $G(s)$ และฟังก์ชันถ่ายโอนของเซ็นเซอร์คือ $H(s)$ ฟังก์ชันถ่ายโอนของระบบวงปิดโดยรวม (Closed-Loop Transfer Function) จะเขียนได้ ดังนี้

$$\frac{Y(s)}{R(s)} = \frac{K_p G(s)}{1 + K_p G(s) H(s)}$$

2. **2nd Order System Response** คือ ระบบทางพลศาสตร์ที่มีลักษณะเด่นคือ เมื่อได้รับการกระตุ้นระบบมักจะแสดงพฤติกรรมการแกว่งตัว (Oscillation) ก่อนที่จะค่อยๆ ลดทอนลงจนหยุดนิ่งและเข้าสู่สภาวะเป้าหมาย พฤติกรรมนี้อธิบายได้ด้วย Transfer Function ในรูปแบบดังนี้

$$G(s) = \frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2}$$

ซึ่งในการวิเคราะห์และการประเมินประสิทธิภาพของระบบอันดับสองมักจะพิจารณาจากพฤติกรรมของระบบเมื่ออยู่ในสภาวะหน่วงต่ำ เนื่องจากเป็นสภาวะที่ระบบสามารถตอบสนองเข้าหาเป้าหมายได้อย่างรวดเร็ว ซึ่งจะนำพารามิเตอร์หลักของระบบมาใช้คำนวณหาค่าชี้วัดหาประสิทธิภาพเมื่อมีการป้อนสัญญาณทดสอบแบบขั้นบันไดได้ 4 พฤติกรรม ดังนี้

Maximum Percent Overshoot คือ ระยะเวลาที่ระบบวิ่งทะลุเป้าหมาย คำนวณได้จากสมการ ดังนี้

$$P.O. = e^{-\left(\frac{\zeta\pi}{\sqrt{1-\zeta^2}}\right)} \times 100$$

Peak Time คือระยะเวลาที่ระบบใช้ในการเดินทางตั้งแต่เริ่มต้นจนไปถึงจุดที่พุ่งเกินสูงสุด คำนวณได้จากสมการ ดังนี้

$$T_p = \frac{\pi}{\omega_n \sqrt{1-\zeta^2}}$$

Settling Time คือระยะเวลาที่ระบบใช้ในการลดการแกว่ง จนกระทั่งสัญญาณในระบบนิ่ง คำนวณได้จากสมการ ดังนี้

$$T_s = \frac{4}{\zeta\omega_n}$$

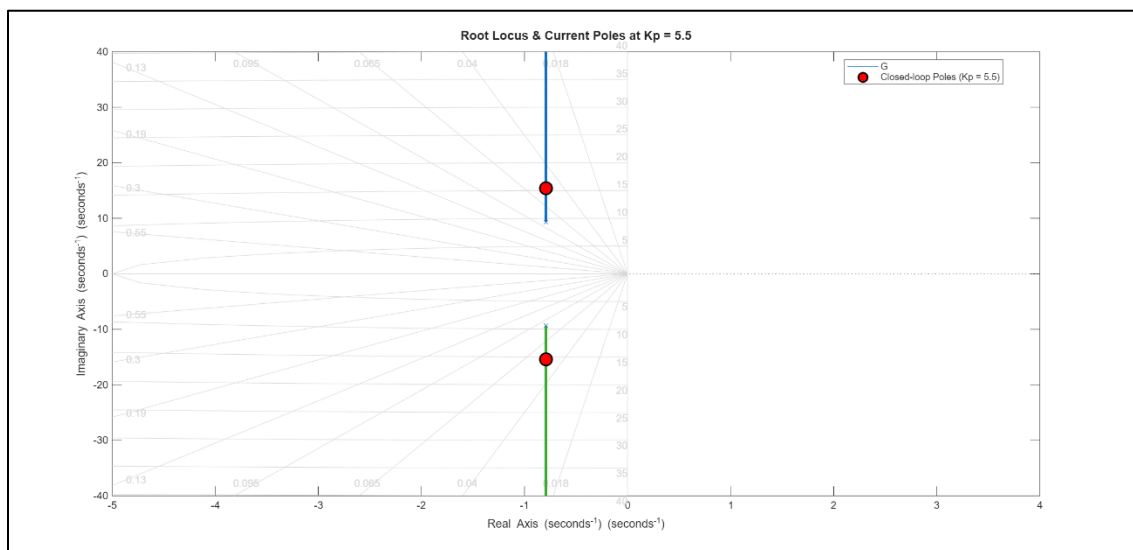
ขั้นตอนการดำเนินงาน

ตอนที่ 1 การวิเคราะห์ P Controller ที่ส่งผลต่อ Characteristics ของ DC Motor ในระบบ 2nd Order System Response

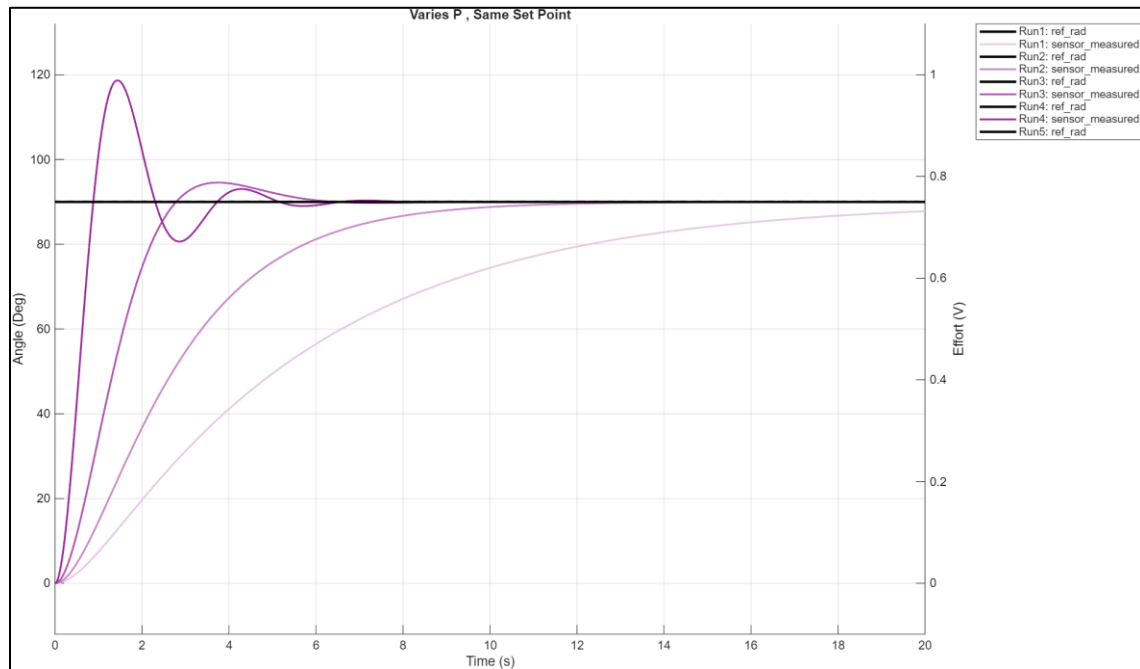
1. P Controller ใน Closed Loop Controller นั้นมีสมการทั่วไปดังนี้

$$Y(s) = \frac{G_p(s)K_p}{1 + G_p(s)K_p} \quad (2.7)$$

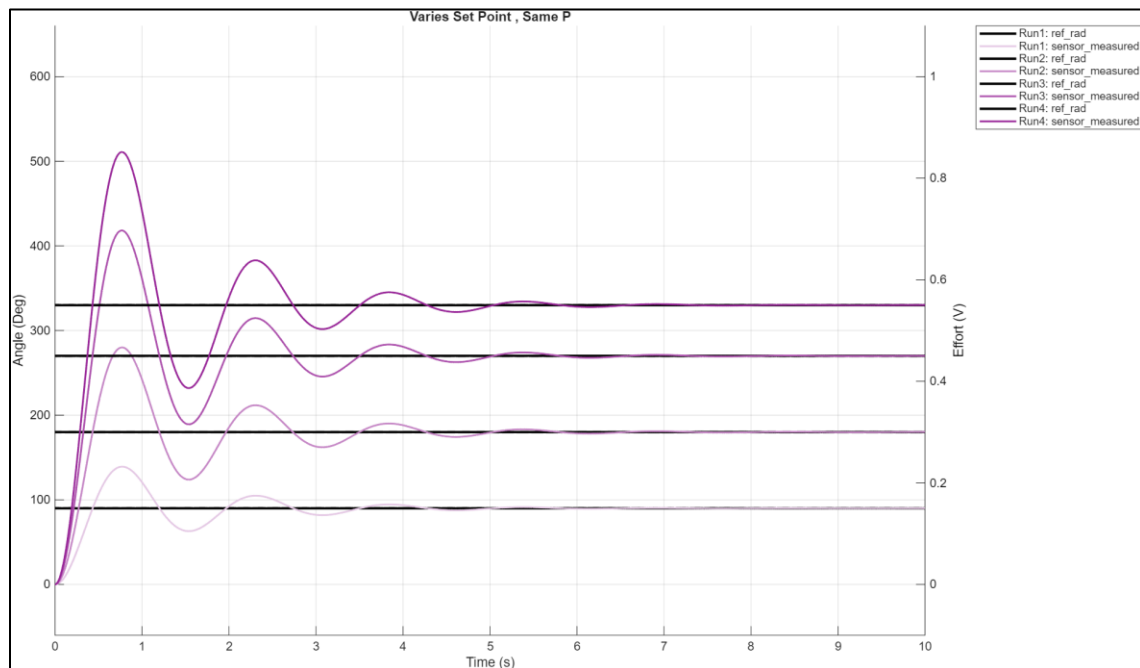
2. พิจารณาสมการ Characteristics จะเป็น $\Delta s = 1 + K_p G_p(s)$ เมื่อ $G_p(s)$ เป็นระบบ 2nd Order System Response จะพบว่า ใน Root Locus Path ของ P Controller จะเป็นดังรูปภาพนี้



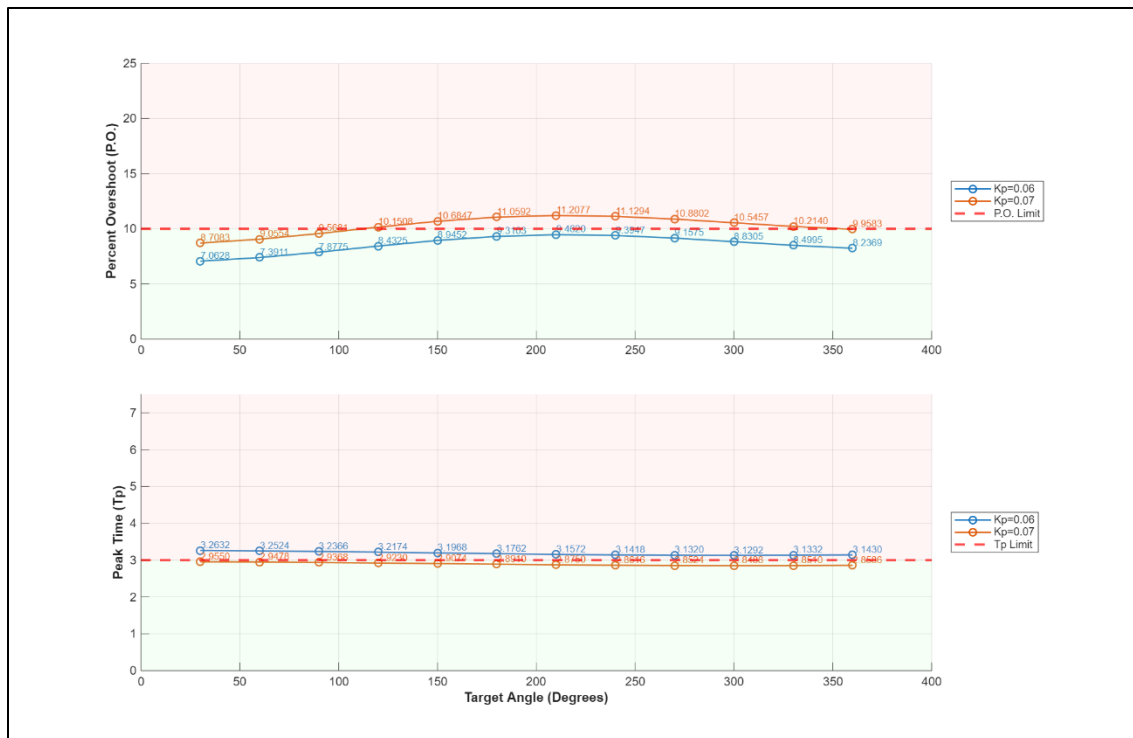
ผลการทดลอง



รูปที่ 12 กราฟเปรียบเทียบ System Response จาก K_p ที่เปลี่ยนไป เมื่อมี Setpoint เท่าเดิม



รูปที่ 13 กราฟเปรียบเทียบ System Response จาก Setpoint ที่เปลี่ยนไปเมื่อมี K_p เท่าเดิม



รูปที่ 14 กราฟเปรียบเทียบความแตกต่างระหว่าง Overshoot Percentage [%] และ Peak Time (Tp)

สรุปผลการทดลอง และ อภิปรายผล

จากการทดลอง Closed Loop P Controller สามารถอธิบายลักษณะการตอบสนองของ K_p ได้ 3 ประเด็น ได้แก่ ความแตกต่างของ System Response ใน K_p ที่แตกต่างกัน การตอบสนองของ P Controller ที่ Setpoint ใด ๆ การคงที่และ ความสัมพันธ์ของ $P.O. [\%]$ กับ $T_p [s]$

ประเด็นที่ 1 ความแตกต่างของ System Response ใน K_p ที่แตกต่างกัน คือ จากรูปที่ 10 จะเห็นได้ว่า K_p ที่แตกต่างกัน จะส่งผลให้มีลักษณะของ System Response แตกต่างกัน สาเหตุนี้เกิดจากสมการ (2.7) หากพิจารณาสมการ Characteristics จะเป็น $\Delta s = 1 + K_p G_p(s)$ เมื่อ $G_p(s)$ เป็นระบบ 2nd Order System Response จะพบว่า ใน Root Locus Path ของ P Controller จะนี้แนวโน้ม Oscillate มากขึ้น

ประเด็นที่ 2 การตอบสนองของ P Controller ที่ Set Point ใด ๆ คือ การใช้ P Controller จะสามารถสังเกตเห็นพฤติกรรมของ $T_p [s]$ มีขนาดที่คงที่ ดังสมการข้อ (2.9)

ประเด็นที่ 3 ความสัมพันธ์ของ $P.O. [\%]$ กับ $T_p [s]$ คือ มีความสัมพันธ์แบบแปรผกผันกัน ก็คือ หาก $P.O. [\%]$ มีค่ามากขึ้น $T_p [s]$ ก็จะมีค่าน้อยลง โดยสามารถสังเกตเห็นพฤติกรรมนี้ได้จาก รูปที่ 12 โดยสาเหตุนี้เกิดจากสมการดังนี้

$$P.O. = 100 \times e^{\left(-\frac{\zeta\pi}{\sqrt{1-\zeta^2}}\right)} \quad (2.8)$$

$$T_p = \frac{\pi}{\omega_n \sqrt{1-\zeta^2}} \quad (2.9)$$

อ้างอิง

[https://eng.libretexts.org/Bookshelves/Industrial_and_Systems_Engineering/Introduction_to_Control_Systems_\(Iqbal\)/04%3A_Control_System_Design_Objectives/4.04%3A_Disturbance_Rejection](https://eng.libretexts.org/Bookshelves/Industrial_and_Systems_Engineering/Introduction_to_Control_Systems_(Iqbal)/04%3A_Control_System_Design_Objectives/4.04%3A_Disturbance_Rejection)
<https://www.electronics-tutorials.ws/systems/closed-loop-system.html>

การทดลองที่ 4 การศึกษาพฤติกรรมของค่าคงที่ K_I ในระบบ PI Controller

จุดประสงค์

1. เพื่อศึกษาพฤติกรรมของ K_I ที่ส่งผลต่อ $P.O.$ [%]
2. เพื่อศึกษาพฤติกรรมของ K_I ที่ส่งผลต่อ $Steady State Error (E_{ss})$ [rad]
3. เพื่อศึกษาการเกิด Integration Windup ที่เกิดจาก K_I ในระบบ Controller

สมมติฐาน

ในการใช้ PI controller จะมีความสัมพันธ์กับ Reference Error ได้ดังสมการนี้ $u(t) = K_p e(t) + K_I \int e(t)dt$ เมื่อ Reference Error ($e(t) = r(t) - y(t)$) พจน์ $K_I \int e(t)dt$ จะมีพฤติกรรมสะสมขนาดของ Feedback มากขึ้นเรื่อย ๆ เพื่อลด Reference Error ให้เข้าใกล้ 0 เมื่อ $t = \infty$ แต่หาก Setpoint มีค่าที่มากจนระบบไม่สามารถตอบสนองได้ทัน ก็จะส่งผลให้ พจน์ $K_I \int e(t)dt$ สะสมค่ามากขึ้นจนมีค่ามากกว่าขีดจำกัดของ Actuator ซึ่งจะส่งผลให้ Recovery Time ของระบบต้องใช้เวลามากขึ้น จนระบบไม่สามารถตอบสนองได้ในเวลาอันสั้น หรือส่งผลให้ $P.O.$ [%] และ Settling Time มากขึ้น จึงเป็นที่มาของ Integration Windup

ตัวแปร

1. ตัวแปรต้น:
 - ค่าคงที่ $K_I = 0.5$
 - สถานะของระบบ Anti Windup (ปิด/เปิด)
2. ตัวแปรตาม:
 - $\theta_{measured}$ [rad]
 - แรงดัน Voltage ที่อยู่ในพจน์ K_I [v]
3. ตัวแปรควบคุม:
 - ระบบนี้ไม่มี Disturbance Feed Forward
 - DC Motor Parameter ใช้ค่าจากการทดลองดังตารางที่ 1
 - ค่าคงที่ $K_p = 0.3$
 - $\theta_{ref} = \frac{11}{6} \pi$ [rad]

เอกสารและงานวิจัยที่เกี่ยวข้อง

1. **I Controller ใน Closed Loop System** คือ 1 ในรูปแบบของ Closed Loop Controller โดยมีสมการทั่วไปว่า

$$\text{Time Domain} \quad u(t) = K_I \int_0^t e(t) dt \quad (2.10)$$

$$\text{S Domain} \quad G_c(s) = \frac{K_I}{s} \quad (2.11)$$

โดยลักษณะพฤติกรรมของ I Controller จะทำให้ *Steady State Error* (E_{ss}) = 0 ในเชิงของการตอบสนองของระบบ แต่หากพิจารณาจากระบบ S Domain จะพบว่าการใช้ I จะเป็นการเติม Poles เข้าที่จุด (0,0) บน S Plane

2. **Integration Windup** คือ ปรากฏการณ์ของ I Controller เมื่อมี Setpoint ที่มีค่ามากเข้าสู่ระบบ ซึ่งหากระบบไม่สามารถตอบสนอง หรือเข้าสู่ Set Point ได้ทันในระยะเวลาอันสั้นก็จะทำให้ I Controller สะสมค่า Feedback มากขึ้น แต่หากค่า Feedback นั้นมีขนาดมากกว่าขีดจำกัดของ Actuator จนสามารถเกิด Overshoot ที่มากขึ้น หรือ หาก Set Point เปลี่ยนไประบบก็ไม่สามารถตอบสนองได้ทันทำให้เกิด Setling Time และ Recovery Time ที่มากขึ้น

ขั้นตอนการดำเนินงาน

ตอนที่ 1 การดำเนินการทดสอบและเก็บข้อมูล

1. แบ่งกรณีทดสอบเป็น 4 กรณีโดยมีการเลือกใช้ตัวแปรดังตารางนี้

ตารางที่ 4 ตารางกรณีการทดสอบ PI Controller

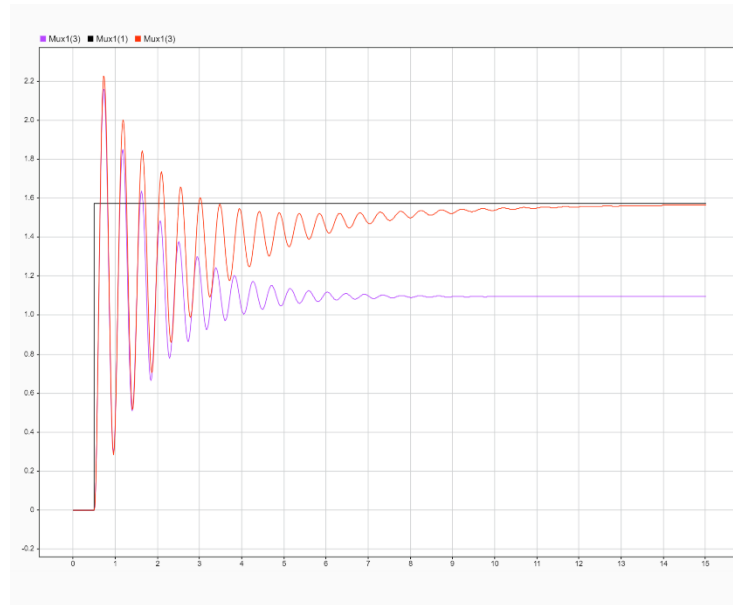
กรณีการทดสอบ	K_P	K_I	$\theta_{ref}[rad]$
P Controller	0.3	0	$\frac{11}{6}\pi$
PI Controller	0.3	0	
PI Controller แบบไม่มี Anti-Windup	0.3	0.5	
PI Controller แบบมี Anti-Windup	0.3	0.5	

2. โดยขั้นตอนการทดลอง กรณีที่ 3 และ 4 คือ การล๊อคมอเตอร์ไม่ให้มอเตอร์หมุน เพื่อให้ Integration Terms สะสมค่าต่อไปเมื่อถึงวินาทีที่ 10 ก็จะปล่อยมอเตอร์ให้หมุนเข้าสู่ Set Point
3. จากการทดลองนี้ได้มีการบันทึกค่าตัวแปรดังนี้
 - 3.1. $\theta_{measured}[rad]$
 - 3.2. $V_{KI}[V]$

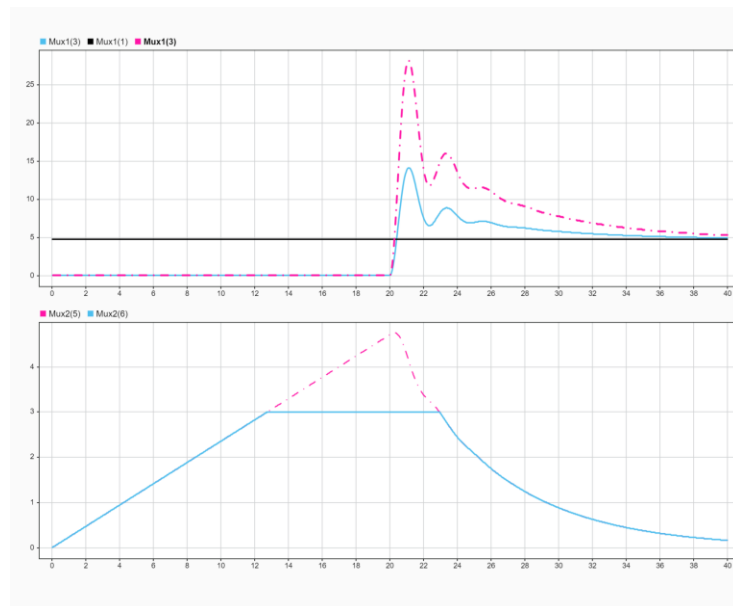
ตอนที่ 2 วิธีวิเคราะห์ผล

2. นำค่า $\theta_{measured}$ มาเปรียบเทียบระหว่าง กรณีที่ 1 - 4 เพื่อวิเคราะห์ผลต่อในส่วนผลการทดลองโดยแบ่งเป็น 2 ประเด็นที่จะดำเนินการวิเคราะห์ผล
 - a. เปรียบเทียบกรณีที่ 1 และ 2 เพื่อดูการเข้าสู่ Steady State ระหว่าง P และ PI Controller
 - b. เปรียบเทียบกรณีที่ 3 และ 4 เรื่อง Integration Windup และ System Response

ผลการทดลอง



รูปที่ 15 ภาพเปรียบเทียบ P (สีส้ม) และ PI Controller (สีชมพู)



รูปที่ 16 ภาพเปรียบเทียบ PI Controller ที่มี Anti Windup (สีฟ้า) และ ไม่มี Anti Windup (สีชมพู)

สรุปผลการทดลอง และ อภิปรายผล

จากผลการทดลองเรื่องพฤติกรรมของ K_I สามารถอธิบายได้ใน 2 ประเด็นได้แก่ การลู่เข้าสู่ Steady State ระหว่าง P และ PI Controller และ เรื่อง Integration Windup และ System Response

ประเด็นที่ 1 การลู่เข้าสู่ Steady State ระหว่าง P และ PI Controller คือจากสมการที่ (2.11) พบว่า ใน System Characteristics การมี Integration Terms จะส่งผลให้ Steady State Error ลู่เข้าใกล้ 0 ในขณะที่ P Controller อย่างเดียวจะมี Steady State Error คงที่ หากเปรียบเทียบถึง Characteristics ก็พบว่า P Controller และ PI Controller มีสมการแตกต่างกันดังนี้

$$P \text{ Controller} \quad \frac{Y(s)}{R(s)} = \frac{K_p G(s)}{1 + K_p G(s)} \quad (2.12)$$

$$P \text{ Poles and Zeroes} \quad \text{ไม่มีเพิ่ม} \quad (2.13)$$

$$PI \text{ Controller} \quad \frac{Y(s)}{R(s)} = \frac{(K_p s + K_I) G(s)}{s + (K_p s + K_I) G(s)} \quad (2.14)$$

$$PI \text{ Poles and Zeroes} \quad \begin{aligned} p &= \{0 + 0j\} \\ z &= \left\{ -\frac{K_I}{K_p} \right\} \end{aligned} \quad (2.15)$$

หากพิจารณาถึงตำแหน่ง Poles จะได้มีผลดังภาพนี้ จะพบว่า ระบบ PI Controller มี Poles เพิ่มอีก 1 จุดที่ตำแหน่ง $0 + 0j$ ซึ่งหมายความว่า ระบบจะมี Steady State Error ลู่เข้าใกล้ 0

ประเด็นที่ 2 Integration Windup และ System Response เนื่องจาก Integration Term สามารถสะสมค่า Feedback มากขึ้นหากระบบยังมี Error อยู่ เมื่อถึงจุดที่ Feedback มีค่ามากกว่าที่ Actuator จะสามารถทำงานได้ หรือ เรียกว่า Saturation และ ยังสะสมค่า Feedback ต่อไปเรื่อย ๆ จะทำให้ระบบมี System Response ที่ช้าลงมากกว่า ดังรูปที่ 16

อ้างอิง

<https://youtu.be/tbgV6caAVcs?si=HjJr6t63y-ypBjUu>

<https://instrumentationtools.com/cascade-control-principle/>

การทดลองที่ 5 การศึกษาพฤติกรรมของค่าคงที่ K_D ในระบบ PD Controller

จุดประสงค์

1. เพื่อศึกษาผลกระทบเรื่อง Noise Sensitivity ของ K_D
2. เพื่อศึกษาการใช้ Filter เพื่อให้ Noise Sensitivity มีผลกระทบน้อยลง

สมมติฐาน

ความสัมพันธ์ของ K_d และ Error มีความสัมพันธ์ดังสมการนี้ $u(t) = K_d \frac{de(t)}{dt}$ หาก $e(t)$ มีอัตราการเปลี่ยนแปลงในระยะเวลาสั้น ๆ จะทำให้ $u(t) \approx \infty$ จากเหตุนี้หากระบบนี้มี Noise เข้าสู่ระบบ ก็จะทำให้การตอบสนองของ $u(t)$ ไม่สามารถทำงานได้ตามที่ออกแบบได้

ตัวแปร

1. ตัวแปรต้น:
 - $K_D = \{0.1, 0.2, 0.3, 0.4, 0.5\}$
 - $Filter\ Coefficient(N) = \{10, 100, 1000\}$
 - $\theta_{ref}[rad] = \{0, \frac{\pi}{2}\}$
2. ตัวแปรตาม:
 - System Response
 - $\theta_{measured}[rad]$
3. ตัวแปรควบคุม:
 - $K_P = 0.06$
 - ระบบนี้มี Disturbance Feed Forward

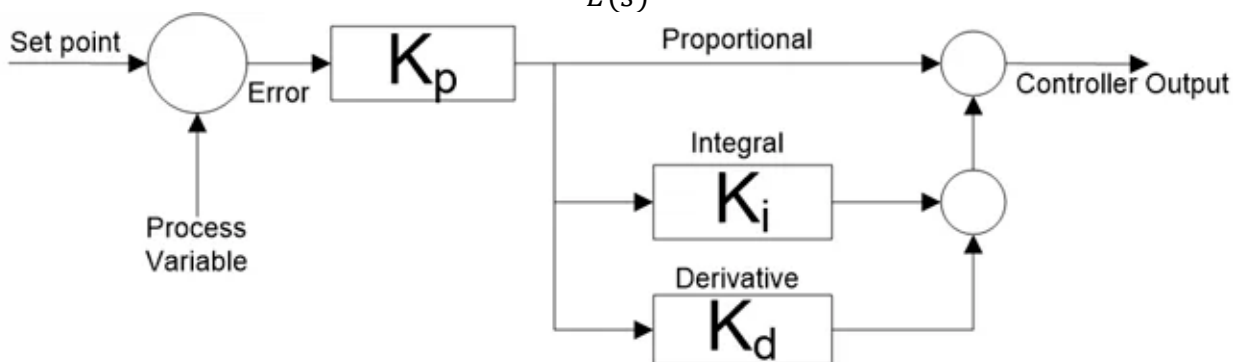
เอกสารและงานวิจัยที่เกี่ยวข้อง

1. **D Controller ใน Closed Loop System** คือองค์ประกอบที่ทำหน้าที่ตอบสนองต่ออัตราการเปลี่ยนแปลงของความคลาดเคลื่อนโดยเป้าหมายหลักคือการเพิ่มความหน่วง (Damping) ให้กับระบบ เพื่อลด Overshoot และช่วยให้ระบบเข้าสู่ Steady State ได้รวดเร็วยิ่งขึ้นแต่มีข้อจำกัดสำคัญคือความไวต่อสัญญาณรบกวน (Noise) ซึ่งอาจทำให้สัญญาณควบคุมสั่นพ้องได้หากไม่มีการกรองสัญญาณที่เหมาะสม

$$u(t) = K_d \frac{de(t)}{dt}$$

และเมื่อแปลงให้อยู่ใน s-domain จะได้สมการดังนี้

$$C(s) = \frac{U(s)}{E(s)} = K_d s$$



ซึ่งหากพิจารณาทั้งระบบแบบป้อนกลับที่มีฟังก์ชันถ่ายโอนของกระบวนการคือ $G(s)$ และฟังก์ชันถ่ายโอนของเซนเซอร์คือ $H(s)$ โดยใช้ตัวควบคุมแบบพีไอดี (PID Controller)

$$\frac{Y(s)}{R(s)} = \frac{(K_d s^2 + K_p + K_i)G(s)}{s + (K_d s^2 + K_p + K_i)G(s)H(s)}$$

2. **Low Pass Filter** คือวงจรคัดกรองสัญญาณที่ยอมให้สัญญาณที่มีความถี่ต่ำกว่าค่า Cut-off Frequency ผ่านไปได้ ใช้เพื่อกำจัดสัญญาณรบกวน (Noise) ที่มีความถี่สูงซึ่งปนมากับเซนเซอร์ เพื่อให้ได้สัญญาณที่เสถียรก่อนนำไปประมวลผลต่อ

$$G(s) = \frac{1}{\tau s + 1}$$

ขั้นตอนการดำเนินงาน

ตอนที่ 1 การดำเนินการทดสอบและเก็บข้อมูล

1. แบ่งกรณีทดสอบเป็น 2 รูปแบบการทดสอบโดยมีการเลือกใช้ตัวแปรดังตารางนี้

ตารางที่ 5 ตารางกรณีการทดสอบ PD Controller เรื่อง Filter Coefficient (N)

การทดสอบที่ 1	K_P	K_D	$\theta_{ref}[rad]$
P Controller	0.06	0.00	0
PD Controller (n=100)	0.06	0.01	
PD Controller (n=100)	0.06	0.02	
PD Controller (n=100)	0.06	0.03	
PD Controller (n=100)	0.06	0.04	
PD Controller (n=100)	0.06	0.05	

ตารางที่ 6 ตารางกรณีการทดสอบ PD Controller เรื่อง Filter Coefficient (N)

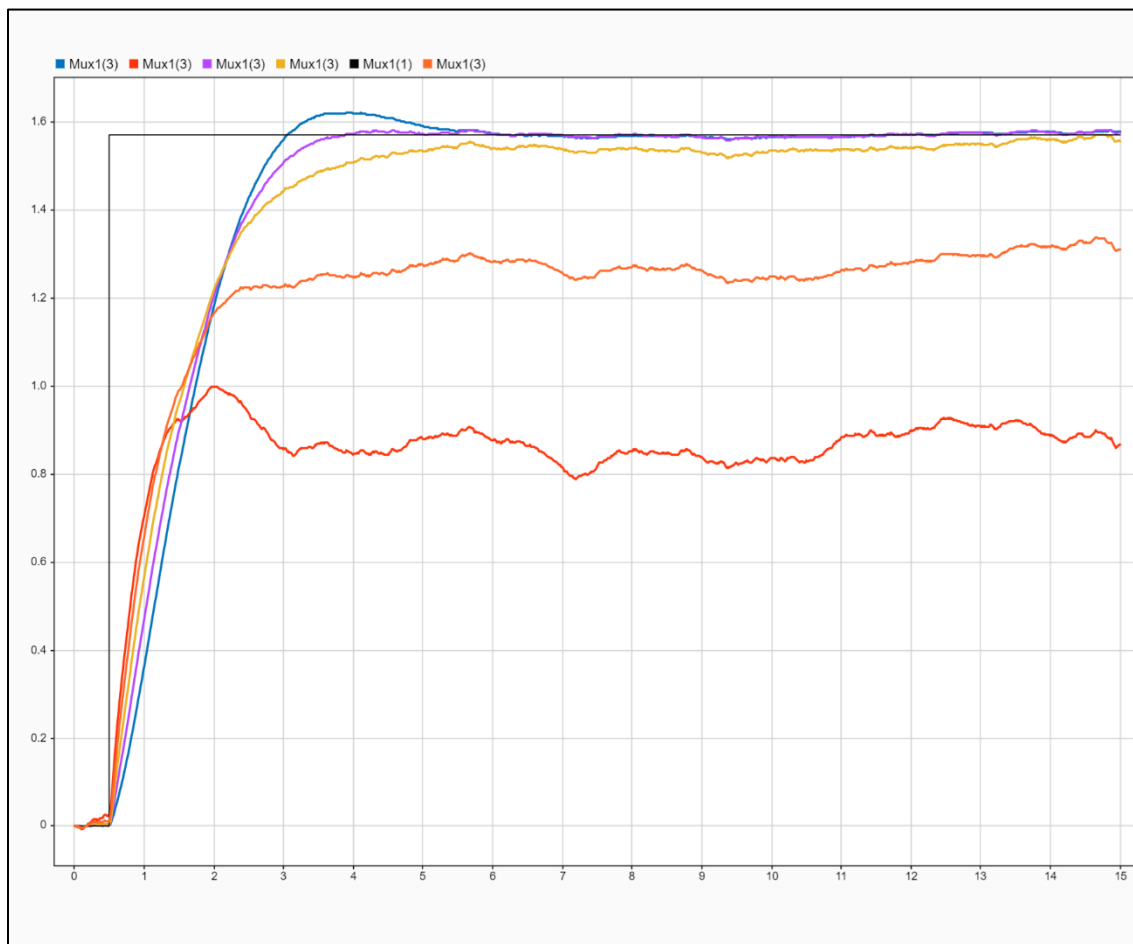
การทดสอบที่ 2	K_P	K_D	$\theta_{ref}[rad]$
P Controller	0.06	0.00	$\frac{\pi}{2}$
PD Controller (n=10)	0.06	0.02	
PD Controller (n=100)	0.06	0.02	
PD Controller (n=1000)	0.06	0.02	

2. โดยขั้นตอนการทดลอง ทั้ง 2 ชุดการทดสอบคือ การใช้ Random Number ให้ทำหน้าที่แทนที่ Random Noise เข้าสู่ระบบ Sensor
3. จากการทดลองนี้ได้มีการบันทึกค่าตัวแปรดังนี้
 - 3.1. $\theta_{measured}[rad]$
 - 3.2. $V_{KP}[V]$
 - 3.3. $V_{KD}[V]$

ตอนที่ 2 วิธีการวิเคราะห์ผล

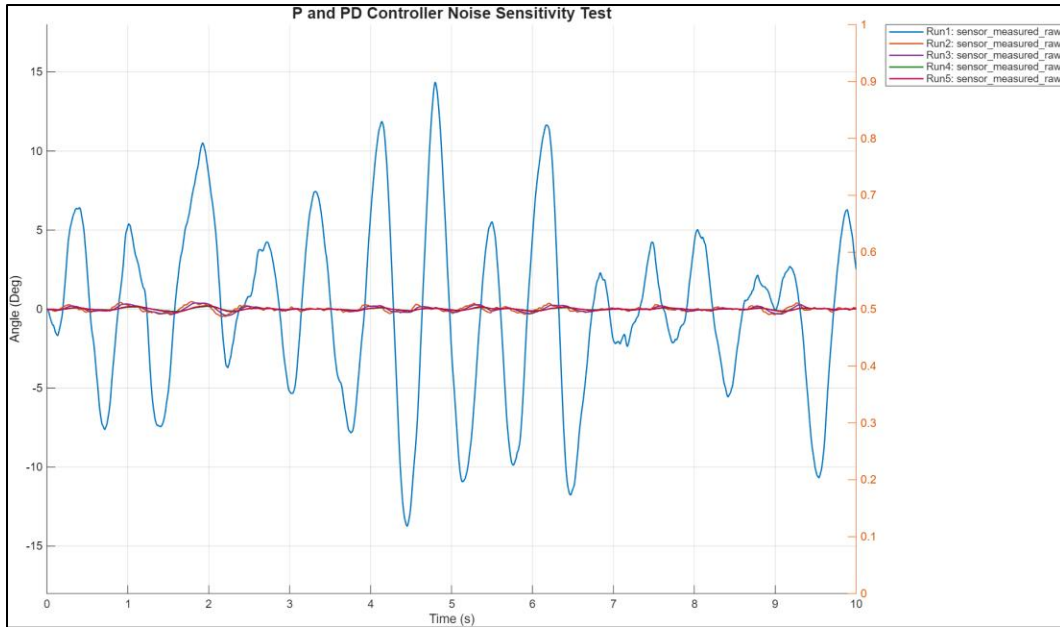
3. นำค่า $\theta_{measured}$, $V_{KP}[V]$, $V_{KD}[V]$ มาเปรียบเทียบระหว่าง 2 การทดสอบ เพื่อวิเคราะห์ผลต่อในส่วนผลการทดลองโดยแบ่งเป็น 3 ประเด็นที่จะดำเนินการวิเคราะห์ผล
 - a. ในการทดสอบที่ 1 จะสังเกตการตอบสนองของ $\theta_{measured}$
 - b. ในการทดสอบที่ 2 จะสังเกตการตอบสนองของ $V_{KP}[V]$, $V_{KD}[V]$
 - c. ในการทดสอบที่ 2 จะสังเกตการตอบสนองของ $\theta_{measured}$

ผลการทดลอง



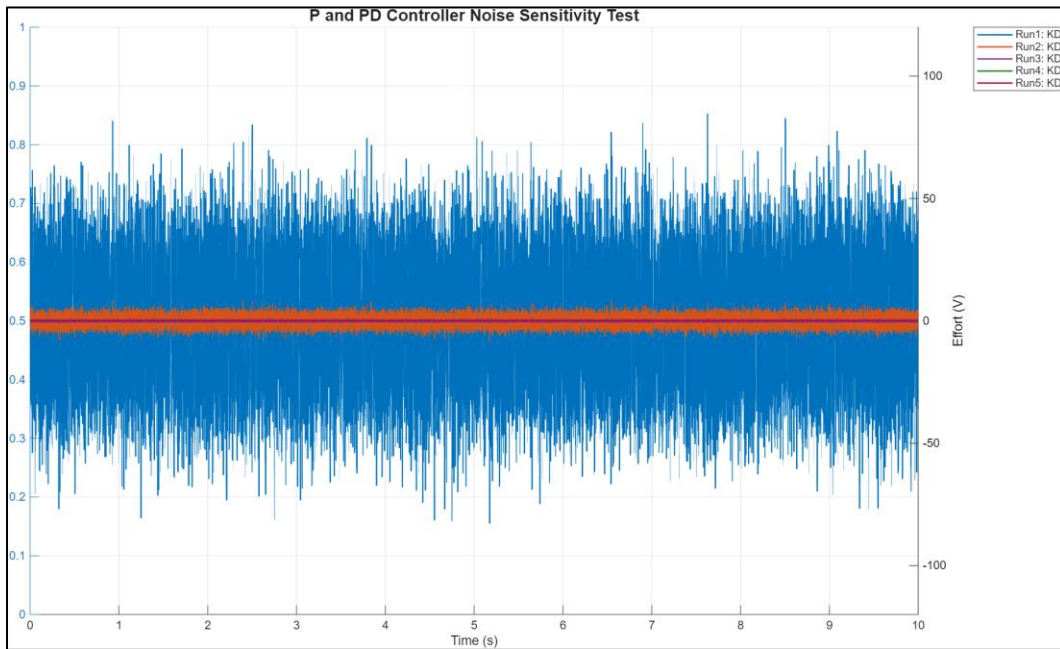
หมายเหตุ : สีแดง (Kd = 0.05) สีส้ม (Kd=0.04) สีเหลือง (Kd = 0.03) สีม่วง (Kd = 0.02) สีฟ้า (Kd = 0.01)

รูปที่ 17 ภาพเปรียบเทียบ Kd



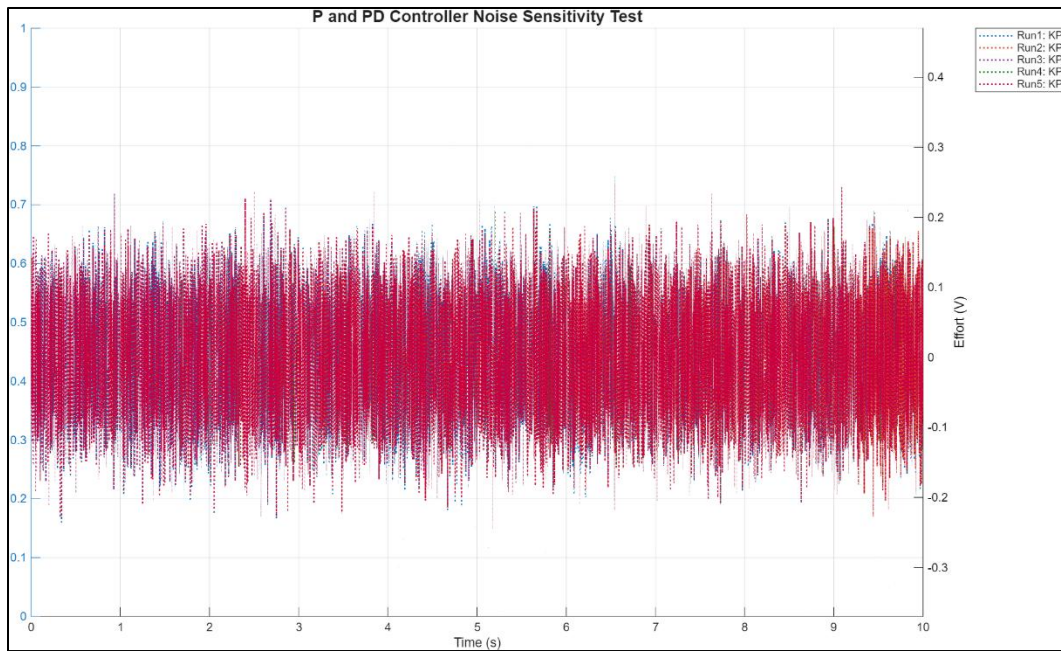
หมายเหตุ : สีฟ้า (N=10) สีม่วง (N=100) สีเขียว (N=1000) สีแดง (KP = 0.06, KD = 0.0)

รูปที่ 18 ภาพการตอบสนองของ $\theta_{measured}$ จากชุดการทดลองที่ 2



หมายเหตุ : สีฟ้า (N=10) สีม่วง (N=100) สีเขียว (N=1000) สีแดง (KP = 0.06, KD = 0.0)

รูปที่ 19 ภาพการตอบสนองของ $V_{KD}[V]$ จากชุดการทดลองที่ 1



รูปที่ 20 ภาพการตอบสนองของ $V_{KP}[V]$ จากชุดการทดลองที่ 1

สรุปผลการทดลอง และ อภิปรายผล

จากผลการทดลองจาก 2 ชุดการทดลองสามารถสรุปผลและอภิปรายได้ใน 2 ประเด็น ได้แก่ Noise Sensitivity ของ K_d และ การใช้ Filter Coefficient (N)

ประเด็นที่ 1 Noise Sensitivity ของ K_d คือ จากภาพที่ 17 ซึ่งคือการทดสอบ Unit Step Function ของระบบ PD Controller ก็พบว่า ในสถานการณ์ที่มี Noise และ K_p เท่ากัน และ K_d แตกต่างกันไป K_d มีค่ามากเท่าไรยิ่งตอบสนองต่อ Noise มากเท่านั้น

ประเด็นที่ 2 การใช้ Filter Coefficient (N) คือ จากภาพที่ 18 และ 19 จะสามารถสังเกตได้ว่า K_d นั้น มีตอบสนองตาม Noise ที่เข้าสู่ระบบ โดยมีพิจารณาภาพที่ 20 ซึ่งคือ K_p จะพบว่า K_p ไม่ได้มีอัตราการขยายตัวได้มากกว่า K_d

จากทั้ง 2 ประเด็นนั้นมีผลลัพธ์การตอบสนองที่เกิดมาดังสมการนี้ จะเห็นได้ว่า พจน์ $\frac{N}{1+N\frac{1}{s}}$ มีคุณสมบัติเป็น Filter เพื่อกรอง Noise ก่อน

$$\text{From PID Tuners} \quad P + D \frac{N}{1 + N \frac{1}{s}} \quad (2.16)$$

อ้างอิง

<https://youtu.be/tbgV6caAVcs?si=HjJr6t63y-ypBjUu>

<https://instrumentationtools.com/cascade-control-principle/>

<https://circuitdigest.com/article/what-is-pid-controller-working-structure-applications>

การทดลองที่ 6 การศึกษาพฤติกรรมของระบบใน Continuous Domain และ Discrete Domain

จุดประสงค์

1. เพื่อเปรียบเทียบความแตกต่างของระบบ Controller ที่ทำงานใน Continuous Domain และ Discrete Domain
2. เพื่อศึกษาผลกระทบของ Sample Time ของ Discrete Domain ที่

สมมติฐาน

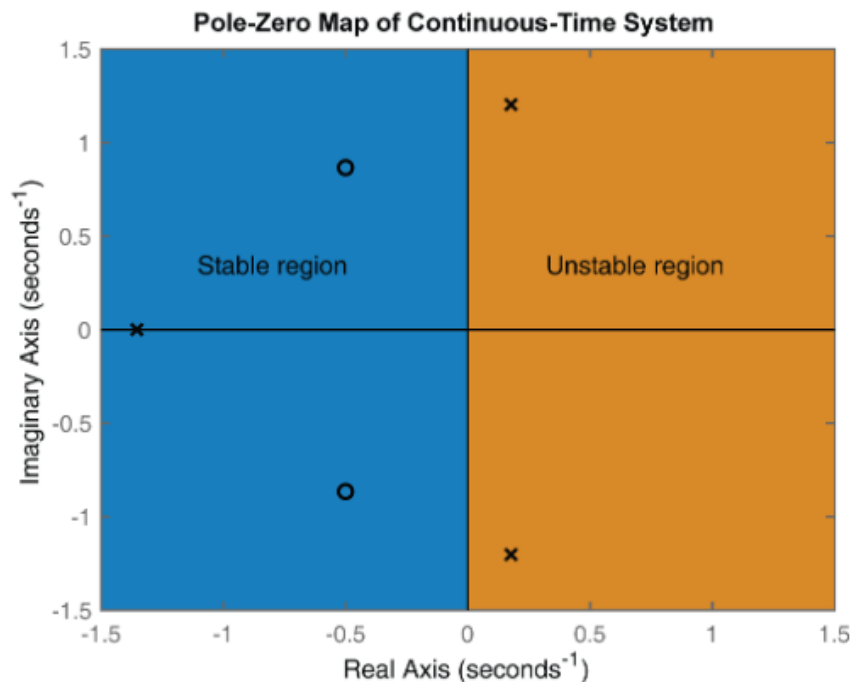
การใช้ Disturbance feed forward และ P controller ส่งผลทำให้ระบบมีความเสถียรที่แตกต่างกัน

ตัวแปร

1. ตัวแปรต้น:
 - Sample Time [Hz] = {1,10,1000}
 - PID(s)
 - PID(z), Forward Euler
 - PID(z), Backward Euler
 - PID(z), Tustin
2. ตัวแปรตาม:
 - System Response
3. ตัวแปรควบคุม:
 - K_p
 - K_I
 - K_D
 - ระบบนี้มี Disturbance Feed Forward

เอกสารและงานวิจัยที่เกี่ยวข้อง

1. **S-Plane** คือ ระนาบจำนวนเชิงซ้อน (Complex Plane) ที่ใช้วิเคราะห์พฤติกรรมและความเสถียรของระบบผ่านการแปลงลาปลาซ ดังสมการ $s = \sigma + j\omega$
โดยมีองค์ประกอบหลักคือ แกนจริง(σ)ในแนวตั้งที่บ่งบอกถึงการขยายตัวหรือการลดลงของสัญญาณ (Damping) และ แกนจินตภาพ($j\omega$)ในแนวนอนที่แทนความถี่ของการสั่น



จากรูปจะแสดงถึงหัวใจสำคัญของการวิเคราะห์บนระนาบนี้คือการหาค่า Poles (ขั้ว) ซึ่งเป็นค่าที่ทำให้ระบบมีค่าเป็นอนันต์ และ Zeros (ศูนย์) ที่ทำให้ระบบเป็นศูนย์ โดยตำแหน่งของ Poles จะเป็นตัวตัดสิน ความเสถียร (Stability) ของระบบโดยตรง หาก Poles ทั้งหมดวางตัวอยู่ทาง ฝั่งซ้าย (Left-Half Plane) ระบบจะถือว่าเสถียรและสามารถกลับเข้าสู่สภาวะสมดุลได้ แต่หากมีเพียงตัวเดียวหลุดไปอยู่ทาง ฝั่งขวา (Right-Half Plane) ระบบจะสูญเสียการควบคุมและไม่เสถียรทันที ส่วนกรณีที่ Poles อยู่บน แกน $j\omega$ พอดีระบบจะเกิดการสั่นค้างแบบคงที่ซึ่งเรียกว่าสภาวะก้ำกึ่ง (Marginally Stable)

ขั้นตอนการดำเนินงาน

ตอนที่ 1 การดำเนินการทดสอบและเก็บข้อมูล

1. แบ่งกรณีทดสอบเป็น 2 ชุดการทดสอบโดยมีการเลือกใช้ตัวแปรดังตารางนี้

ตารางที่ 7 ตารางทดสอบ PID(s), PID(z)

การทดสอบที่ 1	Sample Time [Hz]
PID(s)	1000
PID(z), Forward Euler	1000
PID(z), Backward Euler	1000
PID(z), Tustin	1000

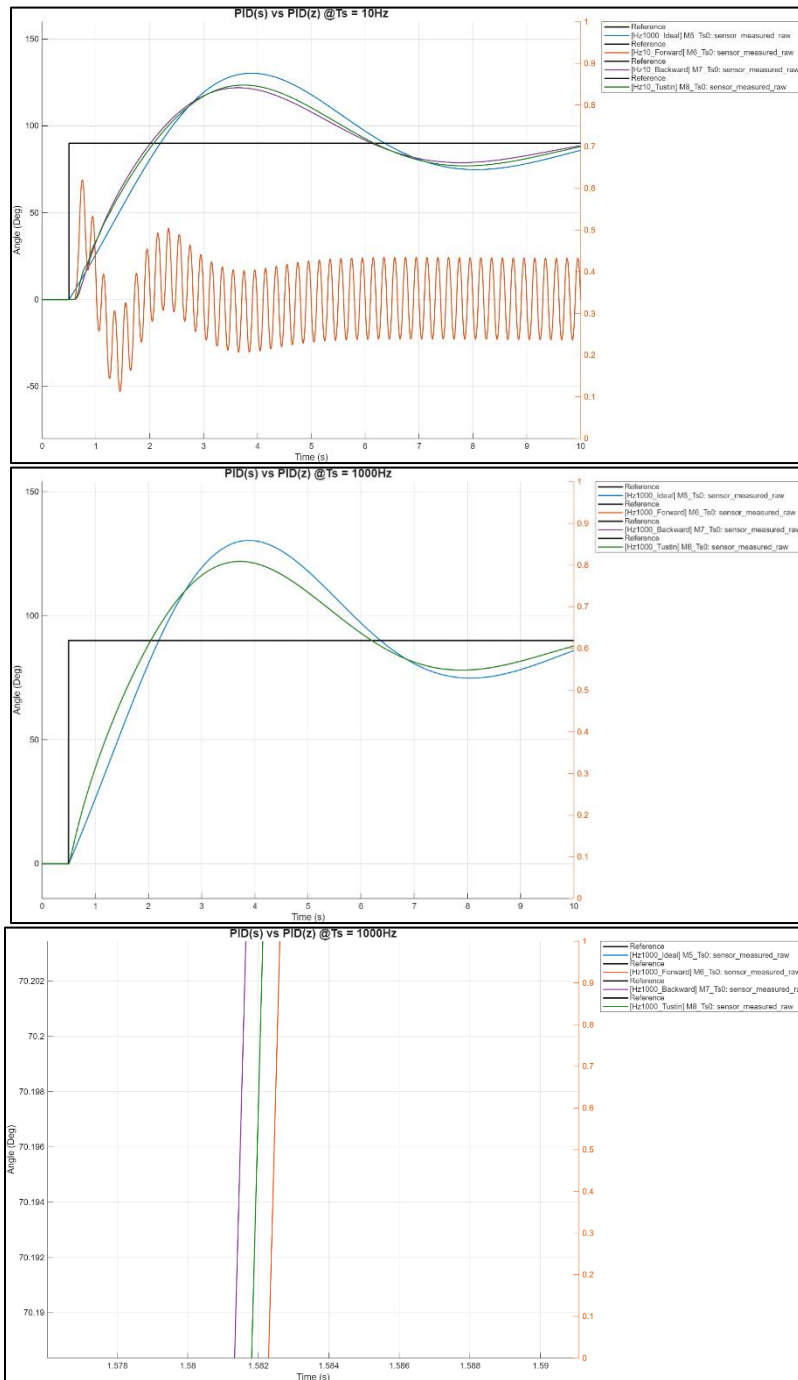
ตารางที่ 8 ตารางทดสอบ PID(z) และ Sample Time [Hz]

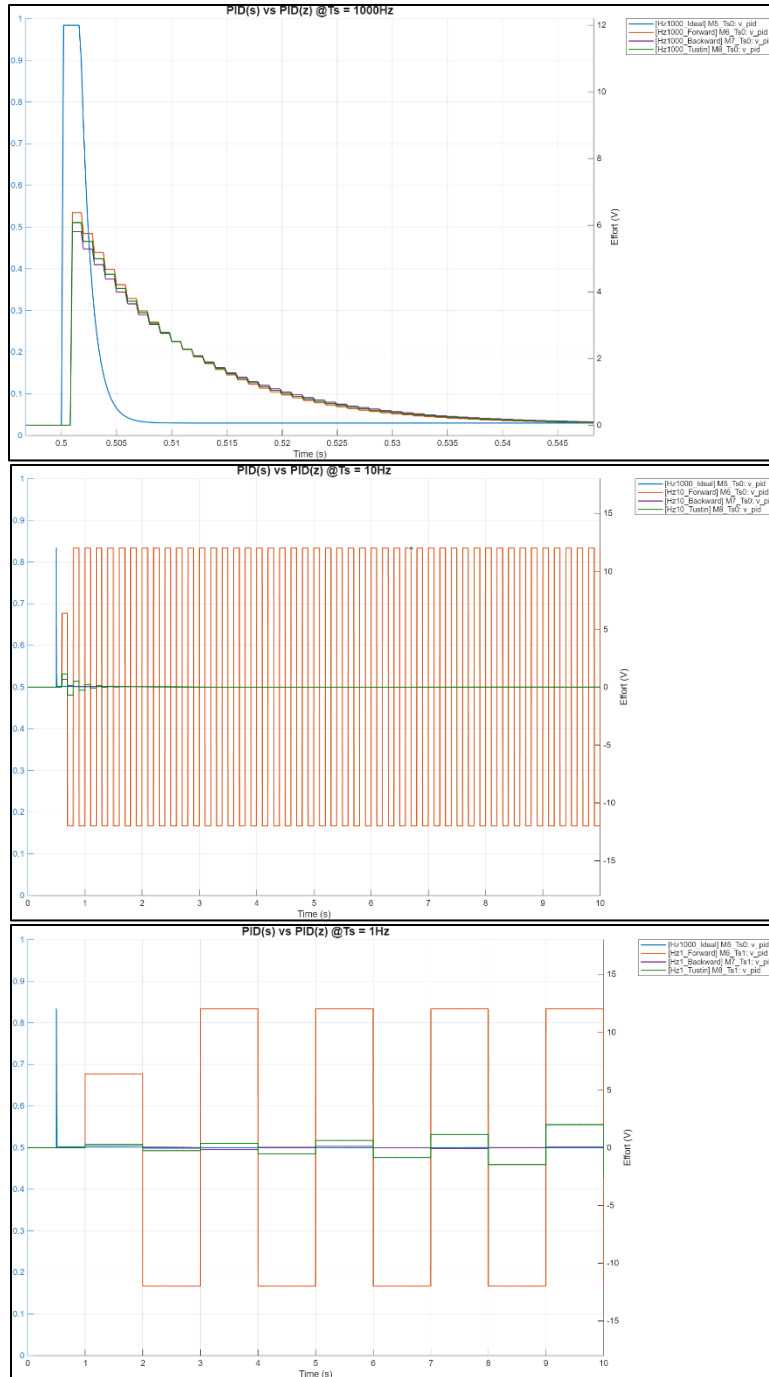
การทดสอบที่ 2	Sample Time [Hz]
PID(z), Forward Euler	1000
PID(z), Backward Euler	1000
PID(z), Tustin	1000
PID(z), Forward Euler	10
PID(z), Backward Euler	10
PID(z), Tustin	10
PID(z), Forward Euler	1
PID(z), Backward Euler	1
PID(z), Tustin	1

ตอนที่ 2 วิธีการวิเคราะห์ผล

1. นำค่า $\theta_{measured}$, V_{PID} [V] มาเปรียบเทียบระหว่าง 2 การทดสอบ เพื่อวิเคราะห์ผลต่อในส่วนผลการทดลองโดยแบ่งเป็น 2 ประเด็นที่จะดำเนินการวิเคราะห์ผล
 - a. ในการทดสอบที่ 1 จะสังเกตการตอบสนองของ $\theta_{measured}$
 - b. ในการทดสอบที่ 2 จะสังเกตการตอบสนองของ V_{PID} [V]

ผลการทดลอง





สรุปผลการทดลอง และ อภิปรายผล

ที่ความถี่ต่างกัน เมื่อนำการตอบสนองของมอเตอร์ที่ผ่านการ Discrete ด้วยวิธีการต่าง ๆ มาเปรียบเทียบ จะสามารถสังเกตได้ว่าความถี่มีผลต่อการตอบสนองของมอเตอร์ เมื่อความถี่ยิ่งมากจะทำให้การตอบสนองแต่ละแบบออกมาใกล้เคียงกันมากขึ้น โดยที่ความถี่ 1000 Hz การตอบสนองของมอเตอร์ที่เกิดขึ้นของ Discretization ทั้ง 3 แบบ มีลักษณะของกราฟที่เกือบทับซ้อนกันมีความคลาดเคลื่อนเพียงเล็กน้อย แต่เมื่อต้องนำมาเปรียบเทียบกับ การตอบสนองในอุดมคติที่อยู่ในรูปแบบ Continuous System กลับมีค่าที่ต่างกันถึงแม้จะใช้ค่า PID เดียวกัน

เมื่อเปลี่ยนความถี่เป็น 10 Hz จะสามารถเห็นการตอบสนองของ Discretization ประเภทต่าง ๆ แยกออกมาจากกันอย่างชัดเจนโดยเฉพาะ Forward Estimation ที่มีแนวโน้มที่ไม่ใกล้เคียงกับประเภทอื่น ๆ เพราะ การ Discrete ด้วยวิธีการ Forward ต้องทำด้วย Sampling Time ที่ไม่ใหญ่มากหรือในทางกลับกันคือการที่มีความถี่มากขึ้น ไม่เช่นนั้นระบบจะไม่เสถียร ในขณะที่วิธีการอื่น ๆ โดยทั่วไปแล้วจะมีความเสถียรอยู่แล้ว

การตอบสนองของแรงดันไฟฟ้าก็มีแนวโน้มที่คล้ายกัน คือเมื่อความถี่มีค่า 1000 Hz การตอบสนองของแรงดันไฟฟ้ามีแนวโน้มที่จะลักษณะคล้ายกับกราฟที่ควรจะเป็นในอุดมคติ แต่กราฟในอุดมคติเป็นกราฟที่เกิดการตอบสนองขึ้นเพียงช่วงเวลาสั้น ๆ เมื่อเปลี่ยนความถี่เป็น 10 หรือ 1 Hz ความละเอียดของข้อมูลที่ได้รับมานั้นต่ำมาก ทำให้มีความถูกต้องที่ต่ำมาก เนื่องจากการทำ Discrete เป็นการประมาณค่าโดยนำค่าบางตำแหน่งของทั้งหมดมาใช้ขึ้นอยู่กับการ Sampling Time ดังนั้นการ Discrete อาจไม่เหมาะสมกับค่าที่มีลักษณะการตอบสนองเพียงช่วงสั้น ๆ หรือหากนำมาใช้ควรเลือก Sampling Time ที่เหมาะสม

อ้างอิง

<https://youtu.be/tbgV6caAVcs?si=HjJr6t63y-ypBjUu>

<https://instrumentationtools.com/cascade-control-principle/>

การทดลองที่ 7 การศึกษาผลกระทบต่อระบบ Cascade Control จาก Transition Rate ความถี่ของ Inner Loop และ Outer Loop และ ระบบที่เปลี่ยนแปลงไปจากตำแหน่งการวาง Saturation และ Transition Block ภายใน Cascade Loop Control

จุดประสงค์

1. เพื่อศึกษาผลกระทบของความถี่ระหว่าง Inner Loop และ Outer Loop
2. เพื่อศึกษาระบบที่เปลี่ยนแปลงไปจากตำแหน่งการวาง Saturation
3. เพื่อศึกษาระบบที่เปลี่ยนแปลงไปจากตำแหน่งการวาง Transition Block

สมมติฐาน

ในการทำ Cascade Control นั้นจะประกอบด้วย 2 Controller ได้แก่ สำหรับ Position Controller และ Velocity Controller โดยที่ Input ของ Velocity Controller จะเกิดมา output ของ Position Controller และ ความถี่ระหว่าง Inner Loop และ Outer Loop ควรมีอัตราส่วนที่มากกว่า

ตัวแปร

1. ตัวแปรต้น:
 - Sampling Ratio = {5, 2, 1, 0.5, 0.2, 0.1}
 - Sampling Rate [Hz] = {1, 10, 100, 1000}
 - Saturation Block (มี / ไม่มี)
 - Transition Block (มี / ไม่มี)
2. ตัวแปรตาม:
 - System Response
 - P.O. [%]
 - Absolute Tracking Error [rad]
3. ตัวแปรควบคุม:
 - K_p
 - K_I
 - K_D
 - ระบบนี้มี Disturbance Feed Forward

เอกสารและงานวิจัยที่เกี่ยวข้อง

1. **Cascade Control** คือการควบคุมกระบวนการที่ใช้ Loop การควบคุมสองวงซ้อนกัน เพื่อเพิ่มประสิทธิภาพในการจัดการกับสัญญาณรบกวน (Disturbances) ก่อนที่สัญญาณนั้นจะส่งผลกระทบต่อตัวแปรหลักของกระบวนการ โดยการทำงาน 2 loops ได้แก่

1. Primary Controller

ทำหน้าที่ควบคุม Primary Variable ที่เราต้องการให้ได้ค่าตาม Setpoint โดยเอาต์พุตของตัวควบคุมนี้จะกลายเป็นค่า Setpoint ให้กับตัวควบคุมวงใน

2. Secondary Controller

ทำหน้าที่ควบคุม Secondary Variable ซึ่งเป็นตัวแปรที่มีการตอบสนองเร็วกว่า Primary Variable และมักจะได้รับผลกระทบจากสัญญาณรบกวนโดยตรง โดยการทำงานจะเป็นดังนี้

Primary Controller จะคำนวณหาค่าความผิดพลาดจาก Setpoint หลัก แล้วส่งเอาต์พุตเอาต์พุตกลายเป็น Setpoint ให้กับ Secondary Controller

Secondary Controller จะรักษาสถานะของตัวแปรรองให้ได้ตามที่ Primary Controller สั่งมา โดยการปรับอุปกรณ์ควบคุมให้ทำงานตาม Set point

2. **Reference Feed Forward (RFF)** คือการวัดค่า ตัวแปรรบกวน (Disturbances) ที่เกิดขึ้นก่อนจะส่งผลกระทบต่อค่าที่ควบคุม (Process Variable) แล้วทำการชดเชยค่าดังกล่าวผ่านตัวควบคุมในทันที เพื่อรักษาเสถียรภาพของระบบก่อนที่ความคลาดเคลื่อน (Error) จะเกิดขึ้นจริง

Transfer function:

$$Y(s) = G_p(s)U(s) + G_d(s)D(s)$$

การหาค่า Feed forward controller:

เพื่อให้ผลกระทบจาก $D(s)$ ต่อ $Y(s)$ เป็น 0 เมื่อพิจารณา $U(s) = G_{ff}(s)D(s)$ จะได้ว่า

$$G_p(s)[G_{ff}(s)D(s)] + G_d(s)D(s) = 0$$

ขั้นตอนการดำเนินงาน

ตอนที่ 1 การดำเนินการทดสอบและเก็บข้อมูล

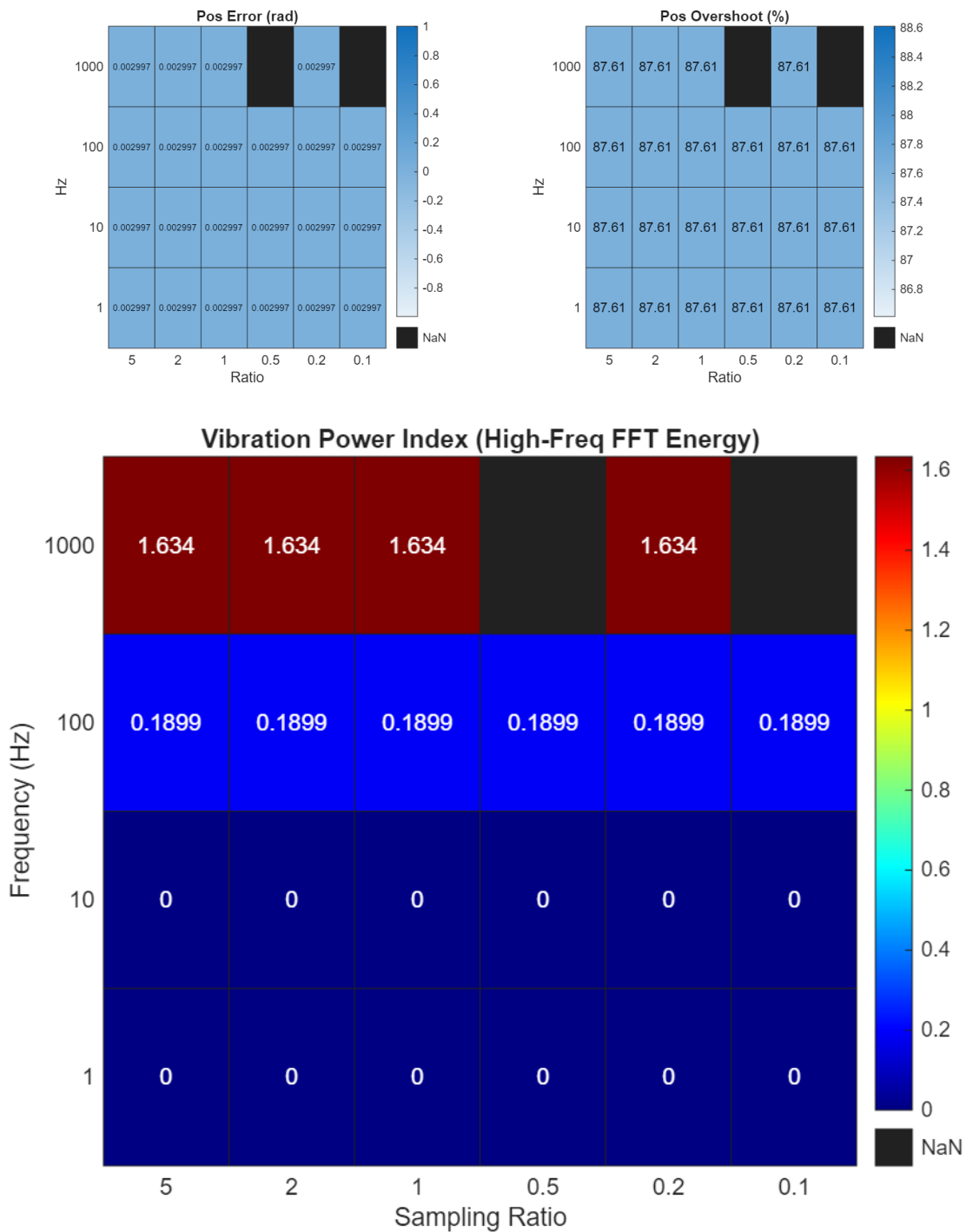
1. แบ่งกรณีทดสอบเป็น 1 ชุดการทดสอบโดยทำการ Cross Test ระหว่าง 4 ตัวแปรนี้
 - 1.1. Sampling Ratio = {5, 2, 1, 0.5, 0.2, 0.1}
 - 1.2. Sampling Rate [Hz] = {1, 10, 100, 1000}
 - 1.3. Saturation Block (มี / ไม่มี)
 - 1.4. Transition Block (มี / ไม่มี)

ตอนที่ 2 วิธีการวิเคราะห์ผล

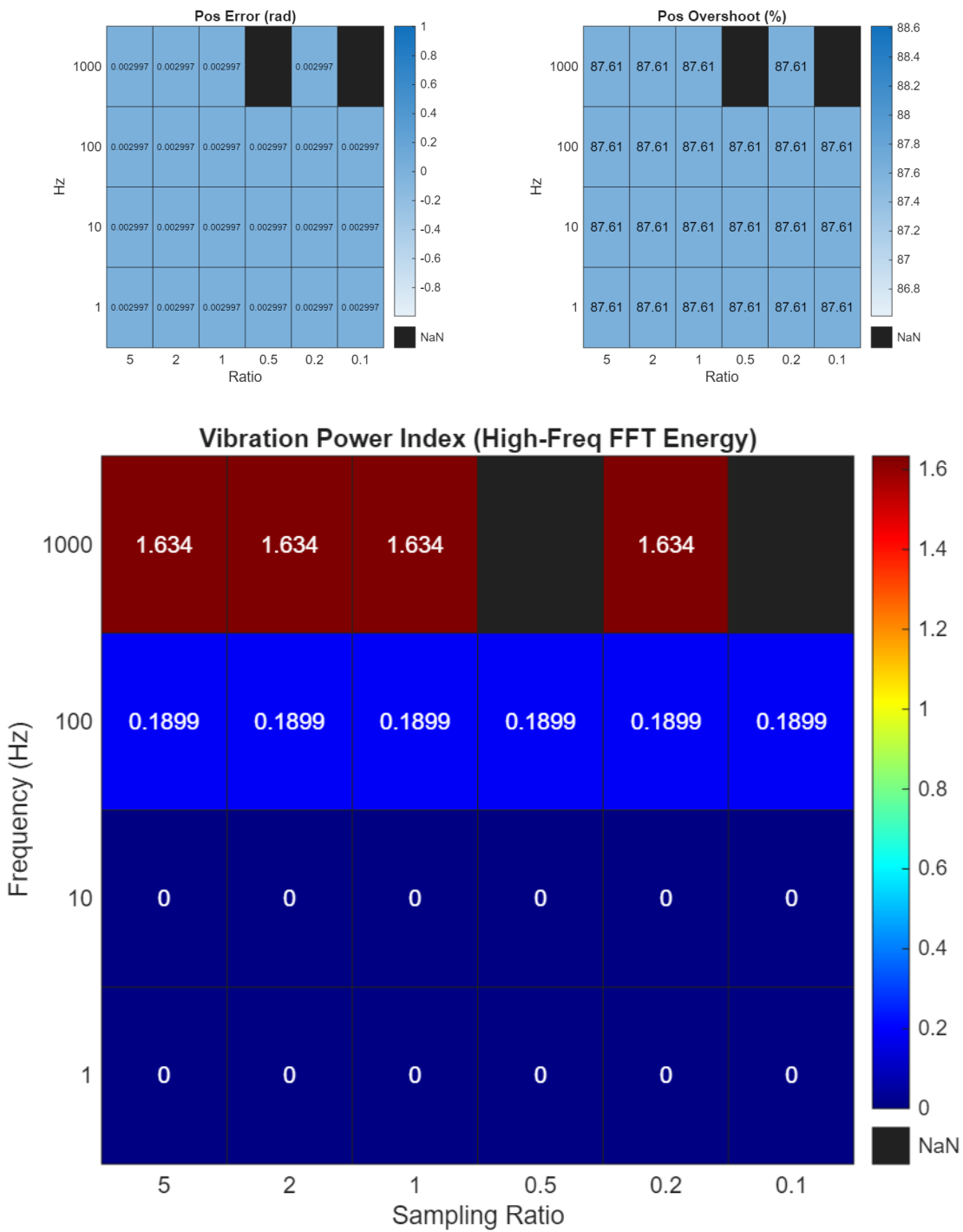
2. นำค่า $P.O.$ [%] และ Absolute Tracking Error มาเปรียบเทียบผลการทดลอง เพื่อวิเคราะห์ผลต่อในส่วนผลการทดลอง
3. นำค่า Absolute Tracing Error มาวิเคราะห์ FFT เพื่อดูความถี่ของระบบเมื่อ ความถี่ของ Outer Loop มากกว่า Inner Loop

ผลการทดลอง

1. ไม่มี Saturation Block และ ไม่มี Transition Block

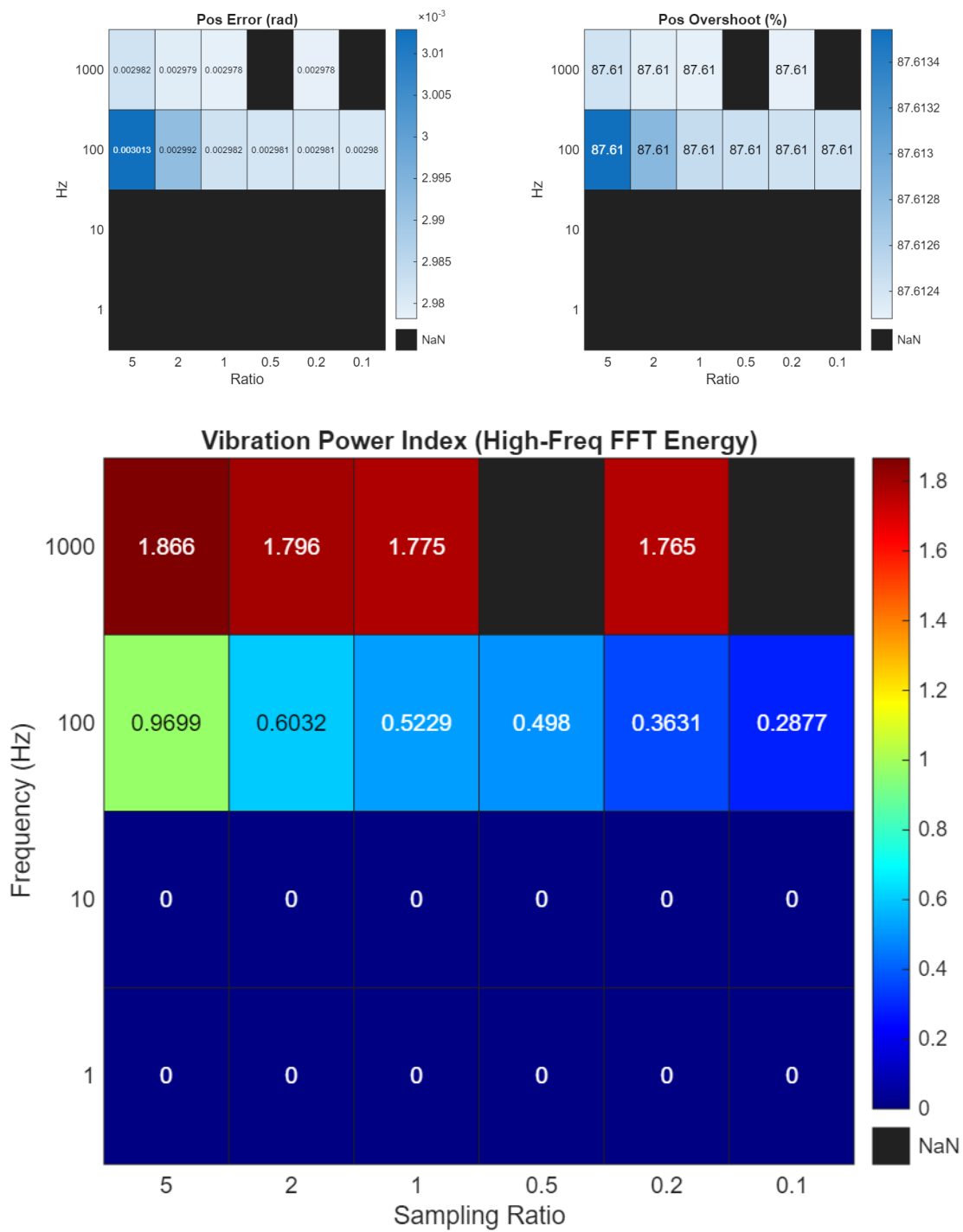


2. ไม่มี Saturation Block และ ไม่มี Transition Block



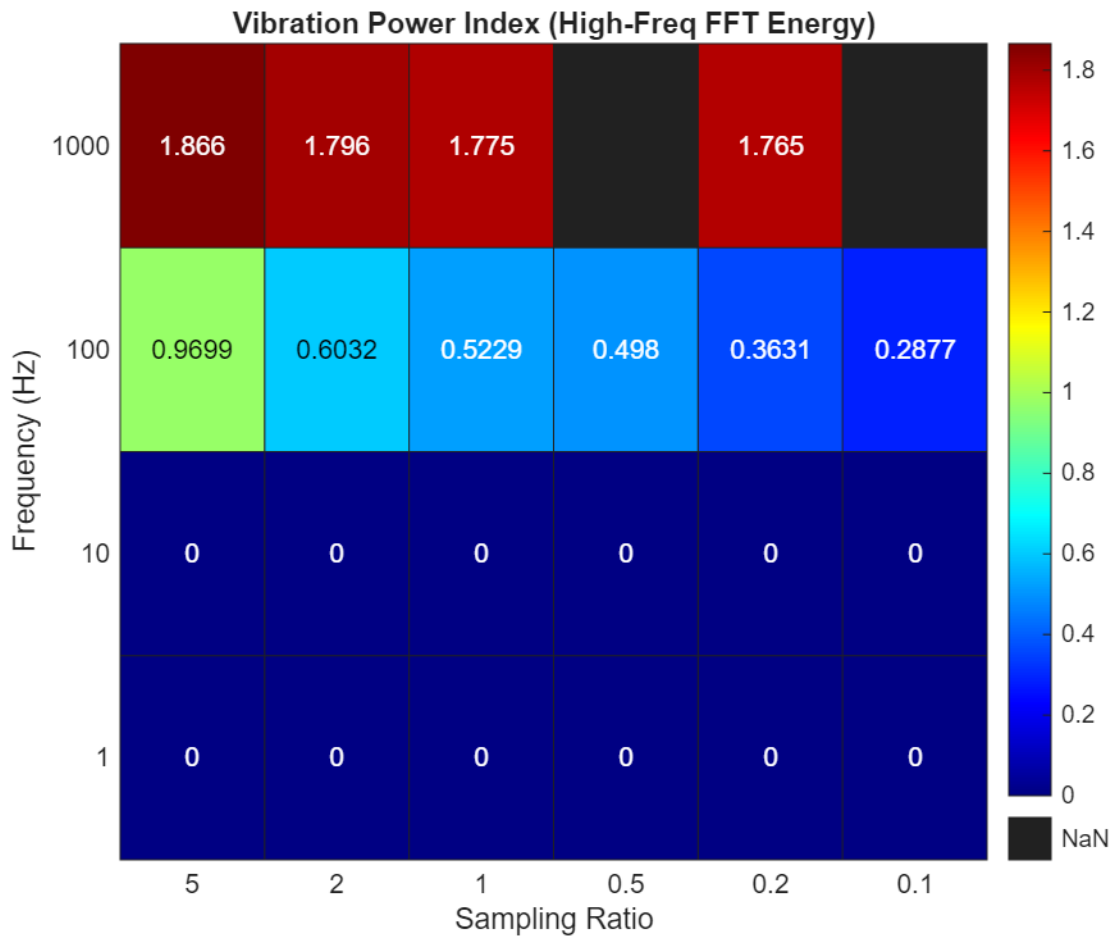
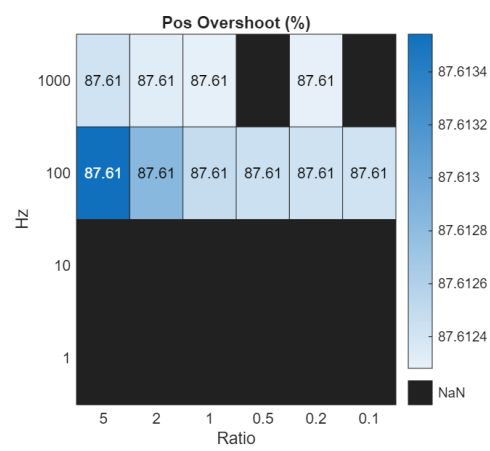
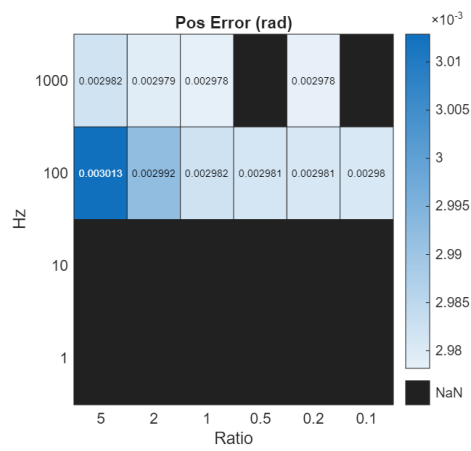
3. ไม่มี Saturation Block และ มี Transition Block

ไม่มี Saturation Block และ Transition Block



ไม่มี Saturation Block และ มี Transition Block

มี Saturation Block และ มี Transition Block



อ้างอิง

<https://youtu.be/tbgV6caAVcs?si=HjJr6t63y-ypBjUu>

<https://instrumentationtools.com/cascade-control-principle/>

Labs 2 Tuning Requirements Section

Requirement 1

1. Design Requirements

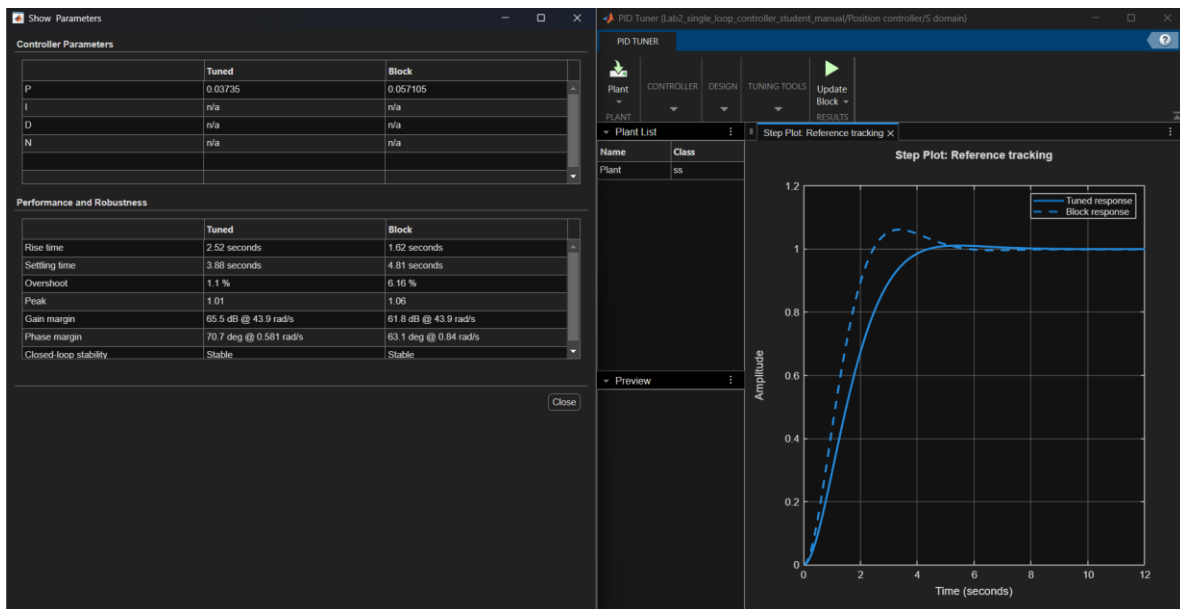
Requirement	Value	Unit
Overshoot Percentage	≤ 10	%
Peak Time	≤ 3	seconds
Voltage Range	$[-12, 12]$	volts

2. Preferred Controller Configuration

a. P Controller

3. Tools for Initial Guess

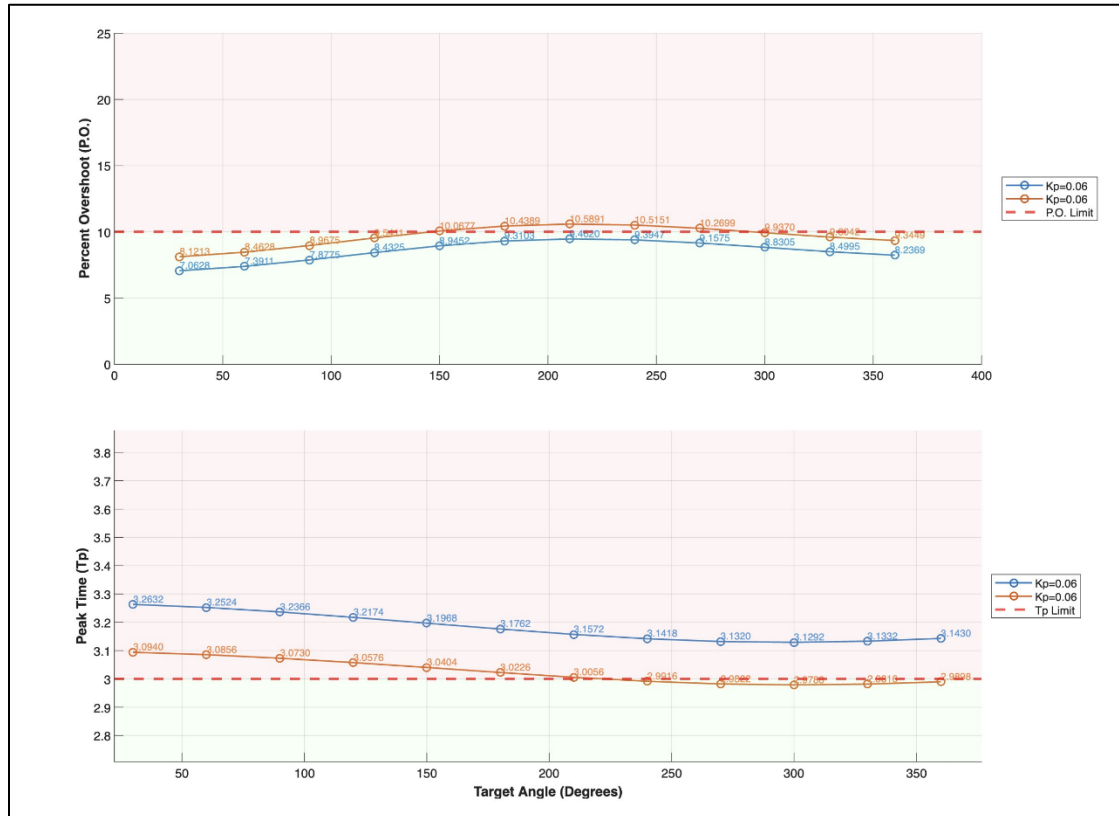
Tools	K_p	K_I	K_D
Hand Calculate	0.06	0	0
PID Tuner	0.03735	0	0



4. Fine Tuning

Tools	K_p	K_I	K_D
Fine Tune	0.064	0	0

Performance Validation



Requirement 2

1. Design Requirements

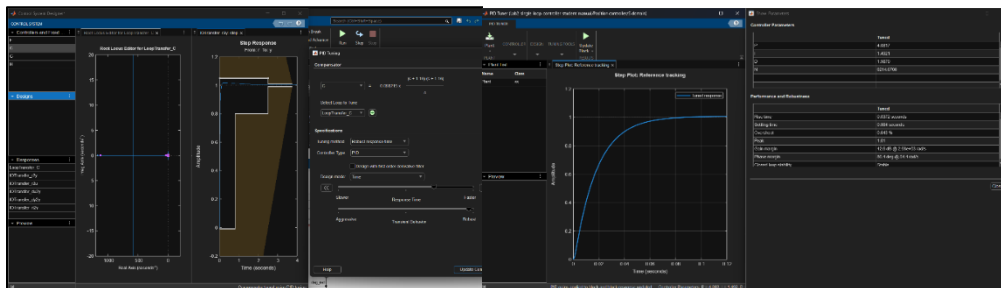
Requirement	Value	Unit
Overshoot Percentage	≤ 5	%
Peak Time	≤ 2.5	seconds
Absolute Steady State Error	$\leq 4 \times 10^{-3}$	rad

2. Preferred Controller Configuration

a. PID Controller

3. Tools for Initial Guess

Tools	K_p	K_I	K_D
PID Tuner	4.081686363763	1.492070692694	1.987930288962
Controller Design Toolbox	720	940	410
Controller Design Toolbox	0.1585	0.09207	0.06821



4. Fine Tuning

Tools	K_p	K_I	K_D
Fine Tune	0.6541	0.001	0.2334

The figure displays three stacked plots showing the performance of four different PID controller configurations (Kp, Ki, Kd) across a range of target angles (0 to 400 degrees). The configurations are:

- Kp=0.49, Ki=0.000, Kd=0.2291 (Blue circles)
- Kp=0.55, Ki=0.000, Kd=0.2434 (Orange circles)
- Kp=0.60, Ki=0.000, Kd=0.2534 (Yellow circles)
- Kp=0.65, Ki=0.000, Kd=0.2334 (Purple circles)

The plots show:

- Percent Overshoot (P.O.):** The top plot shows P.O. values for each configuration, all of which are significantly below the P.O. Limit (red dashed line at approximately 5.5%).
- Settling Time (Ts):** The middle plot shows Ts values for each configuration, all of which are below the Ts Limit (red dashed line at approximately 2.5s). A callout box indicates X 400 and Y 2.5.
- Steady State Error (Ess):** The bottom plot shows Ess values for each configuration, all of which are within the Ess limits (green shaded region between approximately -0.008 and 0.004).

Requirement 3

1. Design Requirements

Requirement	Value	Unit
Overshoot %(Velocity)	≤ 2	%
Absolute Steady State Error (Velocity)	≤ 0.02	rad/s
Overshoot %(Position)	≤ 2	%
Absolute Steady State Error (Position)	$\leq 4 \times 10^{-3}$	rad

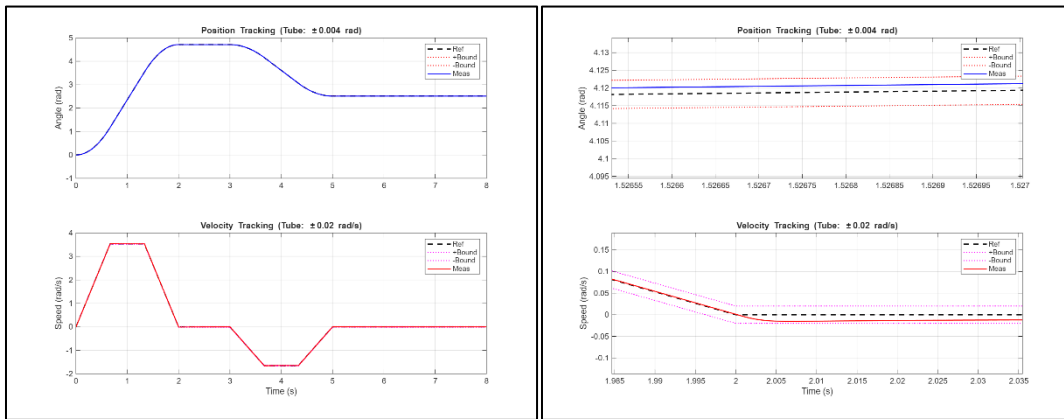
2. Preferred Controller Configuration

a. Cascade Controller

3. Fine Tuning Procedure

Tools	Position			Velocity		
	K_p	K_I	K_D	K_p	K_I	K_D
Fine Tune	5.50	0.05	0.10	8.20	1.20	0.001

4. Performance Validation



ภาคผนวก ก ผลการคำนวณอื่น ๆ

การทดลองที่ 1

คุณสมบัติ	ไม่มี Disturbance Feed Forward	มี Disturbance Feed Forward
Equation of Motion	$T_m = (J_m + m_p L^2)\ddot{\theta} + b_m \dot{\theta} + m_p g L \theta$	$T_m = (J_m + m_p L^2)\ddot{\theta} + b_m \dot{\theta}$
Transfer Function	$\frac{\theta(s)}{V(s)} = \frac{K_t K_p}{(L_m s + R_m)[(J_m + m_p L^2)s^2 + b_m s + m_p g L] + K_e K_t s + K_t K_p}$	$\frac{\theta(s)}{V(s)} = \frac{K_t K_p}{(L_m s + R_m)[(J_m + m_p L^2)s^2 + b_m s] + K_e K_t s + K_t K_p}$
Transfer Function Characteristics	$\Delta s = (L_m s + R_m)[(J_m + m_p L^2)s^2 + b_m s + m_p g L] + K_e K_t s + K_t K_p$	$\Delta s = (L_m s + R_m)[(J_m + m_p L^2)s^2 + b_m s] + K_e K_t s + K_t K_p$
Routh Array	$a_3 = L_m(J_m + m_p L^2)$ $a_2 = L_m b_m + R_m(m_p L^2 + J_m)$ $a_1 = L_m m_p g L + R_m b_m + K_e K_t$ $a_0 = R_m m_p g L + K_p K_t$	$a_3 = L_m(J_m + m_p L^2)$ $a_2 = L_m b_m + R_m(m_p L^2 + J_m)$ $a_1 = R_m b_m + K_e K_t$ $a_0 = K_p K_t$
Kp Range (Variable)	$\frac{-(R_m m_p g L)}{K_t} < K_p < \frac{a_2 a_1 - a_3 (R_m m_p g L)}{a_3 K_t}$	$0 < K_p < \frac{a_2 a_1}{a_3 K_t}$
Kp Range (Value)	$-3.21 < K_p < 67.2139$	$0 < K_p < 67.2139$